

Instructivo completo: Sistema de Ferretería con ASP.NET Core MVC (.NET 10), Supabase y Render

Este documento explica **todo el proyecto FerreteriaApp** desde cero, como si nunca hubiéramos programado en ASP.NET. El sistema implementa el **diagrama de clases UML de la ferretería** (Usuario/Rol con herencia Empleado y Cliente, Categoría, Producto, Proveedor, Inventario, Venta, DetalleVenta, FormaPago, Compra, DetalleCompra y las enumeraciones EstadoVenta y EstadoCompra). Primero se explican los conceptos, después se construye el sistema paso a paso (archivo por archivo, con el código completo y su explicación), y al final se muestra cómo ejecutarlo, probarlo y publicarlo en internet con Render. Incluye íntegro el contenido del **README.md** del proyecto (secciones 1, 2, 3, 4, 5, 6, 7, 8, 9 y 10 y el Anexo A).

Índice

- **Parte 1 — Conceptos: qué es cada cosa**
 - **Parte 2 — Preparar el entorno y crear el proyecto**
 - **Parte 3 — Modelos (las clases que representan los datos)**
 - **Parte 4 — Acceso a datos: DbContext y datos iniciales**
 - **Parte 5 — Helpers (utilidades)**
 - **Parte 6 — Program.cs: el arranque de la aplicación**
 - **Parte 7 — Controladores (la lógica)**
 - **Parte 8 — Vistas (las pantallas)**
 - **Parte 9 — Migraciones y base de datos**
 - **Parte 10 — Ejecutar y probar el sistema**
 - **Parte 11 — Publicar en internet: GitHub + Render + Supabase**
 - **Parte 12 — Personalización**
 - **Parte 13 — Solución de problemas frecuentes**
 - **Parte 14 — Preguntas típicas para defender el proyecto**
 - **Anexo A — README.md del proyecto**
-

Parte 1 — Conceptos: qué es cada cosa

1.1 ¿Qué vamos a construir?

El sistema implementa el **diagrama de clases UML de la ferretería** (ver Parte 3): usuarios con roles y herencia Empleado/Cliente, categorías, proveedores, productos con inventario, ventas con detalle y forma de pago, y compras a proveedores con detalle.

Público (sin iniciar sesión)

- Página de inicio con categorías y productos destacados.
- Catálogo con **búsqueda, filtro por categoría y paginación**.
- Detalle de producto con productos relacionados.
- **Carrito** (se guarda en la sesión; no hace falta cuenta para armarlo).
- Registro (crea un **Cliente**) e inicio de sesión.

Cliente (rol **Cliente**)

- **Finalizar compra** eligiendo la forma de pago: se genera una **Venta** en estado *Pendiente* con sus **DetalleVenta** y se descuenta el stock (todo en una transacción).
- **Mis compras**: historial con estados y detalle. Puede cancelar mientras esté *Pendiente*.
- Editar su perfil y cambiar la contraseña.

Empleado (rol **Empleado**)

- **Panel general** con ventas, compras, stock bajo y productos más vendidos.
- **Registrar venta de mostrador** (elige cliente, forma de pago y productos): queda *Completada* a nombre del empleado.
- **Gestionar ventas**: filtrar por estado y cambiarlo (*Pendiente* → *Completada* → *Devuelta* / *Cancelada*). Cancelar o devolver repone el stock.
- **Registrar compras a proveedores** con detalle; al completarlas ingresa la mercadería (sube el stock y actualiza el precio de compra). No se permite cancelar una compra si dejaría stock negativo.
- **Inventario**: stock actual, mínimo, última actualización, filtro "bajo mínimo" y ajuste manual.
- Crear y editar **productos** (con subida de imagen).

Administrador (rol **Admin**)

- Todo lo del empleado, más: **categorías, proveedores, formas de pago**, eliminar productos, y **usuarios**: alta/edición de empleados, rol Admin/Empleado, activar/desactivar cuentas y tipo de cliente (Regular/Mayorista).

Validaciones (según el diagrama)

- Email único y con formato válido. Contraseña de **mínimo 8 caracteres con mayúsculas, minúsculas y números**. Teléfono solo números de **8 a 15 dígitos**. Estado no nulo.
- Producto: nombre obligatorio (máx. 100), precios de venta y compra > 0, stock y stock mínimo ≥ 0, categoría y proveedor obligatorios.
- Venta/Compra: fecha no nula, total > 0, cantidad > 0, precio unitario > 0, no vender sin

stock, no dejar stock negativo.

Datos iniciales (seed automático)

La primera vez que arranca, la app crea sola los roles, el administrador, un vendedor de ejemplo, 4 formas de pago, 8 categorías, 6 proveedores, los **43 productos** con sus imágenes y su **inventario** inicial.

1.2 ¿Qué es .NET y C#?

- **C#** (se pronuncia "si sharp") es el lenguaje de programación que usamos. Es parecido a Java: tiene clases, tipos, **if**, **for**, etc.
- **.NET** es la plataforma de Microsoft que ejecuta programas en C#. Es gratuita, de código abierto y funciona en Windows, Linux y Mac. Usamos la versión **10**, que es **LTS** (*Long Term Support*: con soporte por varios años).
- El comando **dotnet** (por ejemplo **dotnet run**) es la herramienta de consola de .NET: crea proyectos, compila, ejecuta, instala paquetes.
- Un **paquete NuGet** es una librería que se descarga de internet y se agrega al proyecto (como **npm** en JavaScript o **pip** en Python). Se declaran en el archivo **.csproj**.

1.3 ¿Qué es ASP.NET Core MVC?

ASP.NET Core es la parte de .NET para hacer aplicaciones web. **MVC** es la forma de organizar el código en tres tipos de piezas:

Pieza	Qué es	En nuestro proyecto
Modelo (Model)	Clases de C# que representan los datos: un producto, un pedido, un usuario.	Models/Product.cs , Models/Order.cs , etc.
Vista (View)	Archivos .cshtml que generan el HTML que ve el usuario. Mezclan HTML con C# usando el lenguaje Razor .	Views/Products/Index.cshtml , etc.
Controlador (Controller)	Clases que reciben la petición del navegador, buscan o guardan datos y eligen qué vista mostrar.	Controllers/ProductsController.cs , etc.

La idea es separar responsabilidades: el controlador **decide**, el modelo **representa** y la vista **muestra**.

1.4 ¿Cómo viaja una petición? (el recorrido completo)

Cuando alguien entra a <https://tu-sitio.com/Products/Details/3> pasa esto:

1. El navegador pide la URL **/Products/Details/3**
|
2. El ROUTING de ASP.NET la interpreta con el patrón
"**{controller}/{action}/{id?}**"
→ controlador = **ProductsController**, acción = **Details**, id = **3**
|
3. Se ejecuta el método **ProductsController.Details(3)**
|
4. El controlador le pide a ENTITY FRAMEWORK el producto con Id 3
context.Products.FirstOrDefaultAsync(p => p.Id == 3)

5. Entity Framework lo traduce a SQL y lo manda a POSTGRESQL (Supabase)

`SELECT ... FROM "Products" WHERE "Id" = 3`

6. El controlador recibe el objeto Product y lo pasa a la vista

`return View(product);`

7. RAZOR procesa Views/Products/Details.cshtml y genera HTML

8. El navegador recibe el HTML y lo muestra

Cuando se envía un formulario (por ejemplo "Agregar al carrito") es igual, pero la petición es **POST** en lugar de **GET** y trae datos (el id del producto, la cantidad) que el controlador recibe como parámetros del método.

1.5 ¿Qué es Entity Framework Core?

Es un **ORM** (*Object-Relational Mapper*): una librería que permite trabajar con la base de datos usando objetos de C# en lugar de escribir SQL a mano.

- Cada **clase modelo** se convierte en una **tabla** (`Product` → tabla `Products`).
- Cada **propiedad** se convierte en una **columna** (`Name` → columna `Name`).
- El **DbContext** (`ApplicationDbContext`) es la clase que representa la conexión a la base de datos. Tiene un `DbSet<T>` por cada tabla: `context.Products`, `context.Orders`, etc.
- Las consultas se escriben con **LINQ**, por ejemplo: `context.Products.Where(p => p.Stock > 0).OrderBy(p => p.Name).ToListAsync()` y EF Core las traduce a SQL.
- Las **migraciones** son archivos generados automáticamente que describen cómo crear o modificar las tablas. Así nunca escribimos `CREATE TABLE` a mano.
- **Npgsql** es el "driver" que permite a EF Core hablar con **PostgreSQL** en particular.

1.6 ¿Qué es ASP.NET Core Identity?

Es el sistema de **usuarios, contraseñas y roles** que ya viene con ASP.NET Core. Nos da:

- Tablas de usuarios y roles (`AspNetUsers`, `AspNetRoles`, `AspNetUserRoles`, etc.) creadas solas.
- Contraseñas guardadas **hasheadas** (nunca en texto plano).
- Clases listas para usar: `UserManager` (crear usuarios, asignar roles) y `SignInManager` (iniciar y cerrar sesión).
- Autenticación por **cookie**: al iniciar sesión el navegador recibe una cookie cifrada y en cada petición siguiente ASP.NET sabe quién es el usuario.
- **Roles**: usamos tres, `Admin`, `Empleado` y `Cliente`. Con `[Authorize(Roles = "Admin")]` o `[Authorize(Roles = "Admin,Empleado")]` se protege una acción para que solo la usen esos roles.
- En este proyecto Identity se configura con **claves numéricas** (`int`) y las tablas se renombran (`Usuarios`, `Roles`, `UsuarioRoles`...) para que coincidan con el diagrama.

1.7 ¿Qué es Supabase?

Supabase es un servicio en la nube que ofrece una base de datos **PostgreSQL** gratuita (además de otras cosas que no usamos: autenticación propia, storage, etc.). En este proyecto Supabase es **únicamente la base de datos**: nuestra app se conecta a ella con un *connection string* y Entity

Framework crea las tablas ahí.

PostgreSQL es una base de datos relacional (tablas, filas, columnas, claves foráneas), gratuita y muy usada.

1.8 ¿Qué es Render y Docker?

- **Render** es un hosting: toma nuestro código desde GitHub, lo compila y lo deja corriendo en internet con una URL pública (<https://ferreteria-app.onrender.com>). Tiene plan gratuito.
- **Docker** es una forma de empaquetar la aplicación con todo lo que necesita (el runtime de .NET, los archivos publicados) en una "imagen" que corre igual en cualquier servidor. Render lee nuestro archivo **Dockerfile** (una receta de cómo armar esa imagen) y lo ejecuta.

1.9 Tecnologías usadas

Componente	Tecnología
Framework web	ASP.NET Core MVC — .NET 10 (LTS)
Acceso a datos	Entity Framework Core 10 + Npgsql
Base de datos	PostgreSQL (Supabase)
Autenticación	ASP.NET Core Identity con claves int (cookies, roles Admin , Empleado , Cliente)
Carrito	Sesión de ASP.NET Core (no es una tabla)
Front-end	Razor Views + Bootstrap 5 + Bootstrap Icons
Deploy	Docker en Render

1.10 Glosario rápido

Término	Significado
Acción	Un método público de un controlador que responde a una URL.
async / await	Forma de escribir código que espera operaciones lentas (consultas a la BD) sin bloquear el servidor. Los métodos devuelven Task<...> .
Razor	Sintaxis de las vistas: HTML con C# intercalado usando @ .
Tag Helper	Atributos especiales de Razor (asp-for , asp-action , asp-route-id) que generan el HTML correcto de formularios y enlaces.
_Layout	Plantilla común de todas las páginas (barra de navegación, pie, CSS y JS).
Vista parcial	Un pedazo de vista reutilizable (por ejemplo la tarjeta de producto). Su nombre empieza con _ .
ViewModel	Clase que existe solo para transportar datos entre un formulario y el controlador (no es una tabla). Ejemplo: LoginViewModel .
ViewBag	"Bolsa" dinámica para pasar datos extra del controlador a la vista (por ejemplo la lista de categorías).
TempData	Datos que sobreviven a una redirección; se usan para los mensajes "Producto creado correctamente".
ModelState	Resultado de la validación del formulario. ModelState.IsValid dice si los datos cumplen las reglas ([Required] , [Range] ...).

Término	Significado
Data Annotations	Atributos entre corchetes sobre las propiedades ([Required] , [MaxLength(100)]) que definen reglas de validación y de la tabla.
Connection string	Texto con host, usuario, contraseña y nombre de la base de datos para conectarse.
Migración	Archivo generado por EF Core que crea/modifica tablas.
Seed	Datos iniciales que se cargan automáticamente (roles, admin, productos).
Rol	Etiqueta de un usuario que define permisos (Admin , User).
Transacción	Grupo de operaciones en la BD que se ejecutan todas o ninguna.
Bootstrap	Librería de CSS que da estilo a botones, tablas, tarjetas y hace el diseño responsive.
Herencia TPT	<i>Table Per Type</i> : forma de guardar una herencia de clases en tablas: una tabla para la clase base (Usuarios) y una por cada clase hija (Empleados , Cientes) cuya clave es también clave foránea a la base.
Sesión	Memoria del servidor asociada al navegador por una cookie. Aquí guarda el carrito.
Enum	Tipo con valores fijos con nombre (EstadoVenta.Pendiente). En la base se guarda como número.
Token antifalsificación	Código oculto en cada formulario que impide que otro sitio envíe formularios en nombre del usuario ([ValidateAntiForgeryToken]).

Parte 2 — Preparar el entorno y crear el proyecto

2.1 Prerrequisitos

- **.NET 10 SDK (LTS)**: descargar de <https://dotnet.microsoft.com/download>. Verificar con `dotnet --version` (debe empezar con 10.).
- **Editor**: Visual Studio 2022 (17.14 o superior) con la carga de trabajo "ASP.NET y desarrollo web", o Visual Studio Code con la extensión *C# Dev Kit*.
- Una cuenta y un proyecto creado en **Supabase**: <https://supabase.com>.
- Una cuenta en **GitHub** (<https://github.com>) y otra en **Render** (<https://render.com>) para el deploy.
- (Opcional) La herramienta de EF Core para crear migraciones nuevas:

```
dotnet tool install --global dotnet-ef
```

2.2 Crear el proyecto en Supabase y obtener el connection string

1. Entrar a <https://supabase.com>, crear una cuenta y un **New project**. Elegir un nombre, una contraseña para la base de datos (**anotarla**, se usa en el connection string) y la región más cercana.
2. Esperar 1-2 minutos a que el proyecto se cree.
3. Ir a **Project Settings → Database** (o al botón **Connect** de arriba).
4. En *Connection string* elegir la pestaña **Session pooler** (no *Direct connection*: el pooler tiene salida IPv4 y evita el error de timeout por IPv6).
5. Copiar host, puerto (normalmente 5432) y usuario (formato `postgres.xxxxxxxxxxxx`).
6. Armar la cadena así (reemplazar lo que está entre < >):

```
Host=aws-0-<region>.pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.<tu-project-ref>;Password=<TU_PASSWORD>;SSL Mode=Require;Trust Server Certificate=true
```

Si olvidaste la contraseña de la base de datos, se puede resetear en *Project Settings → Database → Reset database password*.

Si más adelante se despliega en un hosting *serverless* con muchas conexiones cortas (por ejemplo Azure Functions), se usa el **Transaction pooler** (puerto 6543). Para un sitio MVC normal, el Session pooler funciona perfecto y es compatible con las migraciones de EF Core.

2.3 Crear el proyecto ASP.NET Core MVC

Abrir una terminal en la carpeta donde se quiera el proyecto y ejecutar:

```
dotnet new mvc -n FerreteriaApp -f net10.0
cd FerreteriaApp
dotnet tool install --global dotnet-ef
```

`dotnet new mvc` genera un proyecto base con:

- **Program.cs**: punto de entrada.

- `Controllers/HomeController.cs` y `Views/Home/*.cshtml`: una página de ejemplo.
- `Views/Shared/_Layout.cshtml`: plantilla común.
- `wwwroot/`: archivos estáticos (CSS, JS, imágenes). Ya trae Bootstrap y jQuery en `wwwroot/lib`.
- `appsettings.json`: configuración.
- `FerreteriaApp.csproj`: el archivo del proyecto (paquetes, versión de .NET).

2.4 Instalar los paquetes NuGet

Solo se necesitan cinco paquetes:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
dotnet restore
```

Paquete	Para qué sirve
Microsoft.EntityFrameworkCore	El ORM (Entity Framework Core).
Npgsql.EntityFrameworkCore.PostgreSQL	Conector de EF Core para PostgreSQL.
Microsoft.EntityFrameworkCore.Design	Necesario para generar migraciones con <code>dotnet ef</code> .
Microsoft.EntityFrameworkCore.Tools	Herramientas de EF para Visual Studio (Package Manager Console).
Microsoft.AspNetCore.Identity.EntityFrameworkCore	Identity (usuarios y roles) guardando en EF Core.

No se necesita el paquete "Supabase", ni JwtBearer, ni Newtonsoft.Json: la autenticación se hace con cookies de Identity (ya incluida en ASP.NET Core) y todo el acceso a datos pasa por EF Core.

2.4.1 FerreteriaApp.csproj completo

Este es el archivo del proyecto con las versiones fijadas (si hay errores de compilación del tipo "no existe en el namespace", reemplazar todo el contenido por este y correr `dotnet restore`):

FerreteriaApp.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="10.0.12" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.12" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.12">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.12">
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
  </ItemGroup>
</Project>
```

```

    <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
  </PackageReference>
  <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.3" />
</ItemGroup>

</Project>

```

Si hubo que reemplazar el .csproj, desde la carpeta del proyecto:

```

rm -rf bin obj      # en Windows PowerShell: Remove-Item -Recurse -Force bin, obj
dotnet restore
dotnet build

```

2.5 appsettings.json y appsettings.Development.json

appsettings.json es el archivo de configuración. Se lee con `builder.Configuration["Seccion:Clave"]`.

appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Host=aws-0-
<region>.pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.<tu-project-
ref>;Password=<TU_PASSWORD>;SSL Mode=Require;Trust Server Certificate=true"
  },
  "Tienda": {
    "Nombre": "Ferretería Central",
    "Moneda": "Bs",
    "ZonaHoraria": "America/La_Paz"
  },
  "Admin": {
    "Email": "juan@gmail.com",
    "Password": "Juan1234",
    "Nombre": "Juan",
    "Apellido": "Pérez"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Warning"
    }
  },
  "AllowedHosts": "*"
}

```

Sección	Qué contiene
ConnectionStrings:DefaultConn ection	El connection string de la base de datos (paso 2.2).
Tienda	Nombre de la ferretería, símbolo de la moneda y zona horaria para mostrar fechas.
Admin	Email, contraseña, nombre y apellido del administrador que se crea automáticamente la primera vez (es un Empleado con rol Admin).
Logging	Qué tan detallados son los mensajes en la consola.
AllowedHosts	Desde qué dominios se acepta tráfico (* = todos).

Dónde poner la contraseña real de Supabase. Hay dos opciones:

- **Opción A (simple):** reemplazar el valor de `DefaultConnection` en `appsettings.json`.
- **Opción B (recomendada):** crear el archivo `appsettings.Development.json` con solo el connection string. Este archivo está en `.gitignore`, así la contraseña **no se sube a GitHub**. Cuando la app corre en modo *Development* (que es lo que hace `dotnet run`), los valores de este archivo reemplazan a los de `appsettings.json`.

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=aws-0-
<region>.pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.<tu-project-
ref>;Password=<TU_PASSWORD>;SSL Mode=Require;Trust Server Certificate=true"
  }
}
```

En Render (producción) el connection string se carga como **variable de entorno** (ver Parte 11).

2.6 Estructura final del proyecto

```
FerreteriaApp/
├── Controllers/
│   ├── HomeController.cs      Inicio y página de error
│   ├── AccountController.cs   Login, registro (Cliente), logout, perfil, cambiar contraseña
│   ├── ProductosController.cs Catálogo público + CRUD de productos (personal)
│   ├── CategoríasController.cs CRUD de categorías (Admin)
│   ├── ProveedoresController.cs CRUD de proveedores (Admin)
│   ├── FormasPagoController.cs CRUD de formas de pago (Admin)
│   ├── CarritoController.cs   Carrito en sesión (agregar, actualizar, quitar, vaciar)
│   ├── VentasController.cs    Checkout web, mis compras, venta de mostrador, gestión de estados
│   ├── ComprasController.cs   Compras a proveedores y su ingreso al stock
│   ├── InventarioController.cs Inventario y ajustes manuales
│   └── AdminController.cs     Panel general, usuarios y empleados
├── Models/
│   ├── Usuario.cs             Base de Identity (int) + nombre, apellido, estado, fechaCreacion
│   ├── Rol.cs                 Rol de Identity (int) + descripción
│   ├── Empleado.cs / Cliente.cs Heredan de Usuario (tablas Empleados y Clientes)
│   ├── Categoria.cs, Proveedor.cs, Producto.cs, Inventario.cs, FormaPago.cs
│   ├── Venta.cs (+ enum EstadoVenta), DetalleVenta.cs
│   ├── Compra.cs (+ enum EstadoCompra), DetalleCompra.cs
│   ├── CarritoItem.cs         Ítem del carrito (en sesión)
│   └── *ViewModel.cs          Formularios (login, registro, perfil, checkout, venta, compra, empleado)
├── Data/
│   ├── ApplicationDbContext.cs Único DbContext (Identity + tablas de la ferretería)
│   └── DbInitializer.cs        Seed: roles, admin, vendedor, formas de pago, categorías, proveedores, productos,
inventario
├── Helpers/
│   ├── Formato.cs             Precios, fechas y colores de estado
│   ├── CarritoSesion.cs       Leer/guardar el carrito en la sesión
│   ├── UsuarioExtensions.cs   Id numérico del usuario logueado, EsPersonal()
│   └── SpanishIdentityErrorDescriber.cs Mensajes de Identity en español
├── Migrations/
│   └── Migrations de EF Core (ya generadas)
├── Views/
│   └── Vistas Razor (Home, Account, Productos, Categorías, Proveedores, FormasPago, Carrito,
Ventas, Compras, Inventario, Admin, Shared)
├── wwwroot/
│   ├── css/site.css           Estilos propios
│   └── images/productos/       Imágenes de los productos
├── appsettings.json           Configuración (connection string, nombre de la tienda, admin)
├── Program.cs
├── Dockerfile                 Para el deploy en Render
└── render.yaml                Blueprint de Render
```

CRUD significa *Create, Read, Update, Delete*: crear, leer, actualizar y borrar registros.

Parte 3 — Modelos (las clases que representan los datos)

3.0 Del diagrama de clases a las tablas

El punto de partida es el **diagrama de clases UML** de la ferretería. Cada clase del diagrama se convierte en una clase de C# en *Models/* y, gracias a Entity Framework, en una tabla de PostgreSQL:

Clase del diagrama	Tabla	Notas
Usuario	Usuarios	Es el usuario de ASP.NET Core Identity con clave int (IdUsuario). Identity aporta Email , PasswordHash (la contraseña nunca se guarda en texto plano) y las columnas de seguridad. Se agregan Nombre , Apellido , Estado , FechaCreacion .
Rol	Roles	Rol de Identity (IdRol , Nombre , Descripcion). La relación Usuario–Rol queda en UsuarioRoles ; la app asigna un rol por usuario (Admin , Empleado o Cliente).
Empleado hereda Usuario	Empleados	Herencia TPT (una tabla por tipo): IdEmpleado es PK y a la vez FK a Usuarios.IdUsuario . Campos: Cargo , Telefono , Direccion .
Cliente hereda Usuario	Clientes	Igual que Empleado: IdCliente = PK/FK. Campos: Telefono , Direccion , TipoCliente .
Categoria	Categorias	IdCategoria , Nombre , Descripcion , Estado .
Proveedor	Proveedores	IdProveedor , Nombre , Ruc , Telefono , Direccion , Estado .
Producto	Productos	IdProducto , Nombre , Descripcion , PrecioVenta , PrecioCompra , Stock , StockMinimo , Estado , IdCategoria , IdProveedor + ImagenUrl (campo extra para la foto).
Inventario	Inventarios	IdInventario , IdProducto , StockActual , StockMinimo , UltimaActualizacion . Uno por producto, se actualiza en cada venta, compra o ajuste.
FormaPago	FormasPago	IdFormaPago , Nombre , Descripcion , Estado .
Venta	Ventas	IdVenta , Fecha , IdCliente , IdEmpleado (nulo hasta que un empleado completa una venta web), IdFormaPago , Total , Estado (enum EstadoVenta), Observaciones .
DetalleVenta	DetalleVentas	IdDetalleVenta , IdVenta , IdProducto , Cantidad , PrecioUnitario , Subtotal .
Compra	Compras	IdCompra , Fecha , IdProveedor , IdEmpleado , Total , Estado (enum EstadoCompra), Observaciones .
DetalleCompra	DetalleCompras	IdDetalleCompra , IdCompra , IdProducto , Cantidad , PrecioUnitario , Subtotal .
EstadoVenta, EstadoCompra	(enum)	Se guardan como número: Pendiente=0, Completada=1, Cancelada=2, Devuelta=3.

Decisiones de diseño que conviene poder explicar:

- **Identity para Usuario y Rol.** En lugar de programar a mano el login y el guardado de contraseñas (inseguro), **Usuario** hereda de **IdentityUser<int>** y **Rol** de **IdentityRole<int>**. Se renombran las tablas de Identity (**AspNetUsers** → **Usuarios**, etc.) para respetar el diagrama.
- **Herencia Empleado/Cliente con TPT.** La flecha de herencia del diagrama se traduce con *Table Per Type*: **Usuarios** tiene los datos comunes y **Empleados** / **Clientes** los propios, unidas por

la clave. En C# se escribe simplemente `class Empleado : Usuario`.

- **Relaciones.** *Venta "se paga con" FormaPago* se implementa como una forma de pago por venta (`IdFormaPago`); *DetalleVenta "es de" Producto* mediante `IdProducto`; *Producto "tiene" Inventario* con un registro por producto.
- **El carrito no es una tabla.** No está en el diagrama, así que se guarda en la **sesión** del servidor y solo al confirmar se convierte en `Venta + DetalleVenta`.
- **Métodos del diagrama.** Están implementados como métodos de las clases (`Producto.ActualizarStock`, `Venta.CalcularTotal`, `Venta.AgregarDetalle`, `Compra.AgregarDetalle`, `DetalleVenta.CalcularSubtotal`, `Inventario.VerificarStock...`). Los que dependen del usuario logueado (`login`, `realizarCompra`, `registrarVenta`, `gestionarInventario...`) viven en los controladores.

3.1 Data Annotations: las reglas de validación

Antes de ver cada archivo, conviene entender los atributos entre corchetes que van sobre las propiedades:

Atributo	Qué hace
[Key]	Marca la propiedad como clave primaria (<code>IdProducto</code>). EF la crea autoincremental.
[Required]	El campo es obligatorio. Si el formulario viene vacío, <code>ModelState.IsValid</code> es <code>false</code> .
[MaxLength(100)]	Largo máximo del texto (también define el tamaño de la columna).
[Range(0.01, 999999.99)]	Valor mínimo y máximo permitidos. <code>[Range(1, int.MaxValue)]</code> sirve para un combo obligatorio (0 = sin elegir).
[RegularExpression(@"^\d{8,15}\$")]	El texto debe cumplir una expresión regular (acá: solo dígitos, entre 8 y 15).
[Display(Name = "Nombre")]	Texto de la etiqueta que se muestra en los formularios (<code><label asp-for=...></code>).
[DataType(DataType.Password)]	Indica que el campo se muestre como contraseña (<code>type="password"</code>).
[EmailAddress]	Valida que el texto tenga formato de email.
[Compare("Password")]	Valida que dos campos sean iguales (contraseña y confirmación).
[StringLength(100, MinimumLength = 8)]	Largo mínimo y máximo.
[NotMapped]	La propiedad no se guarda en la base de datos (existe solo en memoria).

Así se cumplen las **validaciones del diagrama**: email con formato, teléfono de 8 a 15 dígitos, nombre de producto obligatorio de máximo 100 caracteres, precios > 0 , stock ≥ 0 , categoría y proveedor obligatorios, cantidad y precio unitario > 0 . La regla de contraseña (8 caracteres con mayúscula, minúscula y número) la aplica Identity (Parte 6), y "no vender sin stock" / "no dejar stock negativo" las aplica el código de las clases y los controladores.

Otras cosas que aparecen en los modelos:

- `string?` (con `?`) significa que la propiedad **puede ser null** (opcional).
- `DateTime.UtcNow` guarda la fecha en UTC. Después se convierte a la zona horaria de la tienda al mostrarla (`Helpers/Formato.cs`).
- Una propiedad de tipo otra clase (`public Categoria? Categoria`) junto con su Id (`IdCategoria`) es una **propiedad de navegación**: representa una clave foránea. `ICollection<Producto> Productos` es el lado "muchos" de la relación.

3.2 Models/Usuario.cs y Models/Rol.cs — usuario base y rol

Models/Usuario.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.AspNetCore.Identity;

namespace FerreteriaApp.Models
{
    // Clase base del diagrama: Usuario. Hereda de IdentityUser<int> para que Identity maneje
    // el email, la contraseña (hasheada) y los roles, con claves numéricas (idUser: int).
    // Empleado y Cliente heredan de esta clase (herencia TPT: una tabla por tipo).
    public class Usuario : IdentityUser<int>
    {
        [Required(ErrorMessage = "El nombre es obligatorio"), MaxLength(100)]
        [Display(Name = "Nombre")]
        public string Nombre { get; set; } = string.Empty;

        [Required(ErrorMessage = "El apellido es obligatorio"), MaxLength(100)]
        [Display(Name = "Apellido")]
        public string Apellido { get; set; } = string.Empty;

        // Estado no puede ser nulo: true = activo, false = desactivado (no puede iniciar sesión)
        [Display(Name = "Activo")]
        public bool Estado { get; set; } = true;

        [Display(Name = "Fecha de creación")]
        public DateTime FechaCreacion { get; set; } = DateTime.UtcNow;

        [NotMapped]
        public string NombreCompleto => $"{Nombre} {Apellido}".Trim();

        // validarDatos(): las reglas están en las Data Annotations y en las opciones de Identity
        public bool ValidarDatos() =>
            !string.IsNullOrEmpty(Nombre) && !string.IsNullOrEmpty(Apellido) && !
            string.IsNullOrEmpty(Email);
    }
}
```

Models/Rol.cs

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Identity;

namespace FerreteriaApp.Models
{
    // Rol del diagrama. Hereda de IdentityRole<int>: "Name" es el nombre del rol (columna "Nombre").
    // Roles del sistema: Admin, Empleado y Cliente.
    public class Rol : IdentityRole<int>
    {
        public Rol() { }
    }
}
```

```

    public Rol(string nombre, string? descripcion = null) : base(nombre) { Descripcion = descripcion; }

    [MaxLength(200)]
    [Display(Name = "Descripción")]
    public string? Descripcion { get; set; }
}
}

```

Estado = false **desactiva la cuenta**: el `AccountController` no la deja iniciar sesión.

3.3 Models/Empleado.cs y Models/Cliente.cs — la herencia

Models/Empleado.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Empleado hereda de Usuario (tabla Empleados, clave = IdUsuario).
    // Puede registrar ventas, registrar compras y gestionar el inventario.
    public class Empleado : Usuario
    {
        [Required(ErrorMessage = "El cargo es obligatorio"), MaxLength(50)]
        [Display(Name = "Cargo")]
        public string Cargo { get; set; } = string.Empty;

        [Required(ErrorMessage = "El teléfono es obligatorio")]
        [RegularExpression(@"^\d{8,15}$", ErrorMessage = "El teléfono debe tener solo números (8 a 15 dígitos)")]
        [Display(Name = "Teléfono")]
        public string Telefono { get; set; } = string.Empty;

        [Required(ErrorMessage = "La dirección es obligatoria"), MaxLength(200)]
        [Display(Name = "Dirección")]
        public string Direccion { get; set; } = string.Empty;
    }
}

```

Models/Cliente.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Cliente hereda de Usuario (tabla Clientes, clave = IdUsuario).
    // Es quien se registra desde la web y realiza compras (ventas para la ferretería).
    public class Cliente : Usuario
    {
        [Required(ErrorMessage = "El teléfono es obligatorio")]
        [RegularExpression(@"^\d{8,15}$", ErrorMessage = "El teléfono debe tener solo números (8 a 15 dígitos)")]
        [Display(Name = "Teléfono")]
        public string Telefono { get; set; } = string.Empty;

        [Required(ErrorMessage = "La dirección es obligatoria"), MaxLength(200)]
        [Display(Name = "Dirección")]
        public string Direccion { get; set; } = string.Empty;

        // Regular o Mayorista (lo cambia el administrador)
        [MaxLength(30)]
        [Display(Name = "Tipo de cliente")]
        public string TipoCliente { get; set; } = "Regular";
    }
}

```

```

        public ICollection<Venta> Ventas { get; set; } = new List<Venta>();
    }
}

```

class Empleado : Usuario es toda la herencia en C#. Qué tabla usa cada clase se configura en el DbContext (Parte 4).

3.4 Models/Categoria.cs y Models/Proveedor.cs

Models/Categoria.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    public class Categoria
    {
        [Key]
        public int IdCategoria { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio"), MaxLength(100)]
        [Display(Name = "Nombre")]
        public string Nombre { get; set; } = string.Empty;

        [MaxLength(300)]
        [Display(Name = "Descripción")]
        public string? Descripcion { get; set; }

        [Display(Name = "Activa")]
        public bool Estado { get; set; } = true;

        public ICollection<Producto> Productos { get; set; } = new List<Producto>();
    }
}

```

Models/Proveedor.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    public class Proveedor
    {
        [Key]
        public int IdProveedor { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio"), MaxLength(150)]
        [Display(Name = "Nombre / Razón social")]
        public string Nombre { get; set; } = string.Empty;

        [Required(ErrorMessage = "El RUC/NIT es obligatorio"), MaxLength(20)]
        [Display(Name = "RUC / NIT")]
        public string Ruc { get; set; } = string.Empty;

        [Required(ErrorMessage = "El teléfono es obligatorio")]
        [RegularExpression(@"^\d{8,15}$", ErrorMessage = "El teléfono debe tener solo números (8 a 15 dígitos)")]
        [Display(Name = "Teléfono")]
        public string Telefono { get; set; } = string.Empty;

        [Required(ErrorMessage = "La dirección es obligatoria"), MaxLength(200)]
    }
}

```

```

[Display(Name = "Dirección")]
public string Direccion { get; set; } = string.Empty;

[Display(Name = "Activo")]
public bool Estado { get; set; } = true;

public ICollection<Producto> Productos { get; set; } = new List<Producto>();
public ICollection<Compra> Compras { get; set; } = new List<Compra>();
}
}

```

3.5 Models/Producto.cs — el producto y sus métodos

Models/Producto.cs

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace FerreteriaApp.Models
{
    public class Producto
    {
        [Key]
        public int IdProducto { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio"), MaxLength(100)]
        [Display(Name = "Nombre")]
        public string Nombre { get; set; } = string.Empty;

        [Required(ErrorMessage = "La descripción es obligatoria"), MaxLength(1000)]
        [Display(Name = "Descripción")]
        public string Descripcion { get; set; } = string.Empty;

        [Required, Range(0.01, 999999.99, ErrorMessage = "El precio de venta debe ser mayor a 0")]
        [Display(Name = "Precio de venta")]
        public decimal PrecioVenta { get; set; }

        [Required, Range(0.01, 999999.99, ErrorMessage = "El precio de compra debe ser mayor a 0")]
        [Display(Name = "Precio de compra")]
        public decimal PrecioCompra { get; set; }

        [Required, Range(0, int.MaxValue, ErrorMessage = "El stock no puede ser negativo")]
        [Display(Name = "Stock")]
        public int Stock { get; set; }

        [Required, Range(0, int.MaxValue, ErrorMessage = "El stock mínimo no puede ser negativo")]
        [Display(Name = "Stock mínimo")]
        public int StockMinimo { get; set; }

        [Display(Name = "Activo (visible en el catálogo)")]
        public bool Estado { get; set; } = true;

        [Range(1, int.MaxValue, ErrorMessage = "La categoría es obligatoria")]
        [Display(Name = "Categoría")]
        public int IdCategoria { get; set; }
        public Categoria? Categoria { get; set; }

        [Range(1, int.MaxValue, ErrorMessage = "El proveedor es obligatorio")]
        [Display(Name = "Proveedor")]
        public int IdProveedor { get; set; }
        public Proveedor? Proveedor { get; set; }
    }
}

```

```
// Campo extra (no está en el diagrama): ruta de la imagen del producto
[Display(Name = "URL de imagen")]
public string? ImagenUrl { get; set; }

public ICollection<Inventario> Inventarios { get; set; } = new List<Inventario>();

// Solo para subir la imagen desde el formulario; no se guarda en la base de datos
[NotMapped]
[Display(Name = "Imagen")]
public IFormFile? ImagenArchivo { get; set; }

// ---- Métodos del diagrama ----

// actualizarStock(cantidad): suma (o resta si es negativo) unidades al stock
// y deja sincronizado el registro de Inventario.
public void ActualizarStock(int cantidad)
{
    if (Stock + cantidad < 0)
        throw new InvalidOperationException($"El stock de \"{Nombre}\" no puede quedar negativo.");

    Stock += cantidad;
    foreach (var inv in Inventarios)
        inv.ActualizarStock(cantidad);
}

// verificarStock(): true si hay stock por encima del mínimo
public bool VerificarStock() => Stock > StockMinimo;

public bool ValidarDatos() =>
    !string.IsNullOrEmpty(Nombre) && PrecioVenta > 0 && PrecioCompra > 0 &&
    Stock >= 0 && StockMinimo >= 0 && IdCategoria > 0 && IdProveedor > 0;
}
}
```

- **PrecioVenta** y **PrecioCompra** son **decimal** (no **double**): es el tipo correcto para dinero.
- **ImagenUrl** guarda la **ruta** de la imagen (/images/productos/martillo-generico.png), no la imagen en sí. **ImagenArchivo** ([NotMapped]) solo recibe el archivo subido en el formulario.
- **ActualizarStock(cantidad)** suma o resta unidades y mantiene sincronizado el **Inventario**; lanza una excepción si el stock quedaría negativo (regla del diagrama).
- **VerificarStock()** devuelve **true** si el stock está por encima del mínimo.

3.6 Models/Inventario.cs

Models/Inventario.cs

```
using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Registro de inventario de un producto: stock actual, mínimo y última actualización.
    // Se crea uno por producto y se actualiza en cada venta, compra o ajuste manual.
    public class Inventario
    {
        [Key]
        public int IdInventario { get; set; }

        [Required]
        [Display(Name = "Producto")]
        public int IdProducto { get; set; }
    }
}
```

```

public Producto? Producto { get; set; }

[Required, Range(0, int.MaxValue, ErrorMessage = "El stock no puede ser negativo")]
[Display(Name = "Stock actual")]
public int StockActual { get; set; }

[Required, Range(0, int.MaxValue, ErrorMessage = "El stock mínimo no puede ser negativo")]
[Display(Name = "Stock mínimo")]
public int StockMinimo { get; set; }

[Display(Name = "Última actualización")]
public DateTime UltimaActualizacion { get; set; } = DateTime.UtcNow;

// ---- Métodos del diagrama ----

public void ActualizarStock(int cantidad)
{
    if (StockActual + cantidad < 0)
        throw new InvalidOperationException("El stock del inventario no puede quedar negativo.");

    StockActual += cantidad;
    UltimaActualizacion = DateTime.UtcNow;
}

// true si el stock está por encima del mínimo
public bool VerificarStock() => StockActual > StockMinimo;
}
}

```

3.7 Models/FormaPago.cs

Models/FormaPago.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Catálogo de formas de pago: Efectivo, Tarjeta, Transferencia, QR...
    public class FormaPago
    {
        [Key]
        public int IdFormaPago { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio"), MaxLength(50)]
        [Display(Name = "Nombre")]
        public string Nombre { get; set; } = string.Empty;

        [MaxLength(200)]
        [Display(Name = "Descripción")]
        public string? Descripcion { get; set; }

        [Display(Name = "Activa")]
        public bool Estado { get; set; } = true;

        public ICollection<Venta> Ventas { get; set; } = new List<Venta>();

        public bool ValidarDatos() => !string.IsNullOrEmpty(Nombre);
    }
}

```

3.8 Models/Venta.cs y Models/DetalleVenta.cs — la venta

Models/Venta.cs

```
using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    public enum EstadoVenta
    {
        Pendiente = 0,
        Completada = 1,
        Cancelada = 2,
        Devuelta = 3
    }

    // Una venta de la ferretería a un cliente. Puede originarse en la web (el cliente compra
    // desde el catálogo) o registrarla un empleado en el mostrador.
    public class Venta
    {
        [Key]
        public int IdVenta { get; set; }

        [Required(ErrorMessage = "La fecha es obligatoria")]
        [Display(Name = "Fecha")]
        public DateTime Fecha { get; set; } = DateTime.UtcNow;

        [Required(ErrorMessage = "El cliente es obligatorio")]
        [Display(Name = "Cliente")]
        public int IdCliente { get; set; }
        public Cliente? Cliente { get; set; }

        // Empleado que registró o completó la venta. Es null mientras una venta web está pendiente.
        [Display(Name = "Empleado")]
        public int? IdEmpleado { get; set; }
        public Empleado? Empleado { get; set; }

        [Required(ErrorMessage = "La forma de pago es obligatoria")]
        [Display(Name = "Forma de pago")]
        public int IdFormaPago { get; set; }
        public FormaPago? FormaPago { get; set; }

        [Range(0.01, double.MaxValue, ErrorMessage = "El total debe ser mayor a 0")]
        [Display(Name = "Total")]
        public decimal Total { get; set; }

        [Display(Name = "Estado")]
        public EstadoVenta Estado { get; set; } = EstadoVenta.Pendiente;

        [MaxLength(500)]
        [Display(Name = "Observaciones")]
        public string? Observaciones { get; set; }

        public ICollection<DetalleVenta> Detalles { get; set; } = new List<DetalleVenta>();

        // ---- Métodos del diagrama ----

        public decimal CalcularTotal()
        {
            Total = Detalles.Sum(d => d.CalcularSubtotal());
            return Total;
        }

        // agregarDetalle(producto, cantidad): agrega una línea con el precio de venta actual del producto
    }
}
```

```

public DetalleVenta AgregarDetalle(Producto producto, int cantidad)
{
    if (cantidad <= 0) throw new ArgumentException("La cantidad debe ser mayor a 0");
    if (producto.Stock < cantidad)
        throw new InvalidOperationException($"\"{producto.Nombre}\": stock insuficiente (disponible: {producto.Stock}, pedido: {cantidad}).");

    var detalle = new DetalleVenta
    {
        IdProducto = producto.IdProducto,
        Producto = producto,
        Cantidad = cantidad,
        PrecioUnitario = producto.PrecioVenta
    };
    detalle.CalcularSubtotal();
    Detalles.Add(detalle);
    return detalle;
}

public bool ValidarDatos() =>
    Fecha != default && Detalles.Count > 0 && Detalles.All(d => d.Cantidad > 0 && d.PrecioUnitario > 0) &&
    CalcularTotal() > 0;

// cerrarVenta(): la marca como completada y registra qué empleado la cerró
public void CerrarVenta(int idEmpleado)
{
    Estado = EstadoVenta.Completada;
    IdEmpleado = idEmpleado;
}

// En estos estados el stock está descontado; en Cancelada/Devuelta se devolvió
public bool DescuentaStock() => Estado == EstadoVenta.Pendiente || Estado == EstadoVenta.Completada;
}
}

```

- **EstadoVenta** es un **enum** con los cuatro estados del diagrama.
- **IdEmpleado** es **int?** (opcional): una compra hecha desde la web queda sin empleado hasta que alguien del personal la completa (**CerrarVenta**).
- **AgregarDetalle(producto, cantidad)** crea la línea con el **precio de venta actual** y valida el stock. **CalcularTotal()** suma los subtotales. **DescuentaStock()** dice si en ese estado el stock está descontado (Pendiente/Completada) o devuelto (Cancelada/Devuelta).

Models/DetalleVenta.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    public class DetalleVenta
    {
        [Key]
        public int IdDetalleVenta { get; set; }

        public int IdVenta { get; set; }
        public Venta? Venta { get; set; }

        public int IdProducto { get; set; }
        public Producto? Producto { get; set; }
    }
}

```

```

[Range(1, int.MaxValue, ErrorMessage = "La cantidad debe ser mayor a 0")]
[Display(Name = "Cantidad")]
public int Cantidad { get; set; }

// Precio al momento de la venta (si el producto cambia de precio, el historial no cambia)
[Range(0.01, double.MaxValue, ErrorMessage = "El precio unitario debe ser mayor a 0")]
[Display(Name = "Precio unitario")]
public decimal PrecioUnitario { get; set; }

[Display(Name = "Subtotal")]
public decimal Subtotal { get; set; }

public decimal CalcularSubtotal()
{
    Subtotal = PrecioUnitario * Cantidad;
    return Subtotal;
}
}
}

```

Se copian `PrecioUnitario` y `Subtotal` **en el momento de la venta**. Si después cambia el precio del producto, la venta vieja sigue mostrando lo que se cobró realmente.

3.9 Models/Compra.cs y Models/DetalleCompra.cs — la compra a proveedor

Models/Compra.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    public enum EstadoCompra
    {
        Pendiente = 0,
        Completada = 1,
        Cancelada = 2
    }

    // Compra de mercadería a un proveedor, registrada por un empleado.
    // Al completarse, el stock de los productos aumenta.
    public class Compra
    {
        [Key]
        public int IdCompra { get; set; }

        [Required(ErrorMessage = "La fecha es obligatoria")]
        [Display(Name = "Fecha")]
        public DateTime Fecha { get; set; } = DateTime.UtcNow;

        [Required(ErrorMessage = "El proveedor es obligatorio")]
        [Display(Name = "Proveedor")]
        public int IdProveedor { get; set; }
        public Proveedor? Proveedor { get; set; }

        [Required]
        [Display(Name = "Empleado")]
        public int IdEmpleado { get; set; }
        public Empleado? Empleado { get; set; }

        [Range(0.01, double.MaxValue, ErrorMessage = "El total debe ser mayor a 0")]
        [Display(Name = "Total")]

```

```

public decimal Total { get; set; }

[Display(Name = "Estado")]
public EstadoCompra Estado { get; set; } = EstadoCompra.Pendiente;

[MaxLength(500)]
[Display(Name = "Observaciones")]
public string? Observaciones { get; set; }

public ICollection<DetalleCompra> Detalles { get; set; } = new List<DetalleCompra>();

// ---- Métodos del diagrama ----

public decimal CalcularTotal()
{
    Total = Detalles.Sum(d => d.CalcularSubtotal());
    return Total;
}

// agregarDetalle(producto, cantidad): agrega una línea al precio de compra indicado
public DetalleCompra AgregarDetalle(Producto producto, int cantidad, decimal precioUnitario)
{
    if (cantidad <= 0) throw new ArgumentException("La cantidad debe ser mayor a 0");
    if (precioUnitario <= 0) throw new ArgumentException("El precio unitario debe ser mayor a 0");

    var detalle = new DetalleCompra
    {
        IdProducto = producto.IdProducto,
        Producto = producto,
        Cantidad = cantidad,
        PrecioUnitario = precioUnitario
    };
    detalle.CalcularSubtotal();
    Detalles.Add(detalle);
    return detalle;
}

public bool ValidarDatos() =>
    Fecha != default && Detalles.Count > 0 && Detalles.All(d => d.Cantidad > 0 && d.PrecioUnitario > 0) &&
    CalcularTotal() > 0;
}
}

```

Models/DetalleCompra.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    public class DetalleCompra
    {
        [Key]
        public int IdDetalleCompra { get; set; }

        public int IdCompra { get; set; }
        public Compra? Compra { get; set; }

        public int IdProducto { get; set; }
        public Producto? Producto { get; set; }

        [Range(1, int.MaxValue, ErrorMessage = "La cantidad debe ser mayor a 0")]
        [Display(Name = "Cantidad")]
    }
}

```

```

public int Cantidad { get; set; }

[Range(0.01, double.MaxValue, ErrorMessage = "El precio unitario debe ser mayor a 0")]
[Display(Name = "Precio unitario")]
public decimal PrecioUnitario { get; set; }

[Display(Name = "Subtotal")]
public decimal Subtotal { get; set; }

public decimal CalcularSubtotal()
{
    Subtotal = PrecioUnitario * Cantidad;
    return Subtotal;
}
}

```

3.10 ViewModels: clases solo para formularios

Un **ViewModel** no es una tabla: agrupa exactamente los campos de un formulario con sus validaciones.

LoginViewModel.cs y **RegistroViewModel.cs**

Models/LoginViewModel.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    public class LoginViewModel
    {
        [Required(ErrorMessage = "El email es obligatorio"), EmailAddress(ErrorMessage = "Email inválido")]
        [Display(Name = "Email")]
        public string Email { get; set; } = string.Empty;

        [Required(ErrorMessage = "La contraseña es obligatoria"), DataType(DataType.Password)]
        [Display(Name = "Contraseña")]
        public string Password { get; set; } = string.Empty;

        [Display(Name = "Recordarme")]
        public bool RememberMe { get; set; }
    }
}

```

Models/RegistroViewModel.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Formulario de registro público: crea un Cliente
    public class RegistroViewModel
    {
        [Required(ErrorMessage = "El nombre es obligatorio"), MaxLength(100), Display(Name = "Nombre")]
        public string Nombre { get; set; } = string.Empty;

        [Required(ErrorMessage = "El apellido es obligatorio"), MaxLength(100), Display(Name = "Apellido")]
        public string Apellido { get; set; } = string.Empty;

        [Required(ErrorMessage = "El email es obligatorio"), EmailAddress(ErrorMessage = "Email inválido"),

```

```

Display(Name = "Email")]
public string Email { get; set; } = string.Empty;

// Password mínimo 8 caracteres, con mayúsculas, minúsculas y números (las reglas exactas
// las aplica Identity según Program.cs; acá se valida el largo en el navegador)
[Required(ErrorMessage = "La contraseña es obligatoria")]
[StringLength(100, MinimumLength = 8, ErrorMessage = "La contraseña debe tener al menos 8 caracteres")]
[DataType(DataType.Password)]
[Display(Name = "Contraseña")]
public string Password { get; set; } = string.Empty;

[DataType(DataType.Password)]
[Compare("Password", ErrorMessage = "Las contraseñas no coinciden")]
[Display(Name = "Confirmar contraseña")]
public string ConfirmPassword { get; set; } = string.Empty;

[Required(ErrorMessage = "El teléfono es obligatorio")]
[RegularExpression(@"^\d{8,15}$", ErrorMessage = "El teléfono debe tener solo números (8 a 15 dígitos)")]
[Display(Name = "Teléfono")]
public string Telefono { get; set; } = string.Empty;

[Required(ErrorMessage = "La dirección es obligatoria"), MaxLength(200)]
[Display(Name = "Dirección")]
public string Direccion { get; set; } = string.Empty;
}
}

```

PerfilViewModel.cs y CambiarPasswordViewModel.cs

Models/PerfilViewModel.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Edición del perfil (Cliente o Empleado)
    public class PerfilViewModel
    {
        [Display(Name = "Email")]
        public string Email { get; set; } = string.Empty;

        [Required(ErrorMessage = "El nombre es obligatorio"), MaxLength(100), Display(Name = "Nombre")]
        public string Nombre { get; set; } = string.Empty;

        [Required(ErrorMessage = "El apellido es obligatorio"), MaxLength(100), Display(Name = "Apellido")]
        public string Apellido { get; set; } = string.Empty;

        [Required(ErrorMessage = "El teléfono es obligatorio")]
        [RegularExpression(@"^\d{8,15}$", ErrorMessage = "El teléfono debe tener solo números (8 a 15 dígitos)")]
        [Display(Name = "Teléfono")]
        public string Telefono { get; set; } = string.Empty;

        [Required(ErrorMessage = "La dirección es obligatoria"), MaxLength(200)]
        [Display(Name = "Dirección")]
        public string Direccion { get; set; } = string.Empty;

        // Solo lectura, para mostrar
        public string TipoUsuario { get; set; } = string.Empty;
        public string? Cargo { get; set; }
        public string? TipoCliente { get; set; }
    }
}

```

```
using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Usuario.cambiarPassword(nueva)
    public class CambiarPasswordViewModel
    {
        [Required(ErrorMessage = "Ingresa tu contraseña actual"), DataType(DataType.Password)]
        [Display(Name = "Contraseña actual")]
        public string PasswordActual { get; set; } = string.Empty;

        [Required(ErrorMessage = "Ingresa la nueva contraseña")]
        [StringLength(100, MinimumLength = 8, ErrorMessage = "La contraseña debe tener al menos 8 caracteres")]
        [DataType(DataType.Password)]
        [Display(Name = "Nueva contraseña")]
        public string PasswordNueva { get; set; } = string.Empty;

        [DataType(DataType.Password)]
        [Compare("PasswordNueva", ErrorMessage = "Las contraseñas no coinciden")]
        [Display(Name = "Confirmar nueva contraseña")]
        public string ConfirmarPassword { get; set; } = string.Empty;
    }
}
```

CarritoItem.cs y CheckoutViewModel.cs

```
namespace FerreteriaApp.Models
{
    // Ítem del carrito de compras. El carrito NO es una tabla (no está en el diagrama):
    // se guarda en la sesión del servidor como JSON hasta que el cliente confirma la compra,
    // momento en el que se convierte en una Venta con sus DetalleVenta.
    public class CarritoItem
    {
        public int IdProducto { get; set; }
        public string Nombre { get; set; } = string.Empty;
        public decimal PrecioUnitario { get; set; }
        public int Cantidad { get; set; }
        public string? ImagenUrl { get; set; }

        public decimal Subtotal => PrecioUnitario * Cantidad;
    }
}
```

```
using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    public class CheckoutViewModel
    {
        // Range(1, ...) porque un combo sin elegir llega como 0 y [Required] no lo detecta en un int
        [Range(1, int.MaxValue, ErrorMessage = "Elegí una forma de pago")]
        [Display(Name = "Forma de pago")]
        public int IdFormaPago { get; set; }

        [MaxLength(500)]
    }
}
```

```

[Display(Name = "Observaciones (opcional)")]
public string? Observaciones { get; set; }

// Solo para mostrar el resumen
public List<CarritoItem> Items { get; set; } = new();
public decimal Total => Items.Sum(i => i.Subtotal);
public string DireccionEntrega { get; set; } = string.Empty;
public string Telefono { get; set; } = string.Empty;
}
}

```

RegistrarVentaViewModel.cs y RegistrarCompraViewModel.cs

Formularios con **varias filas** de productos: `Items` es una lista y cada fila llega como `Items[0].IdProducto, Items[0].Cantidad, Items[1].IdProducto...` ASP.NET arma la lista sola.

Models/RegistrarVentaViewModel.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Venta registrada por un empleado en el mostrador (Empleado.registrarVenta)
    public class RegistrarVentaViewModel
    {
        [Range(1, int.MaxValue, ErrorMessage = "Elegí el cliente")]
        [Display(Name = "Cliente")]
        public int IdCliente { get; set; }

        [Range(1, int.MaxValue, ErrorMessage = "Elegí la forma de pago")]
        [Display(Name = "Forma de pago")]
        public int IdFormaPago { get; set; }

        [MaxLength(500)]
        [Display(Name = "Observaciones")]
        public string? Observaciones { get; set; }

        public List<LineaVentaInput> Items { get; set; } = new();
    }

    public class LineaVentaInput
    {
        public int IdProducto { get; set; }

        [Range(1, int.MaxValue, ErrorMessage = "La cantidad debe ser mayor a 0")]
        public int Cantidad { get; set; } = 1;
    }
}

```

Models/RegistrarCompraViewModel.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Compra a un proveedor registrada por un empleado (Empleado.registrarCompra)
    public class RegistrarCompraViewModel
    {
        [Range(1, int.MaxValue, ErrorMessage = "Elegí el proveedor")]
        [Display(Name = "Proveedor")]
        public int IdProveedor { get; set; }
    }
}

```

```

    [MaxLength(500)]
    [Display(Name = "Observaciones")]
    public string? Observaciones { get; set; }

    public List<LineaCompraInput> Items { get; set; } = new();
}

public class LineaCompraInput
{
    public int IdProducto { get; set; }

    [Range(1, int.MaxValue, ErrorMessage = "La cantidad debe ser mayor a 0")]
    public int Cantidad { get; set; } = 1;

    [Range(0.01, double.MaxValue, ErrorMessage = "El precio unitario debe ser mayor a 0")]
    public decimal PrecioUnitario { get; set; }
}
}

```

EmpleadoViewModel.cs y ErrorViewModel.cs

Models/EmpleadoViewModel.cs

```

using System.ComponentModel.DataAnnotations;

namespace FerreteriaApp.Models
{
    // Alta y edición de empleados desde el panel de administración
    public class EmpleadoViewModel
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio"), MaxLength(100), Display(Name = "Nombre")]
        public string Nombre { get; set; } = string.Empty;

        [Required(ErrorMessage = "El apellido es obligatorio"), MaxLength(100), Display(Name = "Apellido")]
        public string Apellido { get; set; } = string.Empty;

        [Required(ErrorMessage = "El email es obligatorio"), EmailAddress(ErrorMessage = "Email inválido"),
        Display(Name = "Email")]
        public string Email { get; set; } = string.Empty;

        // Solo obligatoria al crear; al editar se deja vacía para no cambiarla
        [StringLength(100, MinimumLength = 8, ErrorMessage = "La contraseña debe tener al menos 8 caracteres")]
        [DataType(DataType.Password)]
        [Display(Name = "Contraseña")]
        public string? Password { get; set; }

        [Required(ErrorMessage = "El cargo es obligatorio"), MaxLength(50), Display(Name = "Cargo")]
        public string Cargo { get; set; } = string.Empty;

        [Required(ErrorMessage = "El teléfono es obligatorio")]
        [RegularExpression(@"^\d{8,15}$", ErrorMessage = "El teléfono debe tener solo números (8 a 15 dígitos)")]
        [Display(Name = "Teléfono")]
        public string Telefono { get; set; } = string.Empty;

        [Required(ErrorMessage = "La dirección es obligatoria"), MaxLength(200), Display(Name = "Dirección")]
        public string Direccion { get; set; } = string.Empty;

        [Required, Display(Name = "Rol")]
        public string Rol { get; set; } = "Empleado";
    }
}

```

```

[Display(Name = "Activo")]
public bool Estado { get; set; } = true;
}
}

```

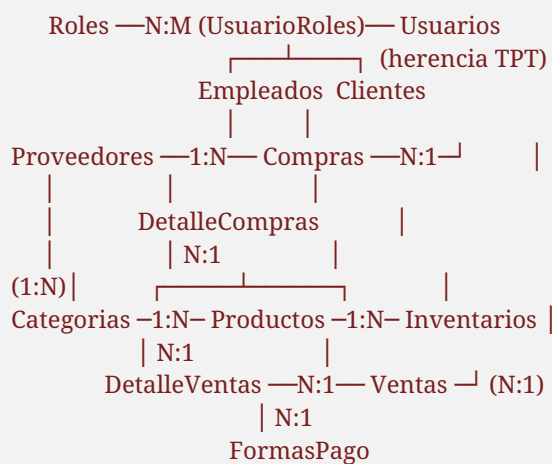
Models/ErrorViewModel.cs

```

namespace FerreteriaApp.Models
{
    public class ErrorViewModel
    {
        public string? RequestId { get; set; }
        public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
    }
}

```

3.11 Cómo quedan las tablas (relaciones)



- Un usuario es Empleado o Cliente; su rol está en **UsuarioRoles**.
- Una categoría y un proveedor tienen muchos productos (obligatorios: no se pueden borrar si tienen productos, se desactivan).
- Un producto tiene su registro de inventario (se borra en cascada con el producto).
- Una venta pertenece a un cliente, opcionalmente a un empleado y a una forma de pago; tiene muchas líneas (**DetalleVentas**, en cascada).
- Una compra pertenece a un proveedor y al empleado que la registró; tiene muchas líneas.
- **No se puede borrar un producto** que aparezca en una venta o compra (se desactiva).

Parte 4 — Acceso a datos: DbContext y datos iniciales

4.1 Data/ApplicationDbContext.cs — la conexión a la base de datos

Data/ApplicationDbContext.cs

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Models;

namespace FerreteriaApp.Data
{
    // Único DbContext: Identity (Usuarios/Roles) + tablas de la ferretería.
    // IdentityDbContext<Usuario, Rol, int>: usuarios y roles propios con clave numérica.
    public class ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : IdentityDbContext<Usuario, Rol, int>(options)
    {
        public DbSet<Empleado> Empleados { get; set; }
        public DbSet<Cliente> Clientes { get; set; }
        public DbSet<Categoria> Categorías { get; set; }
        public DbSet<Proveedor> Proveedores { get; set; }
        public DbSet<Producto> Productos { get; set; }
        public DbSet<Inventario> Inventarios { get; set; }
        public DbSet<FormaPago> FormasPago { get; set; }
        public DbSet<Venta> Ventas { get; set; }
        public DbSet<DetalleVenta> DetalleVentas { get; set; }
        public DbSet<Compra> Compras { get; set; }
        public DbSet<DetalleCompra> DetalleCompras { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // ---- Tablas de Identity con los nombres del diagrama ----
            modelBuilder.Entity<Usuario>(e =>
            {
                e.ToTable("Usuarios");
                e.Property(u => u.Id).HasColumnName("IdUsuario");
            });
            modelBuilder.Entity<Rol>(e =>
            {
                e.ToTable("Roles");
                e.Property(r => r.Id).HasColumnName("IdRol");
                e.Property(r => r.Name).HasColumnName("Nombre");
            });
            modelBuilder.Entity<IdentityUserRole<int>>().ToTable("UsuarioRoles");
            modelBuilder.Entity<IdentityUserClaim<int>>().ToTable("UsuarioClaims");
            modelBuilder.Entity<IdentityUserLogin<int>>().ToTable("UsuarioLogins");
            modelBuilder.Entity<IdentityUserToken<int>>().ToTable("UsuarioTokens");
            modelBuilder.Entity<IdentityRoleClaim<int>>().ToTable("RolClaims");

            // ---- Herencia Usuario -> Empleado / Cliente (TPT: una tabla por tipo) ----
            // La clave de Empleados y Clientes es a la vez FK a Usuarios.IdUsuario.
            modelBuilder.Entity<Empleado>().ToTable("Empleados", t => t.Property(e =>
            e.Id).HasColumnName("IdEmpleado"));
            modelBuilder.Entity<Cliente>().ToTable("Clientes", t => t.Property(c => c.Id).HasColumnName("IdCliente"));

            // ---- Precisión de importes (2 decimales) ----
            modelBuilder.Entity<Producto>().Property(p => p.PrecioVenta).HasPrecision(18, 2);
            modelBuilder.Entity<Producto>().Property(p => p.PrecioCompra).HasPrecision(18, 2);
            modelBuilder.Entity<Venta>().Property(v => v.Total).HasPrecision(18, 2);
        }
    }
}
```

```

modelBuilder.Entity<DetalleVenta>().Property(d => d.PrecioUnitario).HasPrecision(18, 2);
modelBuilder.Entity<DetalleVenta>().Property(d => d.Subtotal).HasPrecision(18, 2);
modelBuilder.Entity<Compra>().Property(c => c.Total).HasPrecision(18, 2);
modelBuilder.Entity<DetalleCompra>().Property(d => d.PrecioUnitario).HasPrecision(18, 2);
modelBuilder.Entity<DetalleCompra>().Property(d => d.Subtotal).HasPrecision(18, 2);

// --- Relaciones ---

// Categoría 1 --- 0..* Producto (contiene). No se puede borrar una categoría con productos.
modelBuilder.Entity<Producto>()
    .HasOne(p => p.Categoria).WithMany(c => c.Productos)
    .HasForeignKey(p => p.IdCategoria).OnDelete(DeleteBehavior.Restrict);

// Proveedor 1 --- 0..* Producto (suministra)
modelBuilder.Entity<Producto>()
    .HasOne(p => p.Proveedor).WithMany(pr => pr.Productos)
    .HasForeignKey(p => p.IdProveedor).OnDelete(DeleteBehavior.Restrict);

// Producto 1 --- 1..* Inventario (tiene). Si se borra el producto, se borra su inventario.
modelBuilder.Entity<Inventario>()
    .HasOne(i => i.Producto).WithMany(p => p.Inventarios)
    .HasForeignKey(i => i.IdProducto).OnDelete(DeleteBehavior.Cascade);

// Cliente 1 --- 0..* Venta (realiza)
modelBuilder.Entity<Venta>()
    .HasOne(v => v.Cliente).WithMany(c => c.Ventas)
    .HasForeignKey(v => v.IdCliente).OnDelete(DeleteBehavior.Restrict);

// Empleado 1 --- 0..* Venta (registra). Opcional: una venta web queda sin empleado hasta completarse.
modelBuilder.Entity<Venta>()
    .HasOne(v => vEmpleado).WithMany()
    .HasForeignKey(v => v.IdEmpleado).OnDelete(DeleteBehavior.Restrict);

// FormaPago 1 --- 0..* Venta (se paga con)
modelBuilder.Entity<Venta>()
    .HasOne(v => v.FormaPago).WithMany(f => f.Ventas)
    .HasForeignKey(v => v.IdFormaPago).OnDelete(DeleteBehavior.Restrict);

// Venta 1 --- 1..* DetalleVenta (tiene). Los detalles se borran con la venta.
modelBuilder.Entity<DetalleVenta>()
    .HasOne(d => d.Venta).WithMany(v => v.Detalles)
    .HasForeignKey(d => d.IdVenta).OnDelete(DeleteBehavior.Cascade);

// DetalleVenta 0..* --- 1 Producto (es de). No se puede borrar un producto vendido.
modelBuilder.Entity<DetalleVenta>()
    .HasOne(d => d.Producto).WithMany()
    .HasForeignKey(d => d.IdProducto).OnDelete(DeleteBehavior.Restrict);

// Proveedor 1 --- 0..* Compra
modelBuilder.Entity<Compra>()
    .HasOne(c => c.Proveedor).WithMany(p => p.Compras)
    .HasForeignKey(c => c.IdProveedor).OnDelete(DeleteBehavior.Restrict);

// Empleado 1 --- 0..* Compra (la realiza)
modelBuilder.Entity<Compra>()
    .HasOne(c => cEmpleado).WithMany()
    .HasForeignKey(c => c.IdEmpleado).OnDelete(DeleteBehavior.Restrict);

// Compra 1 --- 1..* DetalleCompra (tiene)
modelBuilder.Entity<DetalleCompra>()
    .HasOne(d => d.Compra).WithMany(c => c.Detalles)
    .HasForeignKey(d => d.IdCompra).OnDelete(DeleteBehavior.Cascade);

```

```

// DetalleCompra 0..* --- 1 Producto
modelBuilder.Entity<DetalleCompra>()
    .HasOne(d => d.Producto).WithMany()
    .HasForeignKey(d => d.IdProducto).OnDelete(DeleteBehavior.Restrict);
}
}
}

```

Explicación por bloques:

- `IdentityDbContext<Usuario, Rol, int>`: el mismo contexto maneja las tablas de Identity (con nuestras clases y clave `int`) y las de la ferretería.
- **Tablas de Identity renombradas**: `ToTable("Usuarios")`, `HasColumnName("IdUsuario")`, etc. Identity sigue funcionando igual; solo cambian los nombres en PostgreSQL.
- **Herencia TPT**: `Entity<Empleado>().ToTable("Empleados", t => t.Property(e => e.Id).HasColumnName("IdEmpleado"))` le dice a EF que los empleados van en su propia tabla, con la clave llamada `IdEmpleado`. Al haber tablas distintas para la base y las hijas, EF aplica automáticamente *Table Per Type*.
- `HasPrecision(18, 2)`: los importes se guardan con 2 decimales.
- **Relaciones y borrado**:
- **Restrict**: la base de datos **impide** borrar el registro padre si tiene hijos (categoría o proveedor con productos, producto vendido, cliente con ventas...). El código lo maneja desactivando.
- **Cascade**: al borrar el padre se borran los hijos (venta → detalles, producto → inventario).

4.2 Data/DbInitializer.cs — datos iniciales (seed)

Se ejecuta cada vez que arranca la app, pero solo crea lo que falta:

1. Los roles `Admin`, `Empleado` y `Cliente`.
2. El administrador (un `Empleado` con cargo "Administrador" y rol `Admin`), con el email y contraseña de `appsettings.json` → `Admin`.
3. Un vendedor de ejemplo (`vendedor@ferreteria.com` / `Vendedor123`) con rol `Empleado`.
4. Las formas de pago (Efectivo, Tarjeta, Transferencia, QR).
5. Las 8 categorías, los 6 proveedores, los 43 productos y un registro de **inventario** por producto, **solo si la tabla de productos está vacía**.

Data/DbInitializer.cs

```

using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Models;

namespace FerreteriaApp.Data
{
    // Carga los datos iniciales la primera vez que arranca la app:
    // roles, administrador, un empleado de ejemplo, formas de pago, categorías, proveedores,
    // productos e inventario. Si ya hay datos, no hace nada (se puede reiniciar sin problema).
    public static class DbInitializer
    {
        public const string RolAdmin = "Admin";
    }
}

```

```

public const string RolEmpleado = "Empleado";
public const string RolCliente = "Cliente";

public static async Task SeedAsync(IServiceProvider services, IConfiguration config)
{
    var context = services.GetRequiredService<ApplicationDbContext>();
    var userManager = services.GetRequiredService<UserManager<Usuario>>();
    var roleManager = services.GetRequiredService<RoleManager<Rol>>();

    // 1. Roles
    var roles = new[]
    {
        new Rol(RolAdmin, "Administrador del sistema: acceso total"),
        new Rol(RolEmpleado, "Personal de la ferretería: ventas, compras e inventario"),
        new Rol(RolCliente, "Cliente registrado: compra desde el catálogo"),
    };
    foreach (var rol in roles)
    {
        if (!await roleManager.RoleExistsAsync(rol.Name!))
            await roleManager.CreateAsync(rol);
    }

    // 2. Administrador (es un Empleado con rol Admin). Email y contraseña vienen de appsettings.json
    var adminEmail = config["Admin:Email"] ?? "admin@ferreteria.com";
    var adminPassword = config["Admin:Password"] ?? "Admin1234";

    if (await userManager.FindByEmailAsync(adminEmail) == null)
    {
        var admin = new Empleado
        {
            UserName = adminEmail,
            Email = adminEmail,
            EmailConfirmed = true,
            Nombre = config["Admin:Nombre"] ?? "Administrador",
            Apellido = config["Admin:Apellido"] ?? "Ferretería",
            Cargo = "Administrador",
            Telefono = "70000000",
            Direccion = "Oficina central",
            Estado = true
        };
        var result = await userManager.CreateAsync(admin, adminPassword);
        if (result.Succeeded)
            await userManager.AddToRoleAsync(admin, RolAdmin);
    }

    // 3. Un empleado de ejemplo (vendedor) para probar el rol Empleado
    const string vendedorEmail = "vendedor@ferreteria.com";
    if (await userManager.FindByEmailAsync(vendedorEmail) == null)
    {
        var vendedor = new Empleado
        {
            UserName = vendedorEmail,
            Email = vendedorEmail,
            EmailConfirmed = true,
            Nombre = "Carlos",
            Apellido = "Mamani",
            Cargo = "Vendedor",
            Telefono = "71234567",
            Direccion = "Av. Principal 123",
            Estado = true
        };
        if ((await userManager.CreateAsync(vendedor, "Vendedor123")).Succeeded)
            await userManager.AddToRoleAsync(vendedor, RolEmpleado);
    }
}

```

```

    }

    // 4. Formas de pago
    if (!await context.FormasPago.AnyAsync())
    {
        context.FormasPago.AddRange(
            new FormaPago { Nombre = "Efectivo", Descripcion = "Pago en efectivo al recibir o en mostrador" },
            new FormaPago { Nombre = "Tarjeta", Descripcion = "Tarjeta de débito o crédito" },
            new FormaPago { Nombre = "Transferencia", Descripcion = "Transferencia bancaria" },
            new FormaPago { Nombre = "QR", Descripcion = "Pago con código QR" });
        await context.SaveChangesAsync();
    }

    // 5. Categorías, proveedores, productos e inventario (solo si no hay productos)
    if (await context.Productos.AnyAsync()) return;

    var categorias = new Dictionary<string, Categoria>
    {
        ["Herramientas Eléctricas"] = new() { Nombre = "Herramientas Eléctricas", Descripcion = "Taladros, amoladoras, sierras y más herramientas con motor." },
        ["Herramientas Manuales"] = new() { Nombre = "Herramientas Manuales", Descripcion = "Llaves, alicates, martillos, sierras y herramientas de mano." },
        ["Electricidad"] = new() { Nombre = "Electricidad", Descripcion = "Instrumentos de medición, cables, linternas y accesorios eléctricos." },
        ["Plomería y Baño"] = new() { Nombre = "Plomería y Baño", Descripcion = "Grifería, sanitarios y accesorios de plomería." },
        ["Construcción"] = new() { Nombre = "Construcción", Descripcion = "Materiales y herramientas para obra." },
        ["Pintura"] = new() { Nombre = "Pintura", Descripcion = "Brochas, rodillos y espátulas para pintar." },
        ["Jardinería"] = new() { Nombre = "Jardinería", Descripcion = "Herramientas para el cuidado de jardines y áreas verdes." },
        ["Accesorios y Cerrajería"] = new() { Nombre = "Accesorios y Cerrajería", Descripcion = "Discos de corte, bisagras, manijas y repuestos." },
    };
    context.Categorias.AddRange(categorias.Values);

    var proveedores = new Dictionary<string, Proveedor>
    {
        ["Bosch"] = new() { Nombre = "Robert Bosch Bolivia S.R.L.", Ruc = "1020304050", Telefono = "22445566", Direccion = "Av. Blanco Galindo Km 4, Cochabamba" },
        ["DeWalt"] = new() { Nombre = "DeWalt Andina S.A.", Ruc = "1030405060", Telefono = "33556677", Direccion = "Parque Industrial PI-12, Santa Cruz" },
        ["Stanley"] = new() { Nombre = "Stanley Bolivia S.A.", Ruc = "1040506070", Telefono = "22667788", Direccion = "Calle Comercio 456, La Paz" },
        ["Genérico"] = new() { Nombre = "Importadora Ferretera Central S.R.L.", Ruc = "1050607080", Telefono = "44778899", Direccion = "Av. 6 de Agosto 789, Cochabamba" },
        ["Profesional"] = new() { Nombre = "Herramientas Profesionales Import S.R.L.", Ruc = "1060708090", Telefono = "33889900", Direccion = "Av. Cristo Redentor 321, Santa Cruz" },
        ["Daewoo"] = new() { Nombre = "Maquinarias Daewoo Bolivia S.A.", Ruc = "1070809010", Telefono = "22990011", Direccion = "Zona Industrial Villa Fátima, La Paz" },
    };
    context.Proveedores.AddRange(proveedores.Values);
    await context.SaveChangesAsync();

    // Función local para armar cada producto con menos repetición.
    // El precio de compra se estima como el 70% del precio de venta.
    Producto P(string nombre, decimal precioVenta, int stock, int stockMinimo, string categoria, string proveedor, string imagen, string descripcion) => new()
    {
        Nombre = nombre,
        Descripcion = descripcion,
        PrecioVenta = precioVenta,
        PrecioCompra = Math.Round(precioVenta * 0.7m, 2),
        Stock = stock,
    }

```

```

        StockMinimo = stockMinimo,
        Estado = true,
        Categoria = categorias[categoria],
        Proveedor = proveedores[proveedor],
        ImagenUrl = "/images/productos/" + imagen + ".png"
    };

    var productos = new List<Producto>
    {
        // Herramientas Eléctricas
        P("Amoladora Bosch", 350, 15, 3, "Herramientas Eléctricas", "Bosch", "amoladora-bosch",
            "Amoladora angular Bosch de 4 1/2" (115 mm) con motor de 720 W. Ideal para cortar, desbastar y pulir
metal, piedra y cerámica. Empuñadura auxiliar y protección contra re arranque."),
        P("Amoladora DeWalt", 380, 12, 3, "Herramientas Eléctricas", "DeWalt", "amoladora-dewalt",
            "Amoladora angular DeWalt de 4 1/2" con motor de 850 W y 11.000 rpm. Cuerpo compacto, interruptor
deslizante con bloqueo y protector ajustable sin herramientas."),
        P("Taladro Bosch", 340, 18, 3, "Herramientas Eléctricas", "Bosch", "taladro-bosch",
            "Taladro percutor Bosch GSB 16 RE de 750 W con mandril de 13 mm. Velocidad variable, reversible, con
función percutora para perforar concreto y mampostería."),
        P("Taladro Atornillador Bosch", 350, 10, 3, "Herramientas Eléctricas", "Bosch", "taladro-atornillador-bosch",
            "Taladro atornillador inalámbrico Bosch de 18 V con batería de litio, 2 velocidades y 20 posiciones de torque.
Incluye cargador y maletín."),
        P("Sierra Circular Bosch", 450, 8, 2, "Herramientas Eléctricas", "Bosch", "sierra-circular-bosch",
            "Sierra circular Bosch GKS 190 de 1400 W con disco de 7 1/4" (184 mm). Corte a 90° hasta 70 mm de
profundidad, con guía paralela y ajuste de bisel hasta 45°."),
        P("Motosierra Daewoo", 900, 5, 2, "Herramientas Eléctricas", "Daewoo", "motosierra-daewoo",
            "Motosierra a gasolina Daewoo con motor de 2 tiempos de 52 cc y espada de 20". Arranque fácil, freno de
cadena y lubricación automática. Ideal para poda y tala."),

        // Herramientas Manuales
        P("Martillo Genérico", 50, 40, 10, "Herramientas Manuales", "Genérico", "martillo-generico",
            "Martillo de carpintero con cabeza de acero forjado de 16 oz y uña para extraer clavos. Mango ergonómico
con recubrimiento antideslizante."),
        P("Llave Cruz Genérico", 70, 20, 5, "Herramientas Manuales", "Genérico", "llave-cruz-generico",
            "Llave cruz de acero cromado para ruedas de vehículo con medidas 17, 19, 21 y 23 mm. Brazos de 14" para
mayor palanca."),
        P("Llave Inglesa Ajustable Genérico", 90, 25, 5, "Herramientas Manuales", "Genérico", "llave-inglesa-ajustable-
generico",
            "Llave inglesa ajustable de 10" (250 mm) en acero al cromo vanadio con acabado cromado. Apertura
máxima de 30 mm, escala grabada en la mordaza."),
        P("Alicate Stanley", 30, 35, 10, "Herramientas Manuales", "Stanley", "alicate-stanley",
            "Alicate universal Stanley de 8" con mordazas de acero forjado y mangos bimaterial. Corta, sujeta y dobla
alambre con precisión."),
        P("Alicate de Presión Bosi", 55, 22, 5, "Herramientas Manuales", "Profesional", "alicate-de-presion-bosi",
            "Alicate de presión Bosi de 10" con mordaza curva. Tornillo de ajuste y palanca de liberación rápida. Acero
al cromo molibdeno."),
        P("Metro Genérico", 35, 50, 10, "Herramientas Manuales", "Genérico", "metro-generico",
            "Flexómetro de 5 metros con cinta de acero de 19 mm, freno automático, gancho magnético y clip para
cinturón. Carcasa con recubrimiento de goma."),
        P("Sierra de Madera Stanley", 70, 18, 5, "Herramientas Manuales", "Stanley", "sierra-de-madera-stanley",
            "Serrucho Stanley FatMax de 20" (500 mm) con dientes templados de triple filo para cortes rápidos en
madera. Mango ergonómico bimaterial."),
        P("Sierra Mecánica Genérico", 200, 10, 3, "Herramientas Manuales", "Genérico", "sierra-mecanica-generico",
            "Arco de sierra metálico de 12" con marco tubular reforzado y tensor de hoja. Incluye hoja bimetálica de 24
dientes por pulgada para cortar metal y PVC."),
        P("Llave de Carraca Genérico", 50, 20, 5, "Herramientas Manuales", "Genérico", "llave-de-carraca-generico",
            "Llave de carraca (ratchet) de encastre 1/2" con mecanismo de 72 dientes y palanca de reversa. Mango
antideslizante, acero al cromo vanadio."),
        P("Llave Allen Proto", 55, 15, 5, "Herramientas Manuales", "Profesional", "llave-allen-proto",
            "Juego de llaves Allen Proto de 13 piezas en medidas milimétricas (1.5 a 10 mm) con extremo de bola. Incluye
organizador plegable."),
        P("Juego de Llaves Klein Tools", 400, 6, 2, "Herramientas Manuales", "Profesional", "juego-de-llaves-klein-
tools",

```

"Juego de 14 llaves combinadas Klein Tools en medidas milimétricas (6 a 24 mm). Acero forjado con acabado cromado pulido e incluye estuche enrollable."),

P("Cúter Genérico", 12, 60, 15, "Herramientas Manuales", "Genérico", "cutter-generico",

"Cúter profesional de 18 mm con cuerpo metálico, bloqueo automático de hoja y hoja segmentada de acero SK5. Se vende por unidad."),

P("Nivel de Burbuja Johnson", 35, 25, 5, "Herramientas Manuales", "Profesional", "nivel-de-burbuja-johnson",

"Nivel de aluminio Johnson de 24\" (60 cm) con 3 burbujas (horizontal, vertical y 45°). Base fresada y tapas de goma antigolpes."),

P("Calibrador KTC", 70, 12, 3, "Herramientas Manuales", "Profesional", "calibrador-ktc",

"Calibrador vernier KTC de 150 mm en acero inoxidable con escala en mm y pulgadas. Precisión de 0.02 mm, mide exterior, interior y profundidad."),

P("Caja de Herramientas Genérico", 100, 14, 3, "Herramientas Manuales", "Genérico", "caja-de-herramientas-generico",

"Caja de herramientas plástica de 20\" con bandeja extraíble, organizadores en la tapa y cierres metálicos. Resistente a golpes y con asa reforzada."),

P("Hacha Genérico", 120, 10, 3, "Herramientas Manuales", "Genérico", "hacha-generico",

"Hacha de leñador con cabeza de acero forjado de 1.5 kg y mango de madera de 70 cm. Filo templado, ideal para cortar leña y trabajos de campo."),

// Electricidad

P("Multímetro Facom", 420, 8, 2, "Electricidad", "Profesional", "multimetro-facom",

"Multímetro digital Facom con pantalla retroiluminada. Mide voltaje AC/DC, corriente, resistencia, continuidad y capacitancia. Incluye puntas de prueba y funda protectora."),

P("Cable Extensor Genérico", 40, 30, 10, "Electricidad", "Genérico", "cable-extensor-generico",

"Extensión eléctrica de 10 metros con cable 2x1.5 mm, enchufe y toma polarizados. Soporta hasta 10 A, ideal para uso en interiores y talleres."),

P("Linterna Genérico", 25, 45, 10, "Electricidad", "Genérico", "linterna-generico",

"Linterna LED de aluminio con alcance de 200 metros y 3 modos de luz. Resistente al agua, funciona con 2 pilas D (no incluidas)."),

P("Pela Cables Genérico", 80, 15, 5, "Electricidad", "Genérico", "pela-cables-generico",

"Pelacables automático para conductores de 0.5 a 6 mm² con cortador integrado y crimpadora de terminales. Mangos ergonómicos aislados."),

// Plomería y Baño

P("Grifo Genérico", 120, 12, 3, "Plomería y Baño", "Genérico", "grifo-generico",

"Grifo monocomando para lavamanos con acabado cromado y cartucho cerámico de 35 mm. Incluye flexibles de conexión de 1/2\"."),

P("Inodoro Genérico", 620, 4, 2, "Plomería y Baño", "Genérico", "inodoro-generico",

"Inodoro de dos piezas en porcelana blanca con doble descarga (3/6 litros) y asiento de cierre lento. Salida horizontal, incluye kit de instalación."),

// Construcción

P("Pala Genérico", 90, 20, 5, "Construcción", "Genérico", "pala-generico",

"Pala punta de corazón con hoja de acero templado y mango de madera de 120 cm con empuñadura en D. Para excavación y movimiento de tierra."),

P("Carretilla Genérico", 235, 8, 2, "Construcción", "Genérico", "carretilla-generico",

"Carretilla de obra con bandeja de acero galvanizado de 65 litros, chasis reforzado y rueda neumática de 16\". Capacidad de carga de 120 kg."),

P("Calamina Genérico", 130, 40, 10, "Construcción", "Genérico", "calamina-generico",

"Calamina ondulada de acero galvanizado y prepintado, calibre 28, de 0.80 x 3.00 metros. Colores disponibles: azul, rojo y natural. Precio por unidad."),

P("Ladrillo", 5, 2000, 200, "Construcción", "Genérico", "ladrillo",

"Ladrillo de arcilla cocida de 6 huecos, medidas 24 x 12 x 6 cm. Alta resistencia para muros y tabiques. Precio por unidad, consulte por precio por millar."),

P("Escalera de Aluminio Genérico", 220, 6, 2, "Construcción", "Genérico", "escalera-de-aluminio-generico",

"Escalera tipo tijera de aluminio con 6 peldaños antideslizantes y altura de 1.80 m. Soporta hasta 120 kg, liviana y plegable."),

// Pintura

P("Brocha Genérico", 15, 60, 15, "Pintura", "Genérico", "brocha-generico",

"Brocha de 3\" con cerdas naturales y mango de madera. Para pinturas al aceite, látex y barnices. Buena retención de pintura y acabado uniforme."),

P("Rodillo Genérico", 30, 35, 10, "Pintura", "Genérico", "rodillo-generico",

```

        "Rodillo de 9\" de lana sintética con soporte metálico y mango ergonómico. Ideal para pintar paredes y techos
        con látex."),
        P("Espátula Neken", 25, 30, 10, "Pintura", "Profesional", "espatula-neken",
        "Espátula Neken de 3\" con hoja de acero inoxidable flexible y mango de madera. Para aplicar masilla, raspar
        y preparar superficies."),

        // Jardinería
        P("Podadora Genérico", 35, 15, 5, "Jardinería", "Genérico", "podadora-generico",
        "Tijera podadora de setos con hojas de acero de 9\" y mangos de madera de 50 cm. Corte limpio para
        arbustos, cercos vivos y ramas delgadas."),
        P("Paleta Jardinera Genérico", 30, 25, 5, "Jardinería", "Genérico", "paleta-jardinera-generico",
        "Pala de mano para jardín con hoja de acero inoxidable y mango de madera. Perfecta para trasplantar,
        remover tierra y sembrar en macetas."),
        P("Machete Genérico", 55, 20, 5, "Jardinería", "Genérico", "machete-generico",
        "Machete de 18\" con hoja de acero al carbono y mango de polipropileno remachado. Para desmalezar, cortar
        ramas y trabajos de campo."),

        // Accesorios y Cerrajería
        P("Disco Diamante Bosch", 20, 50, 10, "Accesorios y Cerrajería", "Bosch", "disco-diamante-bosch",
        "Disco de corte diamantado Bosch Standard de 4 1/2\" (115 mm) segmentado. Para cortar en seco concreto,
        ladrillo, piedra y cerámica."),
        P("Disco de Corte FirePower", 35, 45, 10, "Accesorios y Cerrajería", "Profesional", "disco-de-corte-firepower",
        "Disco de corte y desbaste FirePower para metal, tipo 27. Disponible en 4 1/2\", 7\" y 9\". Grano abrasivo de
        óxido de aluminio de alta durabilidad."),
        P("Bisagra Genérico", 10, 100, 20, "Accesorios y Cerrajería", "Genérico", "bisagra-generico",
        "Bisagra de acero inoxidable de 3\" x 3\" con pasador fijo y 4 orificios avellanados. Para puertas de madera y
        muebles. Precio por unidad."),
        P("Manija de Puerta Genérico", 230, 10, 3, "Accesorios y Cerrajería", "Genérico", "manija-de-puerta-generico",
        "Manija con placa para puerta principal en acabado dorado, con bocallave para cerradura de embutir.
        Incluye tornillos de fijación."),
    };

    context.Productos.AddRange(productos);

    // Un registro de inventario por producto, con el mismo stock inicial
    context.Inventarios.AddRange(productos.Select(p => new Inventario
    {
        Producto = p,
        StockActual = p.Stock,
        StockMinimo = p.StockMinimo,
        UltimaActualizacion = DateTime.UtcNow
    }));

    await context.SaveChangesAsync();
}
}
}

```

Detalles:

- `userManager.CreateAsync(admin, adminPassword)` crea el usuario **hasheando** la contraseña, y como `admin` es un `Empleado`, EF inserta en `Usuarios` y en `Empleados` (herencia TPT).
- `P(...)` es una **función local** que arma cada producto. El precio de compra se estima como el 70% del de venta.
- Cada producto se asigna a un proveedor según su marca (Bosch, DeWalt, Stanley, Daewoo, genéricos o importadora de herramientas profesionales).

Parte 5 — Helpers (utilidades)

5.1 Helpers/Formato.cs — precios, fechas y colores

Helpers/Formato.cs

```
using System.Globalization;
using FerreteriaApp.Models;

namespace FerreteriaApp.Helpers
{
    // Formato de moneda y fecha en un solo lugar.
    // Los valores se cargan desde appsettings.json ("Tienda") en Program.cs.
    public static class Formato
    {
        public static string Moneda { get; set; } = "Bs";
        public static string ZonaHoraria { get; set; } = "America/La_Paz";

        public static string Precio(decimal valor)
            => $"{{Moneda}} {{valor.ToString("N2", CultureInfo.InvariantCulture)}}";

        public static string Fecha(DateTime utc)
        {
            try
            {
                var tz = TimeZoneInfo.FindSystemTimeZoneById(ZonaHoraria);
                var local = TimeZoneInfo.ConvertTimeFromUtc(DateTime.SpecifyKind(utc, DateTimeKind.Utc), tz);
                return local.ToString("dd/MM/yyyy HH:mm");
            }
            catch (TimeZoneNotFoundException)
            {
                return utc.ToString("dd/MM/yyyy HH:mm");
            }
        }

        // Color de la etiqueta según el estado de la venta
        public static string EstadoBadge(EstadoVenta estado) => estado switch
        {
            EstadoVenta.Pendiente => "bg-warning text-dark",
            EstadoVenta.Completada => "bg-success",
            EstadoVenta.Cancelada => "bg-danger",
            EstadoVenta.Devuelta => "bg-secondary",
            _ => "bg-secondary"
        };

        // Color de la etiqueta según el estado de la compra
        public static string EstadoBadge(EstadoCompra estado) => estado switch
        {
            EstadoCompra.Pendiente => "bg-warning text-dark",
            EstadoCompra.Completada => "bg-success",
            EstadoCompra.Cancelada => "bg-danger",
            _ => "bg-secondary"
        };
    }
}
```

- **Precio(decimal)**: siempre usa punto decimal y separador de miles, con el símbolo configurado.
- **Fecha(DateTime)**: convierte la fecha UTC guardada en la base a la zona horaria de la tienda.
- **EstadoBadge(...)**: color de la etiqueta Bootstrap para cada estado de venta o de compra.

5.2 Helpers/CarritoSesion.cs — el carrito en la sesión

Helpers/CarritoSesion.cs

```
using System.Text.Json;
using FerreteriaApp.Models;

namespace FerreteriaApp.Helpers
{
    // El carrito vive en la sesión del servidor (cookie de sesión + JSON en memoria).
    // No es una tabla: al confirmar la compra se convierte en una Venta.
    public static class CarritoSesion
    {
        private const string Clave = "Carrito";

        public static List<CarritoItem> Obtener(ISession session)
        {
            var json = session.GetString(Clave);
            return string.IsNullOrEmpty(json)
                ? new List<CarritoItem>()
                : JsonSerializer.Deserialize<List<CarritoItem>>(json) ?? new List<CarritoItem>();
        }

        public static void Guardar(ISession session, List<CarritoItem> items)
            => session.SetString(Clave, JsonSerializer.Serialize(items));

        public static void Vaciar(ISession session) => session.Remove(Clave);

        public static int Cantidad(ISession session) => Obtener(session).Sum(i => i.Cantidad);
    }
}
```

La sesión solo guarda texto, así que la lista de `CarritoItem` se convierte a JSON con `JsonSerializer` al guardar y se vuelve a convertir al leer.

5.3 Helpers/UsuarioExtensions.cs — id del usuario logueado

Helpers/UsuarioExtensions.cs

```
using System.Security.Claims;

namespace FerreteriaApp.Helpers
{
    public static class UsuarioExtensions
    {
        // Id numérico del usuario logueado (Identity lo guarda como texto en el claim NameIdentifier)
        public static int GetUsuarioId(this ClaimsPrincipal user)
        {
            var valor = user.FindFirstValue(ClaimTypes.NameIdentifier);
            return int.TryParse(valor, out var id) ? id : 0;
        }

        public static bool EsPersonal(this ClaimsPrincipal user)
            => user.IsInRole("Admin") || user.IsInRole("Empleado");
    }
}
```

Identity guarda el id del usuario en la cookie como texto; `GetUsuarioId()` lo devuelve como `int` para usarlo en `IdCliente` / `IdEmpleado`. `EsPersonal()` es `true` para `Admin` y `Empleado`.

5.4 Helpers/SpanishIdentityErrorDescriber.cs — mensajes de Identity en español

Helpers/SpanishIdentityErrorDescriber.cs

```
using Microsoft.AspNetCore.Identity;

namespace FerreteriaApp.Helpers
{
    // Traduce al español los mensajes de error de Identity (registro, contraseñas, etc.)
    public class SpanishIdentityErrorDescriber : IdentityErrorDescriber
    {
        public override IdentityError DefaultError()
            => new() { Code = nameof(DefaultError), Description = "Ocurrió un error desconocido." };

        public override IdentityError DuplicateEmail(string email)
            => new() { Code = nameof(DuplicateEmail), Description = $"El email '{email}' ya está registrado." };

        public override IdentityError DuplicateUserName(string userName)
            => new() { Code = nameof(DuplicateUserName), Description = $"El usuario '{userName}' ya está registrado." };

        public override IdentityError InvalidEmail(string? email)
            => new() { Code = nameof(InvalidEmail), Description = $"El email '{email}' no es válido." };

        public override IdentityError InvalidUserName(string? userName)
            => new() { Code = nameof(InvalidUserName), Description = $"El usuario '{userName}' no es válido." };

        public override IdentityError PasswordTooShort(int length)
            => new() { Code = nameof>PasswordTooShort, Description = $"La contraseña debe tener al menos {length} caracteres." };

        public override IdentityError PasswordRequiresDigit()
            => new() { Code = nameof>PasswordRequiresDigit, Description = "La contraseña debe tener al menos un número (0-9)." };

        public override IdentityError PasswordRequiresLower()
            => new() { Code = nameof>PasswordRequiresLower, Description = "La contraseña debe tener al menos una letra minúscula (a-z)." };

        public override IdentityError PasswordRequiresUpper()
            => new() { Code = nameof>PasswordRequiresUpper, Description = "La contraseña debe tener al menos una letra mayúscula (A-Z)." };

        public override IdentityError PasswordRequiresNonAlphanumeric()
            => new() { Code = nameof>PasswordRequiresNonAlphanumeric, Description = "La contraseña debe tener al menos un carácter especial." };

        public override IdentityError PasswordRequiresUniqueChars(int uniqueChars)
            => new() { Code = nameof>PasswordRequiresUniqueChars, Description = $"La contraseña debe tener al menos {uniqueChars} caracteres distintos." };

        public override IdentityError PasswordMismatch()
            => new() { Code = nameof>PasswordMismatch, Description = "Contraseña incorrecta." };

        public override IdentityError UserAlreadyInRole(string role)
            => new() { Code = nameof(UserAlreadyInRole), Description = $"El usuario ya tiene el rol '{role}'." };

        public override IdentityError UserNotInRole(string role)
            => new() { Code = nameof(UserNotInRole), Description = $"El usuario no tiene el rol '{role}'." };

        public override IdentityError UserLockoutNotEnabled()
            => new() { Code = nameof(UserLockoutNotEnabled), Description = "El bloqueo no está habilitado para este usuario." };
    }
}
```

Parte 6 — **Program.cs**: el arranque de la aplicación

Program.cs es el primer archivo que se ejecuta. Tiene dos mitades: **antes** de `builder.Build()` se registran los servicios; **después** se arma la "tubería" de middlewares (qué pasa con cada petición, en orden).

Program.cs

```
using System.Globalization;
using Microsoft.AspNetCore.HttpOverrides;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Helpers;
using FerreteriaApp.Models;

// Cultura invariante en toda la app: los precios se leen y escriben siempre con punto decimal (99.50),
// sin importar el idioma de Windows o del servidor. La moneda se muestra con el helper Formato.
CultureInfo.DefaultThreadCurrentCulture = CultureInfo.InvariantCulture;
CultureInfo.DefaultThreadCurrentUICulture = CultureInfo.InvariantCulture;

var builder = WebApplication.CreateBuilder(args);

// Render (y otros hostings) indican por la variable de entorno PORT en qué puerto escuchar
var port = Environment.GetEnvironmentVariable("PORT");
if (!string.IsNullOrEmpty(port))
    builder.WebHost.UseUrls($"http://0.0.0.0:{port}");

builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));

// Identity con nuestras clases Usuario y Rol (claves int).
// Reglas de contraseña del diagrama: mínimo 8 caracteres, con mayúsculas, minúsculas y números.
builder.Services.AddIdentity<Usuario, Rol>(options =>
{
    options.Password.RequiredLength = 8;
    options.Password.RequireUppercase = true;
    options.Password.RequireLowercase = true;
    options.Password.RequireDigit = true;
    options.Password.RequireNonAlphanumeric = false;
    options.User.RequireUniqueEmail = true;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddErrorDescriber<SpanishIdentityErrorDescriber>() // mensajes de Identity en español
.AddDefaultTokenProviders();

builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.LogoutPath = "/Account/Logout";
    options.AccessDeniedPath = "/Account/AccessDenied";
    options.ExpireTimeSpan = TimeSpan.FromMinutes(60);
    options.SlidingExpiration = true;
});

// Sesión: acá se guarda el carrito de compras (no es una tabla de la base de datos)
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromHours(2);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
```

```

});

builder.Services.AddControllersWithViews(options =>
{
    // Mensajes genéricos del model binding en español (ej. cuando un combo llega vacío o un número es inválido)
    var m = options.ModelBindingMessageProvider;
    m.SetValueMustBeNullAccessor(_ => "Este campo es obligatorio.");
    m.SetMissingBindRequiredValueAccessor(campo => $"El campo '{campo}' es obligatorio.");
    m.SetAttemptedValueIsInvalidAccessor((valor, campo) => $"El valor '{valor}' no es válido para {campo}.");
    m.SetValueIsInvalidAccessor(valor => $"El valor '{valor}' no es válido.");
    m.SetNonPropertyAttemptedValueIsInvalidAccessor(valor => $"El valor '{valor}' no es válido.");
    m.SetUnknownValueIsInvalidAccessor(campo => $"El valor ingresado no es válido para {campo}.");
    m.SetMissingKeyOrValueAccessor() => "Falta un valor obligatorio.";
    m.SetNonPropertyValueMustBeANumberAccessor() => "Debe ser un número.";
    m.SetValueMustBeANumberAccessor(campo => $" {campo} debe ser un número.");
});

// Moneda y zona horaria de la tienda (se configuran en appsettings.json → "Tienda")
Formato.Moneda = builder.Configuration["Tienda:Moneda"] ?? "Bs";
Formato.ZonaHoraria = builder.Configuration["Tienda:ZonaHoraria"] ?? "America/La_Paz";

var app = builder.Build();

// En Render la app corre detrás de un proxy que maneja el HTTPS:
// con esto la app reconoce que la petición original fue https (cookies seguras, redirecciones correctas)
var forwardedOptions = new ForwardedHeadersOptions
{
    ForwardedHeaders = ForwardedHeaders.XForwardedFor | ForwardedHeaders.XForwardedProto
};
forwardedOptions.KnownIPNetworks.Clear(); // el proxy de Render no es localhost
forwardedOptions.KnownProxies.Clear();
app.UseForwardedHeaders(forwardedOptions);

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();

app.UseSession();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

// Al arrancar: aplica las migraciones pendientes y carga los datos iniciales
// (roles, administrador, formas de pago, categorías, proveedores, productos e inventario).
using (var scope = app.Services.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();
    await context.Database.MigrateAsync();
    await DbInitializer.SeedAsync(scope.ServiceProvider, app.Configuration);
}

app.Run();

```

Explicación por bloques:

1. **Cultura invariante:** obliga a que los números se lean y escriban con punto decimal (99.50).
 2. **Puerto:** Render indica por la variable `PORT` en qué puerto escuchar.
 3. `AddDbContext`: registra el `ApplicationDbContext` con PostgreSQL (`UseNpgsql`).
 4. `AddIdentity<Usuario, Rol>`: activa usuarios y roles con nuestras clases. Las reglas de contraseña son las del diagrama: **8 caracteres mínimo, con mayúscula, minúscula y número** (sin obligar símbolos); email único. `AddErrorDescriber` pone los mensajes en español.
 5. `ConfigureApplicationCookie`: rutas del login, logout y acceso denegado; sesión de 60 minutos.
 6. `AddSession`: activa la sesión del servidor, donde vive el carrito (2 horas de inactividad).
 7. `AddControllersWithViews` con los mensajes genéricos del model binding en español (por ejemplo cuando un combo llega vacío o un número no es válido).
 8. `UseForwardedHeaders`: para que detrás del proxy de Render la app sepa que la petición fue HTTPS.
 9. **Pipeline** (el orden importa): errores/HSTS solo en producción, `UseHttpsRedirection`, `UseStaticFiles` (sirve `wwwroot`, incluidas las imágenes subidas), `UseRouting`, `UseSession`, `UseAuthentication`, `UseAuthorization` y la ruta `{controller=Home}/{action=Index}/{id?}`.
 10. **Migraciones y seed al arrancar:** `MigrateAsync()` crea o actualiza las tablas y `DbInitializer.SeedAsync` carga los datos iniciales.
-

Parte 7 — Controladores (la lógica)

7.0 Anatomía de un controlador

```
[Authorize(Roles = "Admin,Empleado")]    // ← toda la clase requiere uno de estos roles
public class ComprasController(ApplicationDbContext context) : Controller // ← constructor primario
{
    public async Task<IActionResult> Index() // ← una ACCIÓN: responde a /Compras o /Compras/Index
    {
        var compras = await context.Compras.ToListAsync(); // ← consulta a la BD
        return View(compras); // ← renderiza Views/Compras/Index.cshtml con esos datos
    }

    [HttpPost] // ← solo responde a formularios enviados (POST)
    [ValidateAntiForgeryToken] // ← verifica el token oculto del formulario
    public async Task<IActionResult> Registrar(RegistrarCompraViewModel model) // ← los campos del form
    {
        if (!ModelState.IsValid) return View(model); // ← si falla la validación, vuelve al form
        var compra = new Compra { /* ... */ };
        context.Compras.Add(compra);
        await context.SaveChangesAsync(); // ← recién acá se escribe en la BD
        TempData["Exito"] = "Compra registrada."; // ← mensaje que se muestra en la próxima página
        return RedirectToAction(nameof(Details), new { id = compra.IdCompra });
    }
}
```

Conceptos:

- **Constructor primario:** (`ApplicationDbContext context`) recibe el `DbContext` por **inyección de dependencias** (ASP.NET lo crea y lo pasa solo).
- **`IActionResult`:** lo que devuelve una acción: `View(...)`, `RedirectToAction(...)`, `NotFound()` (404), `Forbidden()` (403), `Challenge()` (mandar al login).
- **GET vs POST:** el método sin atributo muestra la página; el que tiene `[HttpPost]` procesa el formulario. Los dos se llaman igual, C# los distingue por los parámetros.
- **Model binding:** ASP.NET arma el objeto con los campos del formulario que tengan el mismo nombre.
- `AsNoTracking()`: consultas de solo lectura más rápidas. `Include(...)`: carga la entidad relacionada (un JOIN); sin esto `venta.Cliente` sería `null`.
- `TempData["Exito"]` / `["Error"]` / `["Info"]`: mensajes que `_Alertas.cshtml` muestra como cartel verde, rojo o celeste.
- `User.GetUsuarioId()`: id numérico del usuario logueado (helper de la Parte 5).
- **Transacción** (`BeginTransactionAsync ... CommitAsync`): varias escrituras se guardan juntas o ninguna. Si una excepción sale del `using` sin `commit`, se deshace todo.

7.1 Controllers/HomeController.cs — inicio

Controllers/HomeController.cs

```
using System.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Models;
```

```

namespace FerreteriaApp.Controllers
{
    public class HomeController(ApplicationDbContext context) : Controller
    {
        // Página de inicio: categorías activas y productos destacados
        public async Task<IActionResult> Index()
        {
            ViewBag.Categorias = await context.Categorias
                .AsNoTracking()
                .Include(c => c.Productos.Where(p => p.Estado))
                .Where(c => c.Estado)
                .OrderBy(c => c.Nombre)
                .ToListAsync();

            var destacados = await context.Productos
                .AsNoTracking()
                .Include(p => p.Categoria)
                .Where(p => p.Estado && p.Categoria!.Estado && p.Stock > 0)
                .OrderByDescending(p => p.PrecioVenta)
                .Take(8)
                .ToListAsync();

            return View(destacados);
        }

        [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
        public IActionResult Error()
        {
            return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
        }
    }
}

```

7.2 Controllers/AccountController.cs — login, registro, perfil, contraseña

Controllers/AccountController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using FerreteriaApp.Data;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    // Constructor primario: userManager y signInManager quedan disponibles
    // en toda la clase sin declarar campos ni constructor explícito.
    public class AccountController(
        UserManager<Usuario> userManager,
        SignInManager<Usuario> signInManager) : Controller
    {
        [HttpGet]
        public IActionResult Login(string? returnUrl = null)
        {
            ViewBag.ReturnUrl = returnUrl;
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Login(LoginViewModel model, string? returnUrl = null)

```

```

{
    ViewBag.ReturnUrl = returnUrl;
    if (!ModelState.IsValid) return View(model);

    // Usuario.login(email, password): un usuario desactivado (Estado = false) no puede entrar
    var usuario = await userManager.FindByEmailAsync(model.Email);
    if (usuario != null && !usuario.Estado)
    {
        ModelState.AddModelError(string.Empty, "La cuenta está desactivada. Contacte al administrador.");
        return View(model);
    }

    var result = await signInManager.PasswordSignInAsync(
        model.Email, model.Password, model.RememberMe, lockoutOnFailure: false);

    if (result.Succeeded)
    {
        // Si venía de una página protegida (ej. finalizar compra), vuelve a ella
        if (!string.IsNullOrEmpty(returnUrl) && Url.IsLocalUrl(returnUrl))
            return Redirect(returnUrl);

        // El personal va al panel; los clientes al catálogo
        if (usuario is Empleado)
            return RedirectToAction("Index", "Admin");
        return RedirectToAction("Index", "Productos");
    }

    ModelState.AddModelError(string.Empty, "Credenciales inválidas");
    return View(model);
}

[HttpGet]
public IActionResult Register() => View();

// El registro público crea un Cliente con el rol "Cliente"
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Register(RegistroViewModel model)
{
    if (!ModelState.IsValid) return View(model);

    var cliente = new Cliente
    {
        UserName = model.Email,
        Email = model.Email,
        Nombre = model.Nombre,
        Apellido = model.Apellido,
        Telefono = model.Telefono,
        Direccion = model.Direccion,
        TipoCliente = "Regular",
        Estado = true
    };

    var result = await userManager.CreateAsync(cliente, model.Password);
    if (result.Succeeded)
    {
        await userManager.AddToRoleAsync(cliente, DbInitializer.RolCliente);
        await signInManager.SignInAsync(cliente, isPersistent: false);
        TempData["Exito"] = $"¡Bienvenido/a, {cliente.NombreCompleto}! Tu cuenta fue creada.";
        return RedirectToAction("Index", "Productos");
    }

    // Como el usuario es el email, Identity repite el error de duplicado; se muestra uno solo

```

```

        foreach (var error in result.Errors.Where(e => e.Code != "DuplicateUserName"))
            ModelState.AddModelError(string.Empty, error.Description);

        return View(model);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Logout()
    {
        await signInManager.SignOutAsync();
        return RedirectToAction("Index", "Home");
    }

    [HttpGet]
    public IActionResult AccessDenied() => View();

    // ----- PERFIL -----

    [Authorize]
    [HttpGet]
    public async Task<IActionResult> Perfil()
    {
        var usuario = await userManager.GetUserAsync(User);
        if (usuario == null) return NotFound();
        return View(ArmarPerfil(usuario));
    }

    // Cliente.actualizarDatos() / datos del Empleado
    [Authorize]
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Perfil(PerfilViewModel model)
    {
        var usuario = await userManager.GetUserAsync(User);
        if (usuario == null) return NotFound();

        if (!ModelState.IsValid)
        {
            var vm = ArmarPerfil(usuario);
            vm.Nombre = model.Nombre; vm.Apellido = model.Apellido; vm.Telefono = model.Telefono; vm.Direccion =
model.Direccion;
            return View(vm);
        }

        usuario.Nombre = model.Nombre;
        usuario.Apellido = model.Apellido;
        switch (usuario)
        {
            case Cliente c:
                c.Telefono = model.Telefono;
                c.Direccion = model.Direccion;
                break;
            case Empleado e:
                e.Telefono = model.Telefono;
                e.Direccion = model.Direccion;
                break;
        }
        await userManager.UpdateAsync(usuario);

        TempData["Exito"] = "Perfil actualizado correctamente.";
        return RedirectToAction(nameof(Perfil));
    }
}

```

```

// Usuario.cambiarPassword(nueva)
[Authorize]
[HttpGet]
public IActionResult CambiarPassword() => View();

[Authorize]
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CambiarPassword(CambiarPasswordViewModel model)
{
    if (!ModelState.IsValid) return View(model);

    var usuario = await userManager.GetUserAsync(User);
    if (usuario == null) return NotFound();

    var result = await userManager.ChangePasswordAsync(usuario, model.PasswordActual,
model.PasswordNueva);
    if (!result.Succeeded)
    {
        foreach (var error in result.Errors)
            ModelState.AddModelError(string.Empty, error.Description);
        return View(model);
    }

    await signInManager.RefreshSignInAsync(usuario);
    TempData["Exito"] = "Contraseña cambiada correctamente.";
    return RedirectToAction(nameof(Perfil));
}

private static PerfilViewModel ArmarPerfil(Usuario usuario) => usuario switch
{
    Cliente c => new PerfilViewModel
    {
        Email = c.Email ?? "", Nombre = c.Nombre, Apellido = c.Apellido,
        Telefono = c.Telefono, Direccion = c.Direccion,
        TipoUsuario = "Cliente", TipoCliente = c.TipoCliente
    },
    Empleado e => new PerfilViewModel
    {
        Email = e.Email ?? "", Nombre = e.Nombre, Apellido = e.Apellido,
        Telefono = e.Telefono, Direccion = e.Direccion,
        TipoUsuario = "Empleado", Cargo = e.Cargo
    },
    _ => new PerfilViewModel { Email = usuario.Email ?? "", Nombre = usuario.Nombre, Apellido = usuario.Apellido,
TipoUsuario = "Usuario" }
};
}
}

```

- **Login** (`Usuario.login`): si el usuario existe pero `Estado = false`, no lo deja entrar. `PasswordSignInAsync` compara la contraseña con el hash y crea la cookie. El personal va al panel; los clientes al catálogo.
- **Register**: el registro público crea un `Cliente` (tipo Regular) con rol `Cliente`. Identity valida la contraseña con las reglas de `Program.cs`.
- **Perfil** (`Cliente.actualizarDatos`): el `switch (usuario)` con *pattern matching* distingue si es `Cliente` o `Empleado` para guardar teléfono y dirección en la tabla que corresponde.
- **CambiarPassword** (`Usuario.cambiarPassword`): `ChangePasswordAsync` verifica la actual y aplica

las reglas a la nueva.

7.3 Controllers/ProductosController.cs — catálogo y CRUD de productos

Controllers/ProductosController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    // Catálogo público + administración de productos (Admin y Empleado).
    // "env" se usa para saber la carpeta wwwroot donde se guardan las imágenes subidas.
    public class ProductosController(ApplicationDbContext context, IWebHostEnvironment env) : Controller
    {
        private const int ProductosPorPagina = 12;
        private static readonly string[] ExtensionesPermitidas = { ".png", ".jpg", ".jpeg", ".webp" };

        // ----- CATÁLOGO (público) -----

        public async Task<IActionResult> Index(string? buscar, int? categoriaId, int pagina = 1)
        {
            var query = context.Productos
                .AsNoTracking()
                .Include(p => p.Categoria)
                .Where(p => p.Estado && p.Categoria!.Estado);

            if (!string.IsNullOrEmpty(buscar))
            {
                // ILike = LIKE sin distinguir mayúsculas/minúsculas (específico de PostgreSQL)
                var patron = $"%{buscar.Trim()}%";
                query = query.Where(p => EF.Functions.ILike(p.Nombre, patron) || EF.Functions.ILike(p.Descripcion,
                patron));
            }

            if (categoriaId.HasValue)
                query = query.Where(p => p.IdCategoria == categoriaId.Value);

            var total = await query.CountAsync();
            var totalPaginas = Math.Max(1, (int)Math.Ceiling(total / (double)ProductosPorPagina));
            pagina = Math.Clamp(pagina, 1, totalPaginas);

            var productos = await query
                .OrderBy(p => p.Nombre)
                .Skip((pagina - 1) * ProductosPorPagina)
                .Take(ProductosPorPagina)
                .ToListAsync();

            ViewBag.Categorias = await context.Categorias.AsNoTracking().Where(c => c.Estado).OrderBy(c =>
            c.Nombre).ToListAsync();
            ViewBag.Buscar = buscar;
            ViewBag.CategoriaId = categoriaId;
            ViewBag.Pagina = pagina;
            ViewBag.TotalPaginas = totalPaginas;
            ViewBag.Total = total;

            return View(productos);
        }
    }
}
```

```

public async Task<IActionResult> Details(int id)
{
    var producto = await context.Productos
        .AsNoTracking()
        .Include(p => p.Categoria)
        .Include(p => p.Proveedor)
        .FirstOrDefaultAsync(p => p.IdProducto == id);

    if (producto == null) return NotFound();

    // Un producto inactivo solo lo puede ver el personal
    if (!producto.Estado && !User.IsInRole("Admin") && !User.IsInRole("Empleado")) return NotFound();

    ViewBag.Relacionados = await context.Productos
        .AsNoTracking()
        .Where(p => p.Estado && p.IdProducto != id && p.IdCategoria == producto.IdCategoria)
        .OrderBy(p => p.Nombre)
        .Take(4)
        .ToListAsync();

    return View(producto);
}

// ----- ADMINISTRACIÓN (Admin y Empleado) -----

[Authorize(Roles = "Admin,Empleado")]
public async Task<IActionResult> Manage(string? buscar)
{
    var query = context.Productos.AsNoTracking().Include(p => p.Categoria).Include(p =>
p.Proveedor).AsQueryable();

    if (!string.IsNullOrEmpty(buscar))
        query = query.Where(p => EF.Functions.ILike(p.Nombre, $"%{buscar.Trim()}%"));

    ViewBag.Buscar = buscar;
    return View(await query.OrderBy(p => p.Nombre).ToListAsync());
}

[Authorize(Roles = "Admin,Empleado")]
public async Task<IActionResult> Create()
{
    await CargarListasAsync();
    return View(new Producto { StockMinimo = 5 });
}

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin,Empleado")]
public async Task<IActionResult> Create(Producto producto)
{
    if (producto.ImagenArchivo != null && !ImagenValida(producto.ImagenArchivo))
        ModelState.AddModelError("ImagenArchivo", "Solo se permiten imágenes PNG, JPG o WEBP de hasta 5 MB.");

    if (!ModelState.IsValid)
    {
        await CargarListasAsync(producto.IdCategoria, producto.IdProveedor);
        return View(producto);
    }

    if (producto.ImagenArchivo != null)
        producto.ImagenUrl = await GuardarImagenAsync(producto.ImagenArchivo);

    if (string.IsNullOrEmpty(producto.ImagenUrl))

```

```

        producto.ImagenUrl = "/images/default-product.png";

// Todo producto nace con su registro de inventario
producto.Inventarios.Add(new Inventario
{
    StockActual = producto.Stock,
    StockMinimo = producto.StockMinimo,
    UltimaActualizacion = DateTime.UtcNow
});

context.Productos.Add(producto);
await context.SaveChangesAsync();

TempData["Exito"] = $"Producto \"{producto.Nombre}\" creado correctamente.";
return RedirectToAction(nameof(Manage));
}

[Authorize(Roles = "Admin,Empleado")]
public async Task<IActionResult> Edit(int id)
{
    var producto = await context.Productos.FindAsync(id);
    if (producto == null) return NotFound();

    await CargarListasAsync(producto.IdCategoria, producto.IdProveedor);
    return View(producto);
}

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin,Empleado")]
public async Task<IActionResult> Edit(int id, Producto producto)
{
    if (id != producto.IdProducto) return NotFound();

    if (producto.ImagenArchivo != null && !ImagenValida(producto.ImagenArchivo))
        ModelState.AddModelError("ImagenArchivo", "Solo se permiten imágenes PNG, JPG o WEBP de hasta 5 MB.");

    if (!ModelState.IsValid)
    {
        await CargarListasAsync(producto.IdCategoria, producto.IdProveedor);
        return View(producto);
    }

    // Se carga el producto original y se copian solo los campos editables,
    // así no se pierde la imagen si no se subió una nueva.
    var original = await context.Productos.Include(p => p.Inventarios).FirstOrDefaultAsync(p => p.IdProducto == id);
    if (original == null) return NotFound();

    original.Nombre = producto.Nombre;
    original.Descripcion = producto.Descripcion;
    original.PrecioVenta = producto.PrecioVenta;
    original.PrecioCompra = producto.PrecioCompra;
    original.IdCategoria = producto.IdCategoria;
    original.IdProveedor = producto.IdProveedor;
    original.Estado = producto.Estado;
    original.StockMinimo = producto.StockMinimo;

    // Si cambió el stock a mano, se ajusta también el inventario (queda registrada la fecha)
    if (original.Stock != producto.Stock)
        original.ActualizarStock(producto.Stock - original.Stock);
    foreach (var inv in original.Inventarios)
        inv.StockMinimo = producto.StockMinimo;

```

```

        if (producto.ImagenArchivo != null)
            original.ImagenUrl = await GuardarImagenAsync(producto.ImagenArchivo);
        else if (!string.IsNullOrEmpty(producto.ImagenUrl))
            original.ImagenUrl = producto.ImagenUrl;

        await context.SaveChangesAsync();

        TempData["Exito"] = $"Producto \"{original.Nombre}\" actualizado.";
        return RedirectToAction(nameof(Manage));
    }

    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> Delete(int id)
    {
        var producto = await context.Productos.AsNoTracking().Include(p => p.Categoria).FirstOrDefaultAsync(p =>
p.IdProducto == id);
        if (producto == null) return NotFound();

        ViewBag.TieneMovimientos = await TieneMovimientosAsync(id);
        return View(producto);
    }

    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> DeleteConfirmed(int id)
    {
        var producto = await context.Productos.FindAsync(id);
        if (producto == null) return RedirectToAction(nameof(Manage));

        // Si el producto aparece en ventas o compras no se puede borrar (se perdería el historial):
        // en ese caso se desactiva para que deje de mostrarse en el catálogo.
        if (await TieneMovimientosAsync(id))
        {
            producto.Estado = false;
            TempData["Info"] = $"\"{producto.Nombre}\" tiene ventas o compras registradas, por lo que se desactivó en
lugar de eliminarse.";
        }
        else
        {
            context.Productos.Remove(producto); // el inventario se borra en cascada
            TempData["Exito"] = $"Producto \"{producto.Nombre}\" eliminado.";
        }

        await context.SaveChangesAsync();
        return RedirectToAction(nameof(Manage));
    }

    // ----- MÉTODOS AUXILIARES -----

    private async Task<bool> TieneMovimientosAsync(int idProducto) =>
        await context.DetalleVentas.AnyAsync(d => d.IdProducto == idProducto) ||
        await context.DetalleCompras.AnyAsync(d => d.IdProducto == idProducto);

    private async Task CargarListasAsync(int? categoria = null, int? proveedor = null)
    {
        var categorias = await context.Categorias.AsNoTracking().Where(c => c.Estado).OrderBy(c =>
c.Nombre).ToListAsync();
        var proveedores = await context.Proveedores.AsNoTracking().Where(p => p.Estado).OrderBy(p =>
p.Nombre).ToListAsync();
        ViewBag.Categorias = new SelectList(categorias, "IdCategoria", "Nombre", categoria);
        ViewBag.Proveedores = new SelectList(proveedores, "IdProveedor", "Nombre", proveedor);
    }

```

```

private static bool ImagenValida(IFormFile archivo)
{
    var extension = Path.GetExtension(archivo.FileName).ToLowerInvariant();
    return ExtensionesPermitidas.Contains(extension) && archivo.Length > 0 && archivo.Length <= 5 * 1024 * 1024;
}

// Guarda la imagen en wwwroot/images/productos con un nombre único y devuelve la URL pública
private async Task<string> GuardarImagenAsync(IFormFile archivo)
{
    var carpeta = Path.Combine(env.WebRootPath, "images", "productos");
    Directory.CreateDirectory(carpeta);

    var nombre = $"{Guid.NewGuid():N}{Path.GetExtension(archivo.FileName).ToLowerInvariant()}";
    var ruta = Path.Combine(carpeta, nombre);

    using (var stream = new FileStream(ruta, FileMode.Create))
        await archivo.CopyToAsync(stream);

    return $"/images/productos/{nombre}";
}
}
}

```

- **Index (catálogo):** solo productos activos de categorías activas; búsqueda con `ILike` (PostgreSQL, sin distinguir mayúsculas), filtro por categoría y paginación con `Skip/Take`.
- **Create** (Admin y Empleado): valida la imagen, la guarda con un nombre único y crea el producto **junto con su registro de** `Inventario`.
- **Edit:** copia solo los campos editables sobre el original. Si el stock se cambió a mano, llama a `ActualizarStock` para que el inventario quede sincronizado con fecha.
- **Delete** (solo Admin): si el producto aparece en ventas o compras se **desactiva**; si no, se borra y su inventario se borra en cascada.

7.4 `Controllers/CategoriasController.cs`, `ProveedoresController.cs` y `FormasPagoController.cs`

Los tres son CRUD de administrador con el mismo patrón: lista, crear, editar, eliminar. Como estas entidades son obligatorias en otras tablas (un producto necesita categoría y proveedor; una venta necesita forma de pago), si están en uso **se desactivan** en lugar de borrarse.

`Controllers/CategoriasController.cs`

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    // Solo el administrador gestiona las categorías
    [Authorize(Roles = "Admin")]
    public class CategoriasController(ApplicationDbContext context) : Controller
    {
        public async Task<IActionResult> Index()
        {
            var categorias = await context.Categorias
                .AsNoTracking()

```

```

        .Include(c => c.Productos)
        .OrderBy(c => c.Nombre)
        .ToListAsync();
    return View(categorias);
}

public IActionResult Create() => View(new Categoria());

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Categoria categoria)
{
    if (!ModelState.IsValid) return View(categoria);

    if (await context.Categorias.AnyAsync(c => c.Nombre == categoria.Nombre))
    {
        ModelState.AddModelError("Nombre", "Ya existe una categoría con ese nombre.");
        return View(categoria);
    }

    context.Categorias.Add(categoria);
    await context.SaveChangesAsync();
    TempData["Exito"] = $"Categoría \"{categoria.Nombre}\" creada.";
    return RedirectToAction(nameof(Index));
}

public async Task<IActionResult> Edit(int id)
{
    var categoria = await context.Categorias.FindAsync(id);
    if (categoria == null) return NotFound();
    return View(categoria);
}

// Categoria.actualizar()
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, Categoria categoria)
{
    if (id != categoria.IdCategoria) return NotFound();
    if (!ModelState.IsValid) return View(categoria);

    if (await context.Categorias.AnyAsync(c => c.Nombre == categoria.Nombre && c.IdCategoria != id))
    {
        ModelState.AddModelError("Nombre", "Ya existe una categoría con ese nombre.");
        return View(categoria);
    }

    context.Categorias.Update(categoria);
    await context.SaveChangesAsync();
    TempData["Exito"] = $"Categoría \"{categoria.Nombre}\" actualizada.";
    return RedirectToAction(nameof(Index));
}

public async Task<IActionResult> Delete(int id)
{
    var categoria = await context.Categorias
        .AsNoTracking()
        .Include(c => c.Productos)
        .FirstOrDefaultAsync(c => c.IdCategoria == id);
    if (categoria == null) return NotFound();
    return View(categoria);
}

```

```

[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var categoria = await context.Categorias.Include(c => c.Productos).FirstOrDefaultAsync(c => c.IdCategoria == id);
    if (categoria == null) return RedirectToAction(nameof(Index));

    // La categoría es obligatoria en cada producto: si tiene productos, se desactiva en vez de borrarse
    if (categoria.Productos.Count > 0)
    {
        categoria.Estado = false;
        TempData["Info"] = $"\"{categoria.Nombre}\" tiene productos asociados, por lo que se desactivó en lugar de eliminarse.";
    }
    else
    {
        context.Categorias.Remove(categoria);
        TempData["Exito"] = $"Categoría \"{categoria.Nombre}\" eliminada.";
    }

    await context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
}
}

```

Controllers/ProveedoresController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    [Authorize(Roles = "Admin")]
    public class ProveedoresController(ApplicationDbContext context) : Controller
    {
        public async Task<IActionResult> Index()
        {
            var proveedores = await context.Proveedores
                .AsNoTracking()
                .Include(p => p.Productos)
                .Include(p => p.Compras)
                .OrderBy(p => p.Nombre)
                .ToListAsync();
            return View(proveedores);
        }

        public IActionResult Create() => View(new Proveedor());

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Create(Proveedor proveedor)
        {
            if (!ModelState.IsValid) return View(proveedor);

            if (await context.Proveedores.AnyAsync(p => p.Ruc == proveedor.Ruc))
            {
                ModelState.AddModelError("Ruc", "Ya existe un proveedor con ese RUC/NIT.");
                return View(proveedor);
            }
        }
    }
}

```

```

    }

    context.Proveedores.Add(proveedor);
    await context.SaveChangesAsync();
    TempData["Exito"] = $"Proveedor \"{proveedor.Nombre}\" creado.";
    return RedirectToAction(nameof(Index));
}

public async Task<IActionResult> Edit(int id)
{
    var proveedor = await context.Proveedores.FindAsync(id);
    if (proveedor == null) return NotFound();
    return View(proveedor);
}

// Proveedor.actualizar()
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, Proveedor proveedor)
{
    if (id != proveedor.IdProveedor) return NotFound();
    if (!ModelState.IsValid) return View(proveedor);

    if (await context.Proveedores.AnyAsync(p => p.Ruc == proveedor.Ruc && p.IdProveedor != id))
    {
        ModelState.AddModelError("Ruc", "Ya existe un proveedor con ese RUC/NIT.");
        return View(proveedor);
    }

    context.Proveedores.Update(proveedor);
    await context.SaveChangesAsync();
    TempData["Exito"] = $"Proveedor \"{proveedor.Nombre}\" actualizado.";
    return RedirectToAction(nameof(Index));
}

public async Task<IActionResult> Delete(int id)
{
    var proveedor = await context.Proveedores
        .AsNoTracking()
        .Include(p => p.Productos)
        .Include(p => p.Compras)
        .FirstOrDefaultAsync(p => p.IdProveedor == id);
    if (proveedor == null) return NotFound();
    return View(proveedor);
}

[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var proveedor = await context.Proveedores.Include(p => p.Productos).Include(p =>
p.Compras).FirstOrDefaultAsync(p => p.IdProveedor == id);
    if (proveedor == null) return RedirectToAction(nameof(Index));

    // Si tiene productos o compras asociadas, se desactiva (no se puede borrar por las claves foráneas)
    if (proveedor.Productos.Count > 0 || proveedor.Compras.Count > 0)
    {
        proveedor.Estado = false;
        TempData["Info"] = $"\"{proveedor.Nombre}\" tiene productos o compras asociadas, por lo que se desactivó en
lugar de eliminarse.";
    }
    else
    {

```

```

        context.Proveedores.Remove(proveedor);
        TempData["Exito"] = $"Proveedor \"{proveedor.Nombre}\" eliminado.";
    }

    await context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
}
}

```

Controllers/FormasPagoController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    [Authorize(Roles = "Admin")]
    public class FormasPagoController(ApplicationDbContext context) : Controller
    {
        public async Task<IActionResult> Index()
        {
            var formas = await context.FormasPago.AsNoTracking().Include(f => f.Ventas).OrderBy(f =>
f.Nombre).ToListAsync();
            return View(formas);
        }

        public IActionResult Create() => View(new FormaPago());

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Create(FormaPago formaPago)
        {
            if (!ModelState.IsValid) return View(formaPago);

            if (await context.FormasPago.AnyAsync(f => f.Nombre == formaPago.Nombre))
            {
                ModelState.AddModelError("Nombre", "Ya existe una forma de pago con ese nombre.");
                return View(formaPago);
            }

            context.FormasPago.Add(formaPago);
            await context.SaveChangesAsync();
            TempData["Exito"] = $"Forma de pago \"{formaPago.Nombre}\" creada.";
            return RedirectToAction(nameof(Index));
        }

        public async Task<IActionResult> Edit(int id)
        {
            var formaPago = await context.FormasPago.FindAsync(id);
            if (formaPago == null) return NotFound();
            return View(formaPago);
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Edit(int id, FormaPago formaPago)
        {
            if (id != formaPago.IdFormaPago) return NotFound();

```

```

        if (!ModelState.IsValid) return View(formaPago);

        context.FormasPago.Update(formaPago);
        await context.SaveChangesAsync();
        TempData["Exito"] = $"Forma de pago \"{formaPago.Nombre}\" actualizada.";
        return RedirectToAction(nameof(Index));
    }

    public async Task<IActionResult> Delete(int id)
    {
        var formaPago = await context.FormasPago.AsNoTracking().Include(f => f.Ventas).FirstOrDefaultAsync(f =>
f.IdFormaPago == id);
        if (formaPago == null) return NotFound();
        return View(formaPago);
    }

    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> DeleteConfirmed(int id)
    {
        var formaPago = await context.FormasPago.Include(f => f.Ventas).FirstOrDefaultAsync(f => f.IdFormaPago ==
id);
        if (formaPago == null) return RedirectToAction(nameof(Index));

        if (formaPago.Ventas.Count > 0)
        {
            formaPago.Estado = false;
            TempData["Info"] = $"\"{formaPago.Nombre}\" ya se usó en ventas, por lo que se desactivó en lugar de
eliminarse.";
        }
        else
        {
            context.FormasPago.Remove(formaPago);
            TempData["Exito"] = $"Forma de pago \"{formaPago.Nombre}\" eliminada.";
        }

        await context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
}
}

```

7.5 Controllers/CarritoController.cs — carrito en sesión

Controllers/CarritoController.cs

```

using Microsoft.AspNetCore.Mvc;
using FerreteriaApp.Data;
using FerreteriaApp.Helpers;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    // Carrito de compras guardado en la sesión (no en la base de datos).
    // Cualquier visitante puede armar su carrito; para confirmar la compra hay que ser Cliente.
    public class CarritoController(ApplicationDbContext context) : Controller
    {
        public IActionResult Index()
        {
            var items = CarritoSesion.Obtener(HttpContext.Session);

            // Se refresca el stock actual de cada producto para avisar si ya no alcanza

```

```

        var ids = items.Select(i => i.IdProducto).ToList();
        ViewBag.Stock = context.Productos
            .Where(p => ids.Contains(p.IdProducto))
            .ToDictionary(p => p.IdProducto, p => p.Stock);

        return View(items);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Agregar(int idProducto, int cantidad = 1, string? returnUrl = null)
    {
        var producto = await context.Productos.FindAsync(idProducto);

        if (producto == null || !producto.Estado)
        {
            TempData["Error"] = "El producto no está disponible.";
            return Volver(returnUrl);
        }

        if (producto.Stock <= 0)
        {
            TempData["Error"] = $"{producto.Nombre}\" no tiene stock disponible.";
            return Volver(returnUrl);
        }

        cantidad = Math.Max(1, cantidad);

        var items = CarritoSesion.Obtener(HttpContext.Session);
        var item = items.FirstOrDefault(i => i.IdProducto == idProducto);

        if (item == null)
        {
            item = new CarritoItem
            {
                IdProducto = producto.IdProducto,
                Nombre = producto.Nombre,
                PrecioUnitario = producto.PrecioVenta,
                ImagenUrl = producto.ImagenUrl,
                Cantidad = 0
            };
            items.Add(item);
        }

        // No se puede pedir más de lo que hay en stock
        var nuevaCantidad = Math.Min(item.Cantidad + cantidad, producto.Stock);
        if (nuevaCantidad == item.Cantidad)
            TempData["Info"] = $"Ya tenés el máximo disponible de \"{producto.Nombre}\" ({producto.Stock}) en el carrito.";
        else
            TempData["Exito"] = $" \"{producto.Nombre}\" agregado al carrito.";

        item.Cantidad = nuevaCantidad;
        item.PrecioUnitario = producto.PrecioVenta; // por si cambió el precio
        CarritoSesion.Guardar(HttpContext.Session, items);

        return Volver(returnUrl);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Actualizar(int idProducto, int cantidad)
    {

```

```

var items = CarritoSesion.Obtener(HttpContext.Session);
var item = items.FirstOrDefault(i => i.IdProducto == idProducto);
if (item == null) return RedirectToAction(nameof(Index));

if (cantidad <= 0)
{
    items.Remove(item);
}
else
{
    var producto = await context.Productos.FindAsync(idProducto);
    var stock = producto?.Stock ?? 0;
    item.Cantidad = Math.Min(cantidad, stock);
    if (item.Cantidad < cantidad)
        TempData["Info"] = $"Solo hay {stock} unidades de \"{item.Nombre}\".";
    if (item.Cantidad == 0) items.Remove(item);
}

CarritoSesion.Guardar(HttpContext.Session, items);
return RedirectToAction(nameof(Index));
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Quitar(int idProducto)
{
    var items = CarritoSesion.Obtener(HttpContext.Session);
    items.RemoveAll(i => i.IdProducto == idProducto);
    CarritoSesion.Guardar(HttpContext.Session, items);
    return RedirectToAction(nameof(Index));
}

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Vaciar()
{
    CarritoSesion.Vaciar(HttpContext.Session);
    TempData["Info"] = "Carrito vaciado.";
    return RedirectToAction(nameof(Index));
}

// Vuelve a la página desde donde se agregó el producto (catálogo o detalle)
private IActionResult Volver(string? returnUrl)
{
    if (!string.IsNullOrEmpty(returnUrl) && Url.IsLocalUrl(returnUrl))
        return Redirect(returnUrl);
    return RedirectToAction("Index", "Productos");
}
}
}

```

- No tiene `[Authorize]`: cualquier visitante puede armar su carrito. Al ir a "Finalizar compra" se le pide iniciar sesión y el carrito se conserva (vive en la sesión, no en el usuario).
- **Agregar**: si el producto ya está, suma la cantidad; nunca supera el stock (`Math.Min`).
- **Actualizar / Quitar / Vaciar**: modifican la lista y la vuelven a guardar en la sesión.

7.6 Controllers/VentasController.cs — ventas web y de mostrador

Controllers/VentasController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Helpers;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    [Authorize]
    public class VentasController(ApplicationDbContext context, UserManager<Usuario> userManager) : Controller
    {
        // ----- CHECKOUT WEB (Cliente.realizarCompra) -----

        [Authorize(Roles = "Cliente")]
        [HttpGet]
        public async Task<IActionResult> Checkout()
        {
            var items = CarritoSesion.Obtener(HttpContext.Session);
            if (items.Count == 0)
            {
                TempData["Info"] = "Tu carrito está vacío.";
                return RedirectToAction("Index", "Carrito");
            }

            var cliente = (Cliente?)await userManager.GetUserAsync(User);
            if (cliente == null) return Challenge();

            await CargarFormasPagoAsync();
            return View(new CheckoutViewModel
            {
                Items = items,
                DireccionEntrega = cliente.Direccion,
                Telefono = cliente.Telefono
            });
        }

        [Authorize(Roles = "Cliente")]
        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Checkout(CheckoutViewModel model)
        {
            var items = CarritoSesion.Obtener(HttpContext.Session);
            if (items.Count == 0)
            {
                TempData["Info"] = "Tu carrito está vacío.";
                return RedirectToAction("Index", "Carrito");
            }

            var cliente = (Cliente?)await userManager.GetUserAsync(User);
            if (cliente == null) return Challenge();

            model.Items = items;
            model.DireccionEntrega = cliente.Direccion;
            model.Telefono = cliente.Telefono;

            if (!ModelState.IsValid)
            {
                await CargarFormasPagoAsync(model.IdFormaPago);
                return View(model);
            }
        }
    }
}

```

```

// Transacción: o se guarda todo (venta + descuento de stock) o nada
using var transaction = await context.Database.BeginTransactionAsync();

var venta = new Venta
{
    IdCliente = cliente.Id,
    IdFormaPago = model.IdFormaPago,
    Observaciones = model.Observaciones,
    Fecha = DateTime.UtcNow,
    Estado = EstadoVenta.Pendiente
};

// Se verifica el stock de cada producto leyendo la base de datos justo antes de confirmar
var ids = items.Select(i => i.IdProducto).ToList();
var productos = await context.Productos.Include(p => p.Inventarios)
    .Where(p => ids.Contains(p.IdProducto)).ToListAsync();

foreach (var item in items)
{
    var producto = productos.FirstOrDefault(p => p.IdProducto == item.IdProducto);
    if (producto == null || !producto.Estado)
    {
        ModelState.AddModelError(string.Empty, $"{item.Nombre}\" ya no está disponible.");
        continue;
    }
    try
    {
        venta.AgregarDetalle(producto, item.Cantidad); // valida stock y usa el precio actual
        producto.ActualizarStock(-item.Cantidad);    // no permitir vender si no hay stock
    }
    catch (InvalidOperationException ex)
    {
        ModelState.AddModelError(string.Empty, ex.Message);
    }
}

if (!ModelState.IsValid || !venta.ValidarDatos())
{
    await transaction.RollbackAsync();
    if (venta.Detalles.Count == 0) ModelState.AddModelError(string.Empty, "La venta no tiene productos válidos.");
    await CargarFormasPagoAsync(model.IdFormaPago);
    return View(model);
}

venta.CalcularTotal();
context.Ventas.Add(venta);
await context.SaveChangesAsync();
await transaction.CommitAsync();

CarritoSesion.Vaciar(HttpContext.Session);

TempData["Exito"] = $"{Compra #{venta.IdVenta} realizada con éxito! Te contactaremos para coordinar la entrega.";
return RedirectToAction(nameof(Details), new { id = venta.IdVenta });
}

// ----- MIS COMPRAS (Cliente.verHistorial) -----

[Authorize(Roles = "Cliente")]
public async Task<IActionResult> MisCompras()
{

```

```

var idCliente = User.GetUsuarioId();
var ventas = await context.Ventas
    .AsNoTracking()
    .Include(v => v.Detalles)
    .Include(v => v.FormaPago)
    .Where(v => v.IdCliente == idCliente)
    .OrderByDescending(v => v.Fecha)
    .ToListAsync();
return View(ventas);
}

public async Task<IActionResult> Details(int id)
{
    var venta = await context.Ventas
        .AsNoTracking()
        .Include(v => v.Cliente)
        .Include(v => v.Empleado)
        .Include(v => v.FormaPago)
        .Include(v => v.Detalles).ThenInclude(d => d.Producto)
        .FirstOrDefaultAsync(v => v.IdVenta == id);

    if (venta == null) return NotFound();

    // Solo el cliente dueño de la venta o el personal pueden verla
    if (venta.IdCliente != User.GetUsuarioId() && !User.EsPersonal())
        return Forbid();

    return View(venta);
}

// El cliente puede cancelar su compra mientras esté pendiente
[Authorize(Roles = "Cliente")]
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Cancelar(int id)
{
    var venta = await CargarVentaParaCambioAsync(id);
    if (venta == null || venta.IdCliente != User.GetUsuarioId()) return NotFound();

    if (venta.Estado != EstadoVenta.Pendiente)
    {
        TempData["Error"] = "Solo se pueden cancelar compras pendientes.";
        return RedirectToAction(nameof(Details), new { id });
    }

    CambiarEstado(venta, EstadoVenta.Cancelada, null);
    await context.SaveChangesAsync();

    TempData["Info"] = $"Compra #{venta.IdVenta} cancelada.";
    return RedirectToAction(nameof(Details), new { id });
}

// ----- GESTIÓN DE VENTAS (Admin y Empleado) -----

[Authorize(Roles = "Admin,Empleado")]
public async Task<IActionResult> Manage(EstadoVenta? estado)
{
    var query = context.Ventas
        .AsNoTracking()
        .Include(v => v.Cliente)
        .Include(v => v.Empleado)
        .Include(v => v.FormaPago)
        .Include(v => v.Detalles)

```

```

        .AsQueryable();

    if (estado.HasValue)
        query = query.Where(v => v.Estado == estado.Value);

    ViewBag.Estado = estado;
    return View(await query.OrderByDescending(v => v.Fecha).ToListAsync());
}

[HttpPost]
[ValidateAntiForgeryToken]
[Authorize(Roles = "Admin,Empleado")]
public async Task<IActionResult> CambiarEstado(int id, EstadoVenta estado)
{
    var venta = await CargarVentaParaCambioAsync(id);
    if (venta == null) return NotFound();

    if (venta.Estado == estado)
        return RedirectToAction(nameof(Details), new { id });

    try
    {
        CambiarEstado(venta, estado, User.GetUsuarioId());
        await context.SaveChangesAsync();
        TempData["Exito"] = $"Venta #{venta.IdVenta} ahora está \"{estado}\".";
    }
    catch (InvalidOperationException ex)
    {
        TempData["Error"] = ex.Message;
    }

    return RedirectToAction(nameof(Details), new { id });
}

// Venta registrada por un empleado en el mostrador (Empleado.registrarVenta)
[Authorize(Roles = "Admin,Empleado")]
[HttpGet]
public async Task<IActionResult> Registrar()
{
    await CargarListasRegistrarAsync();
    return View(new RegistrarVentaViewModel { Items = { new LineaVentaInput() } });
}

[Authorize(Roles = "Admin,Empleado")]
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Registrar(RegistrarVentaViewModel model)
{
    model.Items = model.Items.Where(i => i.IdProducto > 0).ToList();
    if (model.Items.Count == 0)
        ModelState.AddModelError(string.Empty, "Agregá al menos un producto a la venta.");

    if (!ModelState.IsValid)
    {
        await CargarListasRegistrarAsync(model);
        return View(model);
    }

    using var transaction = await context.Database.BeginTransactionAsync();

    var venta = new Venta
    {
        IdCliente = model.IdCliente,

```

```

        IdFormaPago = model.IdFormaPago,
        Observaciones = model.Observaciones,
        Fecha = DateTime.UtcNow
    };

    var ids = model.Items.Select(i => i.IdProducto).ToList();
    var productos = await context.Productos.Include(p => p.Inventarios)
        .Where(p => ids.Contains(p.IdProducto)).ToListAsync();

    // Si el mismo producto está en dos filas, se suman las cantidades
    foreach (var grupo in model.Items.GroupBy(i => i.IdProducto))
    {
        var producto = productos.FirstOrDefault(p => p.IdProducto == grupo.Key);
        var cantidad = grupo.Sum(i => i.Cantidad);
        if (producto == null) continue;
        try
        {
            venta.AgregarDetalle(producto, cantidad);
            producto.ActualizarStock(-cantidad);
        }
        catch (Exception ex) when (ex is InvalidOperationException or ArgumentException)
        {
            ModelState.AddModelError(string.Empty, ex.Message);
        }
    }

    if (!ModelState.IsValid || !venta.ValidarDatos())
    {
        await transaction.RollbackAsync();
        await CargarListasRegistrarAsync(model);
        return View(model);
    }

    venta.CalcularTotal();
    venta.CerrarVenta(User.GetUsuarioId()); // venta de mostrador: queda completada por el empleado
    context.Ventas.Add(venta);
    await context.SaveChangesAsync();
    await transaction.CommitAsync();

    TempData["Exito"] = $"Venta #{venta.IdVenta} registrada por {Formato.Precio(venta.Total)}.";
    return RedirectToAction(nameof(Details), new { id = venta.IdVenta });
}

// ----- MÉTODOS AUXILIARES -----

private async Task<Venta?> CargarVentaParaCambioAsync(int id) =>
    await context.Ventas
        .Include(v => v.Detalles).ThenInclude(d => d.Producto).ThenInclude(p => p!.Inventarios)
        .FirstOrDefaultAsync(v => v.IdVenta == id);

// Cambia el estado ajustando el stock: Pendiente/Completada lo tienen descontado,
// Cancelada/Devuelta lo devuelven. Al completar se registra el empleado (cerrarVenta).
private static void CambiarEstado(Venta venta, EstadoVenta nuevo, int? idEmpleado)
{
    var descontabaAntes = venta.DescuentaStock();
    var estadoAnterior = venta.Estado;
    venta.Estado = nuevo;
    var descuentoAhora = venta.DescuentaStock();

    try
    {
        if (descontabaAntes && !descuentoAhora)
            foreach (var d in venta.Detalles) d.Producto?.ActualizarStock(d.Cantidad);
    }

```

```

        else if (!descontabaAntes && descuentoAhora)
            foreach (var d in venta.Detalles) d.Producto?.ActualizarStock(-d.Cantidad);
    }
    catch (InvalidOperationException)
    {
        venta.Estado = estadoAnterior;
        throw new InvalidOperationException("No hay stock suficiente para volver a activar la venta.");
    }

    if (nuevo == EstadoVenta.Completada && idEmpleado.HasValue)
        venta.CerrarVenta(idEmpleado.Value);
}

private async Task CargarFormasPagoAsync(int? seleccionada = null)
{
    var formas = await context.FormasPago.AsNoTracking().Where(f => f.Estado).OrderBy(f =>
f.Nombre).ToListAsync();
    ViewBag.FormasPago = new SelectList(formas, "IdFormaPago", "Nombre", seleccionada);
}

private async Task CargarListasRegistrarAsync(RegistrarVentaViewModel? model = null)
{
    var clientes = await context.Clientes.AsNoTracking().Where(c => c.Estado).OrderBy(c => c.Apellido).ThenBy(c =>
c.Nombre)
        .Select(c => new { c.Id, Texto = c.Apellido + ", " + c.Nombre + " (" + c.Email + ")" }).ToListAsync();
    ViewBag.Clientes = new SelectList(clientes, "Id", "Texto", model?.IdCliente);
    await CargarFormasPagoAsync(model?.IdFormaPago);
    ViewBag.Productos = await context.Productos.AsNoTracking().Where(p => p.Estado).OrderBy(p =>
p.Nombre).ToListAsync();
}
}
}

```

- **Checkout (POST)** (`Cliente.realizarCompra`), la parte más importante: 1. Lee el carrito de la sesión y el cliente logueado. 2. Abre una **transacción**. 3. Para cada ítem, `venta.AgregarDetalle(producto, cantidad)` crea la línea con el precio actual y `producto.ActualizarStock(-cantidad)` descuenta stock e inventario. Si no alcanza, la excepción se convierte en un error del formulario ("no permitir vender si no hay stock"). 4. `CalcularTotal()`, se guarda la venta en estado *Pendiente* y se hace **commit**. Recién ahí se vacía el carrito.
- **MisCompras / Details** (`Cliente.verHistorial`): un cliente solo ve sus propias ventas (`Forbid()` si intenta ver otra); el personal las ve todas.
- **Cancelar**: el cliente puede cancelar solo si está *Pendiente*; se repone el stock.
- **Manage / CambiarEstado** (personal): `CambiarEstado` repone o vuelve a descontar el stock según se pase de un estado que lo descuenta (*Pendiente/Completada*) a uno que lo devuelve (*Cancelada/Devuelta*) o viceversa. Al pasar a *Completada* se llama a `CerrarVenta(idEmpleado)`.
- **Registrar** (`Empleado.registrarVenta`): venta de mostrador. Se eligen cliente, forma de pago y filas de productos; se agrupan filas repetidas, se valida el stock y la venta queda *Completada* a nombre del empleado logueado.

7.7 Controllers/ComprasController.cs — compras a proveedores

Controllers/ComprasController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Helpers;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    // Compras de mercadería a proveedores (Empleado.registrarCompra). Al completarse suben el stock.
    [Authorize(Roles = "Admin,Empleado")]
    public class ComprasController(ApplicationDbContext context) : Controller
    {
        public async Task<IActionResult> Index(EstadoCompra? estado)
        {
            var query = context.Compras
                .AsNoTracking()
                .Include(c => c.Proveedor)
                .Include(c => c.Empleado)
                .Include(c => c.Detalles)
                .AsQueryable();

            if (estado.HasValue)
                query = query.Where(c => c.Estado == estado.Value);

            ViewBag.Estado = estado;
            return View(await query.OrderByDescending(c => c.Fecha).ToListAsync());
        }

        public async Task<IActionResult> Details(int id)
        {
            var compra = await context.Compras
                .AsNoTracking()
                .Include(c => c.Proveedor)
                .Include(c => c.Empleado)
                .Include(c => c.Detalles).ThenInclude(d => d.Producto)
                .FirstOrDefaultAsync(c => c.IdCompra == id);

            if (compra == null) return NotFound();
            return View(compra);
        }

        [HttpGet]
        public async Task<IActionResult> Registrar()
        {
            await CargarListasAsync();
            return View(new RegistrarCompraViewModel { Items = { new LineaCompraInput() } });
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Registrar(RegistrarCompraViewModel model)
        {
            model.Items = model.Items.Where(i => i.IdProducto > 0).ToList();
            if (model.Items.Count == 0)
                ModelState.AddModelError(string.Empty, "Agregá al menos un producto a la compra.");

            if (!ModelState.IsValid)
            {
                await CargarListasAsync(model);
                return View(model);
            }
        }
    }
}

```

```

var compra = new Compra
{
    IdProveedor = model.IdProveedor,
    IdEmpleado = User.GetUsuarioId(),
    Observaciones = model.Observaciones,
    Fecha = DateTime.UtcNow,
    Estado = EstadoCompra.Pendiente
};

var ids = model.Items.Select(i => i.IdProducto).ToList();
var productos = await context.Productos.Where(p => ids.Contains(p.IdProducto)).ToListAsync();

foreach (var linea in model.Items)
{
    var producto = productos.FirstOrDefault(p => p.IdProducto == linea.IdProducto);
    if (producto == null) continue;
    try
    {
        compra.AgregarDetalle(producto, linea.Cantidad, linea.PrecioUnitario);
    }
    catch (ArgumentException ex)
    {
        ModelState.AddModelError(string.Empty, $"{producto.Nombre}: {ex.Message}");
    }
}

if (!ModelState.IsValid || !compra.ValidarDatos())
{
    await CargarListasAsync(model);
    return View(model);
}

compra.CalcularTotal();
context.Compras.Add(compra);
await context.SaveChangesAsync();

```

```

TempData["Exito"] = $"Compra #{compra.IdCompra} registrada por {Formato.Precio(compra.Total)}. Al
completarla se sumará el stock.";
return RedirectToAction(nameof(Details), new { id = compra.IdCompra });
}

```

// Pendiente -> Completada: suma el stock. Completada -> Cancelada: lo resta (sin dejarlo negativo).

[HttpPost]

[ValidateAntiForgeryToken]

public async Task<IActionResult> CambiarEstado(int id, EstadoCompra estado)

```

{
    var compra = await context.Compras
        .Include(c => c.Detalles).ThenInclude(d => d.Producto).ThenInclude(p => p!.Inventarios)
        .FirstOrDefaultAsync(c => c.IdCompra == id);

```

```

    if (compra == null) return NotFound();

```

```

    if (compra.Estado == estado) return RedirectToAction(nameof(Details), new { id });

```

```

    using var transaction = await context.Database.BeginTransactionAsync();

```

```

    try
    {

```

```

        var sumabaAntes = compra.Estado == EstadoCompra.Completada;
        var sumaAhora = estado == EstadoCompra.Completada;

```

```

        if (!sumabaAntes && sumaAhora)
        {

```

```

            // Ingresa la mercadería: sube el stock y se actualiza el precio de compra del producto

```

```

        foreach (var d in compra.Detalles)
        {
            d.Producto!.ActualizarStock(d.Cantidad);
            d.Producto.PrecioCompra = d.PrecioUnitario;
        }
    }
    else if (sumabaAntes && !sumaAhora)
    {
        // Se anula una compra ya ingresada: se descuenta, pero no se permite stock negativo
        foreach (var d in compra.Detalles)
            d.Producto!.ActualizarStock(-d.Cantidad);
    }

    compra.Estado = estado;
    await context.SaveChangesAsync();
    await transaction.CommitAsync();
    TempData["Exito"] = $"Compra #{compra.IdCompra} ahora está \"{estado}\".";
}
catch (InvalidOperationException ex)
{
    await transaction.RollbackAsync();
    TempData["Error"] = $"No se puede cambiar el estado: {ex.Message}";
}

return RedirectToAction(nameof(Details), new { id });
}

private async Task CargarListasAsync(RegistrarCompraViewModel? model = null)
{
    var proveedores = await context.Proveedores.AsNoTracking().Where(p => p.Estado).OrderBy(p =>
p.Nombre).ToListAsync();
    ViewBag.Proveedores = new SelectList(proveedores, "IdProveedor", "Nombre", model?.IdProveedor);
    ViewBag.Productos = await context.Productos.AsNoTracking().Where(p => p.Estado).OrderBy(p =>
p.Nombre).ToListAsync();
}
}
}

```

- **Registrar** (`Empleado.registrarCompra`): crea la compra *Pendiente* con sus líneas al precio unitario indicado. **Todavía no toca el stock**: la mercadería no llegó.
- **CambiarEstado**: al pasar a *Completada* ingresa la mercadería (`ActualizarStock(+cantidad)`) y actualiza el `PrecioCompra` del producto con el último precio pagado. Si se cancela una compra ya completada, se descuenta; si el stock quedaría negativo, `ActualizarStock` lanza la excepción y la transacción se deshace ("no permitir compra con stock negativo").

7.8 Controllers/InventarioController.cs — inventario

Controllers/InventarioController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    // Inventario: stock actual, mínimo y última actualización de cada producto (Empleado.gestionarInventario)
    [Authorize(Roles = "Admin,Empleado")]

```

```

public class InventarioController(ApplicationDbContext context) : Controller
{
    public async Task<IActionResult> Index(bool soloBajos = false)
    {
        var query = context.Inventarios
            .AsNoTracking()
            .Include(i => i.Producto).ThenInclude(p => p!.Categoria)
            .AsQueryable();

        if (soloBajos)
            query = query.Where(i => i.StockActual <= i.StockMinimo);

        ViewBag.SoloBajos = soloBajos;
        return View(await query.OrderBy(i => i.Producto!.Nombre).ToListAsync());
    }

    // Ajuste manual de stock (por ejemplo, después de un recuento físico)
    [HttpGet]
    public async Task<IActionResult> Ajustar(int id)
    {
        var inventario = await context.Inventarios.Include(i => i.Producto).FirstOrDefaultAsync(i => i.IdInventario ==
id);
        if (inventario == null) return NotFound();
        return View(inventario);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Ajustar(int id, int stockActual, int stockMinimo, string? motivo)
    {
        var inventario = await context.Inventarios.Include(i => i.Producto).FirstOrDefaultAsync(i => i.IdInventario ==
id);
        if (inventario == null) return NotFound();

        if (stockActual < 0 || stockMinimo < 0)
        {
            ModelState.AddModelError(string.Empty, "El stock y el stock mínimo no pueden ser negativos.");
            return View(inventario);
        }

        // El producto y su inventario se mantienen sincronizados
        inventario.ActualizarStock(stockActual - inventario.StockActual);
        inventario.StockMinimo = stockMinimo;
        inventario.Producto!.Stock = stockActual;
        inventario.Producto.StockMinimo = stockMinimo;

        await context.SaveChangesAsync();

        TempData["Exito"] = $"Inventario de \"{inventario.Producto.Nombre}\" ajustado a {stockActual} unidades." +
            (string.IsNullOrEmpty(motivo) ? "" : $" Motivo: {motivo}");
        return RedirectToAction(nameof(Index));
    }
}

```

Empleado.gestionarInventario: lista el inventario (con filtro "bajo mínimo" usando Inventario.VerificarStock) y permite un **ajuste manual** tras un recuento físico, manteniendo Producto.Stock e Inventario.StockActual iguales.

7.9 Controllers/AdminController.cs — panel y usuarios

Controllers/AdminController.cs

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using FerreteriaApp.Data;
using FerreteriaApp.Helpers;
using FerreteriaApp.Models;

namespace FerreteriaApp.Controllers
{
    // Panel de administración: resumen general (Admin y Empleado) y gestión de usuarios (solo Admin)
    [Authorize(Roles = "Admin,Empleado")]
    public class AdminController(ApplicationDbContext context, UserManager<Usuario> userManager) : Controller
    {
        public async Task<IActionResult> Index()
        {
            ViewBag.TotalProductos = await context.Productos.CountAsync();
            ViewBag.TotalClientes = await context.Clientes.CountAsync();
            ViewBag.TotalEmpleados = await context.Empleados.CountAsync();
            ViewBag.TotalVentas = await context.Ventas.CountAsync();
            ViewBag.VentasPendientes = await context.Ventas.CountAsync(v => v.Estado == EstadoVenta.Pendiente);
            ViewBag.ComprasPendientes = await context.Compras.CountAsync(c => c.Estado == EstadoCompra.Pendiente);
            ViewBag.IngresosVentas = await context.Ventas
                .Where(v => v.Estado == EstadoVenta.Completada)
                .SumAsync(v => (decimal?)v.Total) ?? 0m;
            ViewBag.GastosCompras = await context.Compras
                .Where(c => c.Estado == EstadoCompra.Completada)
                .SumAsync(c => (decimal?)c.Total) ?? 0m;

            // Inventario.verificarStock(): productos en o por debajo del mínimo
            ViewBag.StockBajo = await context.Inventarios
                .AsNoTracking()
                .Include(i => i.Producto)
                .Where(i => i.Producto!.Estado && i.StockActual <= i.StockMinimo)
                .OrderBy(i => i.StockActual)
                .Take(10)
                .ToListAsync();

            ViewBag.UltimasVentas = await context.Ventas
                .AsNoTracking()
                .Include(v => v.Cliente)
                .OrderByDescending(v => v.Fecha)
                .Take(5)
                .ToListAsync();

            // Productos más vendidos (por cantidad) en ventas completadas o pendientes
            ViewBag.MasVendidos = await context.DetalleVentas
                .AsNoTracking()
                .Where(d => d.Venta!.Estado == EstadoVenta.Completada || d.Venta.Estado == EstadoVenta.Pendiente)
                .GroupBy(d => d.Producto!.Nombre)
                .Select(g => new { Nombre = g.Key, Cantidad = g.Sum(d => d.Cantidad), Total = g.Sum(d => d.Subtotal) })
                .OrderByDescending(x => x.Cantidad)
                .Take(5)
                .ToListAsync();

            return View();
        }

        // ----- USUARIOS (solo Admin) -----

        [Authorize(Roles = "Admin")]
        public async Task<IActionResult> Usuarios()
        {

```

```

        var usuarios = await userManager.Users.AsNoTracking().OrderBy(u => u.Apellido).ThenBy(u =>
u.Nombre).ToListAsync();

        var roles = new Dictionary<int, IList<string>>();
        foreach (var u in usuarios)
            roles[u.Id] = await userManager.GetRolesAsync(u);

        ViewBag.Roles = roles;
        return View(usuarios);
    }

    // Activa o desactiva un usuario (Usuario.estado)
    [Authorize(Roles = "Admin")]
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> ToggleEstado(int id)
    {
        var usuario = await userManager.FindByIdAsync(id.ToString());
        if (usuario == null) return NotFound();

        if (usuario.Id == User.GetUsuarioId())
        {
            TempData["Error"] = "No podés desactivar tu propia cuenta.";
            return RedirectToAction(nameof(Usuarios));
        }

        usuario.Estado = !usuario.Estado;
        await userManager.UpdateAsync(usuario);

        TempData["Info"] = usuario.Estado ? $"{usuario.Email} activado." : $"{usuario.Email} desactivado.";
        return RedirectToAction(nameof(Usuarios));
    }

    // Cambia el rol de un empleado entre Empleado y Admin (los clientes siempre son Cliente)
    [Authorize(Roles = "Admin")]
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> CambiarRol(int id)
    {
        var usuario = await userManager.FindByIdAsync(id.ToString());
        if (usuario == null) return NotFound();

        if (usuario is not Empleado)
        {
            TempData["Error"] = "Solo los empleados pueden tener el rol de administrador.";
            return RedirectToAction(nameof(Usuarios));
        }

        if (usuario.Id == User.GetUsuarioId())
        {
            TempData["Error"] = "No podés cambiar tu propio rol.";
            return RedirectToAction(nameof(Usuarios));
        }

        var rolesActuales = await userManager.GetRolesAsync(usuario);
        await userManager.RemoveFromRolesAsync(usuario, rolesActuales);

        if (rolesActuales.Contains(DbInitializer.RolAdmin))
        {
            await userManager.AddToRoleAsync(usuario, DbInitializer.RolEmpleado);
            TempData["Info"] = $"{usuario.Email} ahora es Empleado.";
        }
        else
    }

```

```

    {
        await userManager.AddToRoleAsync(usuario, DbInitializer.RolAdmin);
        TempData["Exito"] = $"{usuario.Email} ahora es Administrador.";
    }

    return RedirectToAction(nameof(Usuarios));
}

// Cambia el tipo de cliente (Regular <-> Mayorista)
[Authorize(Roles = "Admin")]
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CambiarTipoCliente(int id)
{
    var cliente = await context.Clientes.FindAsync(id);
    if (cliente == null) return NotFound();

    cliente.TipoCliente = cliente.TipoCliente == "Mayorista" ? "Regular" : "Mayorista";
    await context.SaveChangesAsync();

    TempData["Info"] = $"{cliente.Email} ahora es cliente {cliente.TipoCliente}.";
    return RedirectToAction(nameof(Usuarios));
}

// ----- EMPLEADOS (solo Admin) -----

[Authorize(Roles = "Admin")]
[HttpGet]
public IActionResult CrearEmpleado() => View(new EmpleadoViewModel());

[Authorize(Roles = "Admin")]
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CrearEmpleado(EmpleadoViewModel model)
{
    if (string.IsNullOrEmpty(model.Password))
        ModelState.AddModelError("Password", "La contraseña es obligatoria.");
    if (!ModelState.IsValid) return View(model);

    var empleado = new Empleado
    {
        UserName = model.Email,
        Email = model.Email,
        EmailConfirmed = true,
        Nombre = model.Nombre,
        Apellido = model.Apellido,
        Cargo = model.Cargo,
        Telefono = model.Telefono,
        Direccion = model.Direccion,
        Estado = model.Estado
    };

    var result = await userManager.CreateAsync(empleado, model.Password!);
    if (!result.Succeeded)
    {
        foreach (var error in result.Errors.Where(e => e.Code != "DuplicateUserName"))
            ModelState.AddModelError(string.Empty, error.Description);
        return View(model);
    }

    var rol = model.Rol == DbInitializer.RolAdmin ? DbInitializer.RolAdmin : DbInitializer.RolEmpleado;
    await userManager.AddToRoleAsync(empleado, rol);
}

```

```

TempData["Exit"] = $"Empleado {empleado.NombreCompleto} creado con rol {rol}.";
return RedirectToAction(nameof(Usuarios));
}

[Authorize(Roles = "Admin")]
[HttpGet]
public async Task<IActionResult> EditarEmpleado(int id)
{
    var empleado = await context.Empleados.FindAsync(id);
    if (empleado == null) return NotFound();

    var roles = await userManager.GetRolesAsync(empleado);
    return View(new EmpleadoViewModel
    {
        Id = empleado.Id, Nombre = empleado.Nombre, Apellido = empleado.Apellido, Email = empleado.Email ?? "",
        Cargo = empleado.Cargo, Telefono = empleado.Telefono, Direccion = empleado.Direccion,
        Estado = empleado.Estado, Rol = roles.Contains(DbInitializer.RolAdmin) ? DbInitializer.RolAdmin :
        DbInitializer.RolEmpleado
    });
}

[Authorize(Roles = "Admin")]
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> EditarEmpleado(int id, EmpleadoViewModel model)
{
    if (id != model.Id) return NotFound();
    if (!ModelState.IsValid) return View(model);

    var empleado = await context.Empleados.FindAsync(id);
    if (empleado == null) return NotFound();

    empleado.Nombre = model.Nombre;
    empleado.Apellido = model.Apellido;
    empleado.Cargo = model.Cargo;
    empleado.Telefono = model.Telefono;
    empleado.Direccion = model.Direccion;
    if (empleado.Id != User.GetUsuarioId()) empleado.Estado = model.Estado;

    if (empleado.Email != model.Email)
    {
        await userManager.SetEmailAsync(empleado, model.Email);
        await userManager.SetUserNameAsync(empleado, model.Email);
    }

    var result = await userManager.UpdateAsync(empleado);
    if (!result.Succeeded)
    {
        foreach (var error in result.Errors) ModelState.AddModelError(string.Empty, error.Description);
        return View(model);
    }

    // Contraseña nueva solo si se escribió algo
    if (!string.IsNullOrEmpty(model.Password))
    {
        var token = await userManager.GeneratePasswordResetTokenAsync(empleado);
        var pwd = await userManager.ResetPasswordAsync(empleado, token, model.Password);
        if (!pwd.Succeeded)
        {
            foreach (var error in pwd.Errors) ModelState.AddModelError(string.Empty, error.Description);
            return View(model);
        }
    }
}

```

```

// Rol (no se puede cambiar el propio)
if (empleado.Id != User.GetUsuarioId())
{
    var rolesActuales = await userManager.GetRolesAsync(empleado);
    var rolNuevo = model.Rol == DbInitializer.RolAdmin ? DbInitializer.RolAdmin : DbInitializer.RolEmpleado;
    if (!rolesActuales.Contains(rolNuevo))
    {
        await userManager.RemoveFromRolesAsync(empleado, rolesActuales);
        await userManager.AddToRoleAsync(empleado, rolNuevo);
    }
}

TempData["Exito"] = $"Empleado {empleado.NombreCompleto} actualizado.";
return RedirectToAction(nameof(Usuarios));
}
}
}

```

- **Index (panel):** totales, ventas completadas, compras completadas, stock bajo (desde **Inventarios**), últimas ventas y más vendidos (con **GroupBy**).
 - **Usuarios** (solo Admin): lista todos con su tipo (Empleado/Cliente) y rol.
 - **ToggleEstado:** activa/desactiva (**Usuario.estado**). Nadie puede desactivarse a sí mismo.
 - **CambiarRol:** alterna Empleado ↔ Admin. Solo para empleados: un cliente nunca puede ser admin.
 - **CambiarTipoCliente:** Regular ↔ Mayorista.
 - **CrearEmpleado / EditarEmpleado:** alta y edición de empleados (con contraseña y rol). En la edición, la contraseña se cambia solo si se escribe una nueva (**ResetPasswordAsync** con token).
-

Parte 8 — Vistas (las pantallas)

8.0 Cómo funciona una vista Razor

Una vista es un archivo `.cshtml` en `Views/<Controlador>/<Accion>.cshtml`. Es HTML normal donde se puede intercalar C# con `@`:

```
@model List<Producto>          @* tipo de dato que recibe la vista (lo que pasó el controlador con View(...)) *@
@{
    ViewData["Title"] = "Catálogo"; @* bloque de código C#; Title lo usa el _Layout para el <title> *@
}

<h2>@ViewData["Title"]</h2>    @* imprimir un valor *@

@foreach (var p in Model)        @* Model es el objeto recibido; acá una lista de productos *@
{
    <p>@p.Nombre — @Formato.Precio(p.PrecioVenta)</p>
}

@if (User.IsInRole("Admin"))    @* condicional *@
{
    <a asp-action="Create">Crear</a>
}
```

Tag Helpers (atributos que empiezan con `asp-`):

Tag helper	Qué genera
<code><form asp-action="Login" method="post"></code>	<code><form action="/Account/Login" method="post"></code> más el token antifalsificación oculto.
<code><a asp-controller="Productos" asp-action="Details" asp-route-id="@p.IdProducto"></code>	<code></code>
<code><label asp-for="Email"></code>	<code><label for="Email">Email</label></code> (usa el <code>[Display]</code> del modelo).
<code><input asp-for="Email"></code>	<code><input type="email" id="Email" name="Email" value="..."></code> con los atributos de validación.
<code></code>	Muestra el mensaje de error de ese campo.
<code><div asp-validation-summary="ModelOnly"></code>	Muestra los errores generales (" All " muestra también los de cada campo).
<code><select asp-for="IdCategoria" asp-items="ViewBag.Categorias"></code>	Un combo con las opciones de una lista.
<code><partial name="_ProductoCard" model="p" /></code>	Inserta una vista parcial pasándole un modelo.

Archivos especiales de la carpeta `Views/`:

- `_ViewImports.cshtml`: `using` y tag helpers disponibles en **todas** las vistas.
- `_ViewStart.cshtml`: dice que todas las vistas usan `_Layout`.
- `Shared/_Layout.cshtml`: la plantilla. `@RenderBody()` es donde se inserta cada vista.
- `Shared/_ValidationScriptsPartial.cshtml`: scripts de jQuery que validan el formulario en el navegador antes de enviarlo. Se incluye con `@section Scripts { ... }`.

Bootstrap: las clases como `btn btn-warning`, `card`, `table table-hover`, `row`, `col-md-4` vienen de Bootstrap 5.

Los íconos (<i class="bi bi-cart3">) son de Bootstrap Icons.

8.1 Views/_ViewImports.cshtml y Views/_ViewStart.cshtml

Views/_ViewImports.cshtml

```
@using FerreteriaApp
@using FerreteriaApp.Models
@using FerreteriaApp.Helpers
@using Microsoft.AspNetCore.Identity
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Views/_ViewStart.cshtml

```
@{
    Layout = "_Layout";
}
```

8.2 Views/Shared/_Layout.cshtml — la plantilla común

Views/Shared/_Layout.cshtml

```
@inject IConfiguration config
@{
    var nombreTienda = config["Tienda:Nombre"] ?? "Ferretería";
    var logueado = User.Identity?.IsAuthenticated == true;
    var esAdmin = logueado && User.IsInRole("Admin");
    var esPersonal = logueado && (esAdmin || User.IsInRole("Empleado"));
    var esCliente = logueado && User.IsInRole("Cliente");

    // Cantidad de productos en el carrito (vive en la sesión) para el numerito de la barra
    var cantidadCarrito = esPersonal ? 0 : CarritoSesion.Cantidad(Context.Session);
}
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - @nombreTienda</title>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
</head>
<body class="d-flex flex-column min-vh-100">
    <header>
        <nav class="navbar navbar-expand-lg navbar-dark bg-dark border-bottom border-warning border-3">
            <div class="container">
                <a class="navbar-brand fw-bold" asp-controller="Home" asp-action="Index">
                    <i class="bi bi-tools text-warning"></i> @nombreTienda
                </a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarPrincipal"
                    aria-controls="navbarPrincipal" aria-expanded="false" aria-label="Menú">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse" id="navbarPrincipal">
                    <ul class="navbar-nav me-auto">
                        <li class="nav-item">
                            <a class="nav-link" asp-controller="Home" asp-action="Index">Inicio</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" asp-controller="Productos" asp-action="Index">Catálogo</a>
                        </li>
                    </ul>
                </div>
            </div>
        </nav>
    </header>
```

```

        </li>
        @if (esCliente)
        {
            <li class="nav-item">
                <a class="nav-link" asp-controller="Ventas" asp-action="MisCompras">Mis compras</a>
            </li>
        }
        @if (esPersonal)
        {
            <li class="nav-item dropdown">
                <a class="nav-link dropdown-toggle text-warning" href="#" role="button" data-bs-toggle="dropdown"
aria-expanded="false">
                    <i class="bi bi-briefcase"></i> Gestión
                </a>
                <ul class="dropdown-menu">
                    <li><a class="dropdown-item" asp-controller="Admin" asp-action="Index"><i class="bi bi-
speedometer2"></i> Panel general</a></li>
                    <li><hr class="dropdown-divider"></li>
                    <li><a class="dropdown-item" asp-controller="Ventas" asp-action="Registrar"><i class="bi bi-cash-
coin"></i> Registrar venta</a></li>
                    <li><a class="dropdown-item" asp-controller="Ventas" asp-action="Manage"><i class="bi bi-
receipt"></i> Ventas</a></li>
                    <li><a class="dropdown-item" asp-controller="Compras" asp-action="Index"><i class="bi bi-
truck"></i> Compras a proveedores</a></li>
                    <li><a class="dropdown-item" asp-controller="Inventario" asp-action="Index"><i class="bi bi-
clipboard-data"></i> Inventario</a></li>
                    <li><a class="dropdown-item" asp-controller="Productos" asp-action="Manage"><i class="bi bi-box-
seam"></i> Productos</a></li>
                    @if (esAdmin)
                    {
                        <li><hr class="dropdown-divider"></li>
                        <li><a class="dropdown-item" asp-controller="Categorias" asp-action="Index"><i class="bi bi-
tags"></i> Categorías</a></li>
                        <li><a class="dropdown-item" asp-controller="Proveedores" asp-action="Index"><i class="bi bi-
building"></i> Proveedores</a></li>
                        <li><a class="dropdown-item" asp-controller="FormasPago" asp-action="Index"><i class="bi bi-
credit-card"></i> Formas de pago</a></li>
                        <li><a class="dropdown-item" asp-controller="Admin" asp-action="Usuarios"><i class="bi bi-
people"></i> Usuarios y empleados</a></li>
                    }
                </ul>
            </li>
        }
    </ul>

    <form class="d-flex me-lg-3 my-2 my-lg-0" asp-controller="Productos" asp-action="Index" method="get"
role="search">
        <input class="form-control form-control-sm me-2" type="search" name="buscar" placeholder="Buscar
producto..." value="@ViewBag.Buscar" aria-label="Buscar" />
        <button class="btn btn-sm btn-outline-light" type="submit"><i class="bi bi-search"></i></button>
    </form>

    <ul class="navbar-nav align-items-center">
        @if (!esPersonal)
        {
            <li class="nav-item">
                <a class="nav-link position-relative" asp-controller="Carrito" asp-action="Index" title="Carrito">
                    <i class="bi bi-cart3 fs-5"></i>
                    @if (cantidadCarrito > 0)
                    {
                        <span class="badge rounded-pill bg-warning text-dark badge-carrito">@cantidadCarrito</span>
                    }
                </a>
            </li>
        }
    </ul>

```

```

    </li>
  }
  @if (logueado)
  {
    <li class="nav-item dropdown">
      <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown" aria-
expanded="false">
        <i class="bi bi-person-circle"></i> @(User.Identity?.Name)
      </a>
      <ul class="dropdown-menu dropdown-menu-end">
        <li><a class="dropdown-item" asp-controller="Account" asp-action="Perfil"><i class="bi bi-
person"></i> Mi perfil</a></li>
        <li><a class="dropdown-item" asp-controller="Account" asp-action="CambiarPassword"><i
class="bi bi-key"></i> Cambiar contraseña</a></li>
        @if (esCliente)
        {
          <li><a class="dropdown-item" asp-controller="Ventas" asp-action="MisCompras"><i class="bi bi-
bag"></i> Mis compras</a></li>
        }
        <li><hr class="dropdown-divider"></li>
        <li>
          <form asp-controller="Account" asp-action="Logout" method="post">
            <button type="submit" class="dropdown-item"><i class="bi bi-box-arrow-right"></i> Cerrar
sesión</button>
          </form>
        </li>
      </ul>
    </li>
  }
  else
  {
    <li class="nav-item">
      <a class="nav-link" asp-controller="Account" asp-action="Login">Ingresar</a>
    </li>
    <li class="nav-item">
      <a class="btn btn-warning btn-sm ms-lg-2" asp-controller="Account" asp-
action="Register">Registrarse</a>
    </li>
  }
</ul>
</div>
</div>
</nav>
</header>

<div class="container flex-grow-1">
  <main role="main" class="py-4">
    <partial name="_Alertas" />
    @RenderBody()
  </main>
</div>

<footer class="bg-dark text-light py-3 mt-auto">
  <div class="container d-flex flex-wrap justify-content-between align-items-center small">
    <span>&copy; @DateTime.Now.Year - @nombreTienda</span>
    <span><i class="bi bi-geo-alt"></i> Av. Principal 123 &nbsp; &nbsp; <i class="bi bi-telephone"></i> 700-00000
&nbsp; &nbsp; &nbsp; Lun a Sáb 8:00 - 19:00</span>
  </div>
</footer>

<script src="~/lib/jquery/dist/jquery.min.js"></script>
<script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
<script src="~/js/site.js" asp-append-version="true"></script>

```

```
@await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

- Con `User.IsInRole(...)` se arma el menú según el rol: **Gestión** para el personal (con las opciones de Admin al final), **Mis compras** y el carrito para clientes y visitantes.
- El numerito del carrito sale de la sesión (`CarritoSesion.Cantidad`), sin consultar la base.
- El "Cerrar sesión" es un `<form method="post">` porque Logout es [HttpPost].

8.3 Views/Shared/_Alertas.cshtml, _ProductoCard.cshtml y Error.cshtml

Views/Shared/_Alertas.cshtml

```
@* Mensajes que los controladores dejan en TempData (se muestran una sola vez) *@
@if (TempData["Exito"] != null)
{
    <div class="alert alert-success alert-dismissible fade show" role="alert">
        <i class="bi bi-check-circle-fill"></i> @TempData["Exito"]
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Cerrar"></button>
    </div>
}
@if (TempData["Error"] != null)
{
    <div class="alert alert-danger alert-dismissible fade show" role="alert">
        <i class="bi bi-exclamation-triangle-fill"></i> @TempData["Error"]
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Cerrar"></button>
    </div>
}
@if (TempData["Info"] != null)
{
    <div class="alert alert-info alert-dismissible fade show" role="alert">
        <i class="bi bi-info-circle-fill"></i> @TempData["Info"]
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Cerrar"></button>
    </div>
}
```

Views/Shared/_ProductoCard.cshtml

```
@model Producto
@* Tarjeta de producto reutilizada en Inicio, Catálogo y relacionados *@
<div class="card h-100 producto-card shadow-sm">
    <a asp-controller="Productos" asp-action="Details" asp-route-id="@Model.IdProducto" class="producto-img-wrap">
        
    </a>
    <div class="card-body d-flex flex-column">
        @if (Model.Categoria != null)
        {
            <small class="text-muted text-uppercase">@Model.Categoria.Nombre</small>
        }
        <h6 class="card-title mb-1">
            <a asp-controller="Productos" asp-action="Details" asp-route-id="@Model.IdProducto" class="text-decoration-
none text-dark">@Model.Nombre</a>
        </h6>
        <div class="mt-auto pt-2 d-flex justify-content-between align-items-center">
            <span class="fw-bold fs-5 text-precio">@Formato.Precio(Model.PrecioVenta)</span>
            @if (Model.Stock > 0)
            {
                <small class="text-success"><i class="bi bi-check-circle"></i> Stock: @Model.Stock</small>
            }
        </div>
    </div>
</div>
```

```

    }
    else
    {
        <small class="text-danger"><i class="bi bi-x-circle"></i> Sin stock</small>
    }
</div>
</div>
<div class="card-footer bg-white border-0 pt-0">
    @if (User.EsPersonal())
    {
        <a asp-controller="Productos" asp-action="Edit" asp-route-id="@Model.IdProducto" class="btn btn-outline-dark
btn-sm w-100"><i class="bi bi-pencil"></i> Editar</a>
    }
    else if (Model.Stock > 0)
    {
        <form asp-controller="Carrito" asp-action="Agregar" method="post" class="d-grid">
            <input type="hidden" name="idProducto" value="@Model.IdProducto" />
            <input type="hidden" name="cantidad" value="1" />
            <input type="hidden" name="returnUrl" value="@Context.Request.Path@Context.Request.QueryString" />
            <button type="submit" class="btn btn-warning btn-sm"><i class="bi bi-cart-plus"></i> Agregar al
carrito</button>
        </form>
    }
    else
    {
        <button class="btn btn-secondary btn-sm w-100" disabled>No disponible</button>
    }
</div>
</div>

```

Views/Shared/Error.cshtml

```

@model ErrorViewModel
@{
    ViewData["Title"] = "Error";
}

<div class="text-center py-5">
    <i class="bi bi-exclamation-octagon text-danger" style="font-size: 4rem;"></i>
    <h1 class="text-danger mt-3">Ocurrió un error</h1>
    <p class="lead">No se pudo procesar tu solicitud. Intentá nuevamente más tarde.</p>
    @if (Model.ShowRequestId)
    {
        <p class="text-muted small"><strong>ID de solicitud:</strong> <code>@Model.RequestId</code></p>
    }
    <a asp-controller="Home" asp-action="Index" class="btn btn-warning mt-3">Volver al inicio</a>
</div>

```

8.4 Views/Home/Index.cshtml — portada

Views/Home/Index.cshtml

```

@model List<Producto>
@inject IConfiguration config
@{
    ViewData["Title"] = "Inicio";
    var categorias = (List<Categoria>)ViewBag.Categorias;
}

<section class="hero p-4 p-md-5 mb-4">
    <div class="row align-items-center">

```

```

<div class="col-md-8">
  <h1 class="display-5 fw-bold">@config["Tienda:Nombre"]</h1>
  <p class="lead mb-4">Todo para tu obra, tu taller y tu hogar: herramientas eléctricas y manuales, materiales de construcción, plomería, electricidad, pintura y jardinería.</p>
  <a asp-controller="Productos" asp-action="Index" class="btn btn-warning btn-lg"><i class="bi bi-grid"></i> Ver catálogo</a>
  @if (User.Identity?.IsAuthenticated != true)
  {
    <a asp-controller="Account" asp-action="Register" class="btn btn-outline-light btn-lg ms-2">Crear cuenta</a>
  }
</div>
<div class="col-md-4 text-center d-none d-md-block">
  <i class="bi bi-tools" style="font-size: 9rem; opacity: .85;"></i>
</div>
</div>
</section>

<h4 class="mb-3"><i class="bi bi-tags"></i> Categorías</h4>
<div class="d-flex flex-wrap gap-2 mb-5">
  @foreach (var c in categorias)
  {
    <a asp-controller="Productos" asp-action="Index" asp-route-categoriaId="@c.IdCategoria" class="categoria-chip">
      @c.Nombre <span class="badge bg-secondary rounded-pill">@c.Productos.Count</span>
    </a>
  }
</div>

<div class="d-flex justify-content-between align-items-center mb-3">
  <h4 class="mb-0"><i class="bi bi-star"></i> Productos destacados</h4>
  <a asp-controller="Productos" asp-action="Index" class="btn btn-sm btn-outline-dark">Ver todos <i class="bi bi-arrow-right"></i></a>
</div>
<div class="row row-cols-1 row-cols-sm-2 row-cols-md-3 row-cols-lg-4 g-3 mb-4">
  @foreach (var producto in Model)
  {
    <div class="col">
      <partial name="_ProductoCard" model="producto" />
    </div>
  }
</div>

```

8.5 Vistas de cuenta (Views/Account/)

Views/Account/Login.cshtml

```

@model LoginViewModel
@{
  ViewData["Title"] = "Iniciar sesión";
}

<div class="form-auth">
  <div class="card shadow-sm">
    <div class="card-body p-4">
      <h2 class="mb-4 text-center"><i class="bi bi-box-arrow-in-right"></i> Iniciar sesión</h2>
      <form asp-action="Login" asp-route-returnUrl="@ViewBag.ReturnUrl" method="post">
        <div asp-validation-summary="ModelOnly" class="text-danger"></div>
        <div class="form-group mb-3">
          <label asp-for="Email"></label>
          <input asp-for="Email" class="form-control" autofocus />
          <span asp-validation-for="Email" class="text-danger"></span>
        </div>
        <div class="form-group mb-3">

```

```

        <label asp-for="Password"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
    </div>
    <div class="form-group form-check mb-3">
        <input asp-for="RememberMe" class="form-check-input" />
        <label asp-for="RememberMe" class="form-check-label"></label>
    </div>
    <button type="submit" class="btn btn-warning w-100">Iniciar sesión</button>
</form>
<p class="text-center mt-3 mb-0">
    ¿No tienes cuenta? <a asp-action="Register">Registrate</a>
</p>
</div>
</div>
</div>

```

```
@section Scripts { <partial name="_ValidationScriptsPartial" /> }
```

Views/Account/Register.cshtml

```

@model RegistroViewModel
@{
    ViewData["Title"] = "Registro";
}

<div class="form-auth">
    <div class="card shadow-sm">
        <div class="card-body p-4">
            <h2 class="mb-4 text-center"><i class="bi bi-person-plus"></i> Crear cuenta de cliente</h2>
            <form asp-action="Register" method="post">
                <div asp-validation-summary="ModelOnly" class="text-danger"></div>
                <div class="row">
                    <div class="col-md-6 form-group mb-3">
                        <label asp-for="Nombre"></label>
                        <input asp-for="Nombre" class="form-control" autofocus />
                        <span asp-validation-for="Nombre" class="text-danger"></span>
                    </div>
                    <div class="col-md-6 form-group mb-3">
                        <label asp-for="Apellido"></label>
                        <input asp-for="Apellido" class="form-control" />
                        <span asp-validation-for="Apellido" class="text-danger"></span>
                    </div>
                </div>
                <div class="form-group mb-3">
                    <label asp-for="Email"></label>
                    <input asp-for="Email" class="form-control" />
                    <span asp-validation-for="Email" class="text-danger"></span>
                </div>
                <div class="row">
                    <div class="col-md-6 form-group mb-3">
                        <label asp-for="Password"></label>
                        <input asp-for="Password" class="form-control" />
                        <span asp-validation-for="Password" class="text-danger"></span>
                        <small class="text-muted">Mínimo 8 caracteres, con mayúsculas, minúsculas y números.</small>
                    </div>
                    <div class="col-md-6 form-group mb-3">
                        <label asp-for="ConfirmPassword"></label>
                        <input asp-for="ConfirmPassword" class="form-control" />
                        <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
                    </div>
                </div>
            </form>
        </div>
    </div>

```

```

<div class="form-group mb-3">
  <label asp-for="Telefono"></label>
  <input asp-for="Telefono" class="form-control" placeholder="Solo números, 8 a 15 dígitos" />
  <span asp-validation-for="Telefono" class="text-danger"></span>
</div>
<div class="form-group mb-3">
  <label asp-for="Direccion"></label>
  <input asp-for="Direccion" class="form-control" placeholder="Se usará como dirección de entrega" />
  <span asp-validation-for="Direccion" class="text-danger"></span>
</div>
<button type="submit" class="btn btn-warning w-100">Registrarse</button>
</form>
<p class="text-center mt-3 mb-0">
  ¿Ya tenés cuenta? <a asp-action="Login">Iniciá sesión</a>
</p>
</div>
</div>
</div>
@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

Views/Account/Perfil.cshtml

```

@model PerfilViewModel
@{
  ViewData["Title"] = "Mi perfil";
}

<div class="form-auth">
  <div class="card shadow-sm">
    <div class="card-body p-4">
      <h2 class="mb-1"><i class="bi bi-person"></i> Mi perfil</h2>
      <p class="text-muted">
        @Model.TipoUsuario
        @if (!string.IsNullOrEmpty(Model.Cargo)) { <span>· @Model.Cargo</span> }
        @if (!string.IsNullOrEmpty(Model.TipoCliente)) { <span>· @Model.TipoCliente</span> }
      </p>
      <form asp-action="Perfil" method="post">
        <div asp-validation-summary="ModelOnly" class="text-danger"></div>
        <input type="hidden" asp-for="TipoUsuario" />
        <input type="hidden" asp-for="Cargo" />
        <input type="hidden" asp-for="TipoCliente" />
        <div class="form-group mb-3">
          <label asp-for="Email"></label>
          <input asp-for="Email" class="form-control" readonly />
        </div>
        <div class="row">
          <div class="col-md-6 form-group mb-3">
            <label asp-for="Nombre"></label>
            <input asp-for="Nombre" class="form-control" />
            <span asp-validation-for="Nombre" class="text-danger"></span>
          </div>
          <div class="col-md-6 form-group mb-3">
            <label asp-for="Apellido"></label>
            <input asp-for="Apellido" class="form-control" />
            <span asp-validation-for="Apellido" class="text-danger"></span>
          </div>
        </div>
        <div class="form-group mb-3">
          <label asp-for="Telefono"></label>
          <input asp-for="Telefono" class="form-control" />
          <span asp-validation-for="Telefono" class="text-danger"></span>
        </div>
      </form>
    </div>
  </div>
</div>

```

```

        </div>
        <div class="form-group mb-3">
            <label asp-for="Direccion"></label>
            <input asp-for="Direccion" class="form-control" />
            <span asp-validation-for="Direccion" class="text-danger"></span>
        </div>
        <button type="submit" class="btn btn-warning">Guardar cambios</button>
        <a asp-action="CambiarPassword" class="btn btn-outline-secondary"><i class="bi bi-key"></i> Cambiar
contraseña</a>
    </form>
</div>
</div>
</div>
@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

Views/Account/CambiarPassword.cshtml

```

@model CambiarPasswordViewModel
@{
    ViewData["Title"] = "Cambiar contraseña";
}

<div class="form-auth">
    <div class="card shadow-sm">
        <div class="card-body p-4">
            <h2 class="mb-4"><i class="bi bi-key"></i> Cambiar contraseña</h2>
            <form asp-action="CambiarPassword" method="post">
                <div asp-validation-summary="ModelOnly" class="text-danger"></div>
                <div class="form-group mb-3">
                    <label asp-for="PasswordActual"></label>
                    <input asp-for="PasswordActual" class="form-control" autofocus />
                    <span asp-validation-for="PasswordActual" class="text-danger"></span>
                </div>
                <div class="form-group mb-3">
                    <label asp-for="PasswordNueva"></label>
                    <input asp-for="PasswordNueva" class="form-control" />
                    <span asp-validation-for="PasswordNueva" class="text-danger"></span>
                    <small class="text-muted">Mínimo 8 caracteres, con mayúsculas, minúsculas y números.</small>
                </div>
                <div class="form-group mb-3">
                    <label asp-for="ConfirmarPassword"></label>
                    <input asp-for="ConfirmarPassword" class="form-control" />
                    <span asp-validation-for="ConfirmarPassword" class="text-danger"></span>
                </div>
                <button type="submit" class="btn btn-warning">Cambiar contraseña</button>
                <a asp-action="Perfil" class="btn btn-outline-secondary">Cancelar</a>
            </form>
        </div>
    </div>
</div>
@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

Views/Account/AccessDenied.cshtml

```

@{
    ViewData["Title"] = "Acceso denegado";
}

<div class="text-center py-5">

```

```

<i class="bi bi-shield-lock text-danger" style="font-size: 4rem;"></i>
<h2 class="mt-3">Acceso denegado</h2>
<p class="lead">No tenés permisos para ver esta página.</p>
<a asp-controller="Home" asp-action="Index" class="btn btn-warning">Volver al inicio</a>
</div>

```

8.6 Vistas de productos (Views/Productos/)

Index.cshtml — catálogo con filtros y paginación

Views/Productos/Index.cshtml

```

@model List<Producto>
@{
    ViewData["Title"] = "Catálogo";
    var categorias = (List<Categoria>)ViewBag.Categorias;
    int? categoriaId = ViewBag.CategoriaId;
    string? buscar = ViewBag.Buscar;
    int pagina = ViewBag.Pagina;
    int totalPaginas = ViewBag.TotalPaginas;
    int total = ViewBag.Total;
    var categoriaActual = categorias.FirstOrDefault(c => c.IdCategoria == categoriaId);
}

<div class="d-flex flex-wrap justify-content-between align-items-center mb-3">
    <h2 class="mb-0">
        <i class="bi bi-grid"></i>
        @(categoriaActual?.Nombre ?? "Catálogo")
        <small class="text-muted fs-6">(@total producto@(total == 1 ? "" : "s"))</small>
    </h2>
    @if (User.IsPersonal())
    {
        <div>
            <a asp-action="Create" class="btn btn-warning"><i class="bi bi-plus-lg"></i> Crear producto</a>
            <a asp-action="Manage" class="btn btn-outline-dark"><i class="bi bi-table"></i> Administrar</a>
        </div>
    }
</div>

<div class="row g-4">
    <aside class="col-lg-3">
        <div class="card shadow-sm">
            <div class="card-body">
                <form asp-action="Index" method="get">
                    <label class="form-label fw-bold">Buscar</label>
                    <div class="input-group mb-3">
                        <input type="search" name="buscar" class="form-control" placeholder="Nombre, descripción..."
value="@buscar" />
                        <button class="btn btn-warning" type="submit"><i class="bi bi-search"></i></button>
                    </div>
                    @if (categoriaId.HasValue)
                    {
                        <input type="hidden" name="categoriaId" value="@categoriaId" />
                    }
                </form>

                <label class="form-label fw-bold">Categorías</label>
                <div class="list-group list-group-flush">
                    <a asp-action="Index" asp-route-buscar="@buscar"
class="list-group-item list-group-item-action @(categoriaId == null ? "active" : "")">Todas</a>
                    @foreach (var c in categorias)
                    {

```

```

        <a asp-action="Index" asp-route-categoriaId="@c.IdCategoria" asp-route-buscar="@buscar"
        class="list-group-item list-group-item-action @(categoriaId == c.IdCategoria ? "active" :
        "")">@c.Nombre</a>
    }
</div>
</div>
</div>
</aside>

<div class="col-lg-9">
    @if (!string.IsNullOrEmpty(buscar))
    {
        <p class="text-muted">
            Resultados para "<strong>@buscar</strong>"
            <a asp-action="Index" asp-route-categoriaId="@categoriaId" class="ms-2 small">Limpiar búsqueda</a>
        </p>
    }

    @if (Model.Count == 0)
    {
        <div class="alert alert-light border text-center py-5">
            <i class="bi bi-search fs-1 text-muted"></i>
            <p class="mb-0 mt-2">No se encontraron productos.</p>
        </div>
    }
    else
    {
        <div class="row row-cols-1 row-cols-sm-2 row-cols-xl-3 g-3">
            @foreach (var producto in Model)
            {
                <div class="col">
                    <partial name="_ProductoCard" model="producto" />
                </div>
            }
        </div>

        @if (totalPaginas > 1)
        {
            <nav class="mt-4" aria-label="Paginación">
                <ul class="pagination justify-content-center">
                    <li class="page-item @(pagina == 1 ? "disabled" : "")">
                        <a class="page-link" asp-action="Index" asp-route-pagina="@(pagina - 1)" asp-route-buscar="@buscar"
                        asp-route-categoriaId="@categoriaId">&laquo;</a>
                    </li>
                    @for (int i = 1; i <= totalPaginas; i++)
                    {
                        <li class="page-item @(i == pagina ? "active" : "")">
                            <a class="page-link" asp-action="Index" asp-route-pagina="@i" asp-route-buscar="@buscar" asp-
                            route-categoriaId="@categoriaId">@i</a>
                        </li>
                    }
                    <li class="page-item @(pagina == totalPaginas ? "disabled" : "")">
                        <a class="page-link" asp-action="Index" asp-route-pagina="@(pagina + 1)" asp-route-buscar="@buscar"
                        asp-route-categoriaId="@categoriaId">&raquo;</a>
                    </li>
                </ul>
            </nav>
        }
    }
</div>
</div>

```

Details.cshtml

Views/Productos/Details.cshtml

```
@model Producto
@{
    ViewData["Title"] = Model.Nombre;
    var relacionados = (List<Producto>)ViewBag.Relacionados;
}

<nav aria-label="breadcrumb">
    <ol class="breadcrumb">
        <li class="breadcrumb-item"><a asp-controller="Home" asp-action="Index">Inicio</a></li>
        <li class="breadcrumb-item"><a asp-action="Index">Catálogo</a></li>
        @if (Model.Categoria != null)
        {
            <li class="breadcrumb-item"><a asp-action="Index" asp-route-
categoriaId="@Model.IdCategoria">@Model.Categoria.Nombre</a></li>
        }
        <li class="breadcrumb-item active" aria-current="page">@Model.Nombre</li>
    </ol>
</nav>

<div class="card shadow-sm mb-4">
    <div class="card-body p-4">
        <div class="row g-4">
            <div class="col-md-5">
                
            </div>
            <div class="col-md-7">
                @if (!Model.Estado)
                {
                    <span class="badge bg-secondary mb-2">Producto inactivo (solo visible para el personal)</span>
                }
                <h2>@Model.Nombre</h2>
                <p class="text-muted mb-1">
                    @if (Model.Categoria != null)
                    {
                        <span><strong>Categoría:</strong> @Model.Categoria.Nombre</span>
                    }
                    @if (Model.Proveedor != null && User.EsPersonal())
                    {
                        <span class="ms-3"><strong>Proveedor:</strong> @Model.Proveedor.Nombre</span>
                    }
                </p>
                <p class="display-6 text-precio fw-bold my-3">@Formato.Precio(Model.PrecioVenta)</p>
                @if (User.EsPersonal())
                {
                    <p class="text-muted small mb-2">Precio de compra: @Formato.Precio(Model.PrecioCompra) · Stock
mínimo: @Model.StockMinimo</p>
                }
                <p>@Model.Descripcion</p>

                @if (Model.Stock > 0)
                {
                    <p class="text-success"><i class="bi bi-check-circle-fill"></i> Disponible: <strong>@Model.Stock</strong>
unidades</p>
                    @if (!User.EsPersonal())
                    {
                        <form asp-controller="Carrito" asp-action="Agregar" method="post" class="row g-2 align-items-center">
                            <input type="hidden" name="idProducto" value="@Model.IdProducto" />
                            <input type="hidden" name="returnUrl" value="@Context.Request.Path" />
```

```

        <div class="col-auto">
            <label for="cantidad" class="col-form-label">Cantidad</label>
        </div>
        <div class="col-auto">
            <input type="number" id="cantidad" name="cantidad" class="form-control" value="1" min="1"
max="@Model.Stock" style="width: 90px" />
        </div>
        <div class="col-auto">
            <button type="submit" class="btn btn-warning"><i class="bi bi-cart-plus"></i> Agregar al
carrito</button>
        </div>
    </form>
}
}
else
{
    <p class="text-danger"><i class="bi bi-x-circle-fill"></i> Sin stock por el momento</p>
}

    <div class="mt-4">
        <a asp-action="Index" class="btn btn-outline-secondary"><i class="bi bi-arrow-left"></i> Volver al
catálogo</a>
        @if (User.EsPersonal())
        {
            <a asp-action="Edit" asp-route-id="@Model.IdProducto" class="btn btn-outline-primary"><i class="bi bi-
pencil"></i> Editar</a>
        }
        @if (User.IsInRole("Admin"))
        {
            <a asp-action="Delete" asp-route-id="@Model.IdProducto" class="btn btn-outline-danger"><i class="bi bi-
trash"></i> Eliminar</a>
        }
    </div>
</div>
</div>
</div>
</div>
</div>

@if (relacionados.Count > 0)
{
    <h5 class="mb-3">Productos relacionados</h5>
    <div class="row row-cols-1 row-cols-sm-2 row-cols-md-4 g-3">
        @foreach (var p in relacionados)
        {
            <div class="col">
                <partial name="_ProductoCard" model="p" />
            </div>
        }
    </div>
}
}

```

Manage.cshtml — tabla de administración

Views/Productos/Manage.cshtml

```

@model List<Producto>
@{
    ViewData["Title"] = "Administrar productos";
    string? buscar = ViewBag.Buscar;
}

<div class="d-flex flex-wrap justify-content-between align-items-center mb-3">
    <h2 class="mb-0"><i class="bi bi-box-seam"></i> Productos <small class="text-muted

```

```

fs-6">(@Model.Count)</small></h2>
<div class="d-flex gap-2">
  <form asp-action="Manage" method="get" class="d-flex">
    <input type="search" name="buscar" class="form-control form-control-sm me-2" placeholder="Buscar..."
value="@buscar" />
    <button class="btn btn-sm btn-outline-dark" type="submit"><i class="bi bi-search"></i></button>
  </form>
  <a asp-action="Create" class="btn btn-sm btn-warning"><i class="bi bi-plus-lg"></i> Nuevo producto</a>
</div>
</div>

<div class="card shadow-sm">
  <div class="table-responsive">
    <table class="table table-hover align-middle mb-0">
      <thead class="table-dark">
        <tr>
          <th></th>
          <th>Nombre</th>
          <th>Categoría</th>
          <th>Proveedor</th>
          <th class="text-end">P. compra</th>
          <th class="text-end">P. venta</th>
          <th class="text-center">Stock</th>
          <th class="text-center">Estado</th>
          <th class="text-end">Acciones</th>
        </tr>
      </thead>
      <tbody>
        @foreach (var p in Model)
        {
          <tr class="@p.Estado ? "" : "table-secondary">
            <td></td>
            <td><a asp-action="Details" asp-route-id="@p.IdProducto"
class="text-decoration-none">@p.Nombre</a></td>
            <td>@p.Categoria?.Nombre</td>
            <td class="small">@p.Proveedor?.Nombre</td>
            <td class="text-end text-muted">@Formato.Precio(p.PrecioCompra)</td>
            <td class="text-end">@Formato.Precio(p.PrecioVenta)</td>
            <td class="text-center">
              @if (p.Stock == 0)
              {
                <span class="badge bg-danger">0</span>
              }
              else if (!p.VerificarStock())
              {
                <span class="badge bg-warning text-dark" title="En o por debajo del mínimo
(@p.StockMinimo)">@p.Stock</span>
              }
              else
              {
                <span class="text-success">@p.Stock</span>
              }
            </td>
            <td class="text-center">
              @if (p.Estado)
              {
                <span class="badge bg-success">Activo</span>
              }
              else
              {
                <span class="badge bg-secondary">Inactivo</span>
              }
            </td>
          </tr>
        }
      </tbody>
    </table>
  </div>
</div>

```

```
 <i class="bi bi-pencil"></i></a> @if (User.IsInRole("Admin")) { <i class="bi bi-trash"></i></a> } </td> </tr> } @if (Model.Count == 0) { |<td colspan="9" class="text-center text-muted py-4">No hay productos.</td></tr> } </tbody> </table> </div> </div>  |  | |
```

_Form.cshtml, Create.cshtml, Edit.cshtml, Delete.cshtml

Para no repetir el formulario dos veces, los campos están en una vista parcial. El `<input type="file">` requiere que el `<form>` tenga `enctype="multipart/form-data"`.

Views/Productos/_Form.cshtml

```

@model Producto
@* Campos compartidos por Crear y Editar *@
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="form-group mb-3">
<label asp-for="Nombre"></label>
<input asp-for="Nombre" class="form-control" />
<span asp-validation-for="Nombre" class="text-danger"></span>
</div>
<div class="form-group mb-3">
<label asp-for="Descripcion"></label>
<textarea asp-for="Descripcion" class="form-control" rows="4"></textarea>
<span asp-validation-for="Descripcion" class="text-danger"></span>
</div>
<div class="row">
<div class="col-md-6 form-group mb-3">
<label asp-for="IdCategoria"></label>
<select asp-for="IdCategoria" asp-items="ViewBag.Categorias" class="form-select">
<option value="">-- Elegir categoría --</option>
</select>
<span asp-validation-for="IdCategoria" class="text-danger"></span>
</div>
<div class="col-md-6 form-group mb-3">
<label asp-for="IdProveedor"></label>
<select asp-for="IdProveedor" asp-items="ViewBag.Proveedores" class="form-select">
<option value="">-- Elegir proveedor --</option>
</select>
<span asp-validation-for="IdProveedor" class="text-danger"></span>
</div>
</div>
<div class="row">
<div class="col-md-3 form-group mb-3">
<label asp-for="PrecioCompra"></label>
<div class="input-group">
<span class="input-group-text">@Formato.Moneda</span>

```

```

        <input asp-for="PrecioCompra" class="form-control" type="number" step="0.01" min="0.01" />
    </div>
    <span asp-validation-for="PrecioCompra" class="text-danger"></span>
</div>
<div class="col-md-3 form-group mb-3">
    <label asp-for="PrecioVenta"></label>
    <div class="input-group">
        <span class="input-group-text">@Formato.Moneda</span>
        <input asp-for="PrecioVenta" class="form-control" type="number" step="0.01" min="0.01" />
    </div>
    <span asp-validation-for="PrecioVenta" class="text-danger"></span>
</div>
<div class="col-md-3 form-group mb-3">
    <label asp-for="Stock"></label>
    <input asp-for="Stock" class="form-control" type="number" min="0" />
    <span asp-validation-for="Stock" class="text-danger"></span>
</div>
<div class="col-md-3 form-group mb-3">
    <label asp-for="StockMinimo"></label>
    <input asp-for="StockMinimo" class="form-control" type="number" min="0" />
    <span asp-validation-for="StockMinimo" class="text-danger"></span>
</div>
</div>
<div class="row">
    <div class="col-md-6 form-group mb-3">
        <label asp-for="ImagenArchivo"></label>
        <input asp-for="ImagenArchivo" class="form-control" type="file" accept=".png,.jpg,.jpeg,.webp" />
        <span asp-validation-for="ImagenArchivo" class="text-danger"></span>
        <small class="text-muted">PNG, JPG o WEBP, hasta 5 MB.</small>
    </div>
    <div class="col-md-6 form-group mb-3">
        <label asp-for="ImagenUrl"></label>
        <input asp-for="ImagenUrl" class="form-control" placeholder="O pegá la URL de una imagen" />
        <small class="text-muted">Si subís un archivo, este campo se ignora.</small>
    </div>
</div>
<div class="form-group form-check mb-3">
    <input asp-for="Estado" class="form-check-input" />
    <label asp-for="Estado" class="form-check-label"></label>
</div>

```

Views/Productos/Create.cshtml

```

@model Producto
@{
    ViewData["Title"] = "Crear producto";
}

<h2 class="mb-3"><i class="bi bi-plus-circle"></i> Crear producto</h2>

<div class="card shadow-sm">
    <div class="card-body p-4">
        <form asp-action="Create" method="post" enctype="multipart/form-data">
            <partial name="_Form" model="Model" />
            <button type="submit" class="btn btn-warning">Guardar</button>
            <a asp-action="Manage" class="btn btn-outline-secondary">Cancelar</a>
        </form>
    </div>
</div>

@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

```

@model Producto
@{
    ViewData["Title"] = "Editar producto";
}

<h2 class="mb-3"><i class="bi bi-pencil-square"></i> Editar producto</h2>

<div class="card shadow-sm">
    <div class="card-body p-4">
        <div class="row">
            <div class="col-md-3 text-center mb-3">
                <p class="text-muted small mb-1">Imagen actual</p>
                
            </div>
            <div class="col-md-9">
                <form asp-action="Edit" method="post" enctype="multipart/form-data">
                    <input type="hidden" asp-for="IdProducto" />
                    <partial name="_Form" model="Model" />
                    <button type="submit" class="btn btn-warning">Guardar cambios</button>
                    <a asp-action="Manage" class="btn btn-outline-secondary">Cancelar</a>
                </form>
            </div>
        </div>
    </div>
</div>

@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

```

@model Producto
@{
    ViewData["Title"] = "Eliminar producto";
    bool tieneMovimientos = ViewBag.TieneMovimientos;
}

<div class="form-auth">
    <div class="card shadow-sm border-danger">
        <div class="card-body p-4 text-center">
            <i class="bi bi-trash text-danger" style="font-size: 3rem;"></i>
            <h3 class="mt-2">¿Eliminar este producto?</h3>
            
            <h4>@Model.Nombre</h4>
            <p class="text-muted">@Model.Descripcion</p>
            <p><strong>Precio:</strong> @Formato.Precio(Model.PrecioVenta) &nbsp;<strong>Stock:</strong> @Model.Stock</p>

            @if (tieneMovimientos)
            {
                <div class="alert alert-warning small">
                    Este producto aparece en ventas o compras ya registradas. Para conservar el historial,
                    <strong>se desactivará</strong> en lugar de eliminarse.
                </div>
            }

            <form asp-action="Delete" method="post">
                <input type="hidden" asp-for="IdProducto" />
                <button type="submit" class="btn btn-danger">Sí, @(tieneMovimientos ? "desactivar" : "eliminar")</button>
            </form>
        </div>
    </div>
</div>

```

```

        <a asp-action="Manage" class="btn btn-outline-secondary">Cancelar</a>
    </form>
</div>
</div>
</div>

```

8.7 Catálogos de administración: Views/Categorias/, Views/Proveedores/, Views/FormasPago/

Las tres carpetas siguen el mismo patrón (lista, *_Form* compartido, crear, editar, eliminar). Se muestra completa la de categorías; proveedores y formas de pago son iguales cambiando los campos.

Views/Categorias/Index.cshtml

```

@model List<Categoria>
@{
    ViewData["Title"] = "Categorías";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h2 class="mb-0"><i class="bi bi-tags"></i> Categorías</h2>
    <a asp-action="Create" class="btn btn-warning"><i class="bi bi-plus-lg"></i> Nueva categoría</a>
</div>

<div class="card shadow-sm">
    <div class="table-responsive">
        <table class="table table-hover align-middle mb-0">
            <thead class="table-dark">
                <tr>
                    <th>Nombre</th>
                    <th>Descripción</th>
                    <th class="text-center">Productos</th>
                    <th class="text-center">Estado</th>
                    <th class="text-end">Acciones</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var c in Model)
                {
                    <tr class="@((c.Estado ? "" : "table-secondary"))">
                        <td class="fw-bold">@c.Nombre</td>
                        <td class="text-muted">@c.Descripcion</td>
                        <td class="text-center">
                            <a asp-controller="Productos" asp-action="Index" asp-route-categoriaId="@c.IdCategoria" class="badge bg-secondary text-decoration-none">@c.Productos.Count</a>
                        </td>
                        <td class="text-center"><span class="badge @(c.Estado ? "bg-success" : "bg-secondary")">@(c.Estado ? "Activa" : "Inactiva")</span></td>
                        <td class="text-end text-nowrap">
                            <a asp-action="Edit" asp-route-id="@c.IdCategoria" class="btn btn-sm btn-outline-primary" title="Editar"><i class="bi bi-pencil"></i></a>
                            <a asp-action="Delete" asp-route-id="@c.IdCategoria" class="btn btn-sm btn-outline-danger" title="Eliminar"><i class="bi bi-trash"></i></a>
                        </td>
                    </tr>
                }
                @if (Model.Count == 0)
                {
                    <tr><td colspan="5" class="text-center text-muted py-4">No hay categorías.</td></tr>
                }
            </tbody>
        </table>
    </div>

```

```
    </table>
  </div>
</div>
```

Views/Categorias/_Form.cshtml

```
@model Categoria
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="form-group mb-3">
  <label asp-for="Nombre"></label>
  <input asp-for="Nombre" class="form-control" autofocus />
  <span asp-validation-for="Nombre" class="text-danger"></span>
</div>
<div class="form-group mb-3">
  <label asp-for="Descripcion"></label>
  <textarea asp-for="Descripcion" class="form-control" rows="3"></textarea>
  <span asp-validation-for="Descripcion" class="text-danger"></span>
</div>
<div class="form-group form-check mb-3">
  <input asp-for="Estado" class="form-check-input" />
  <label asp-for="Estado" class="form-check-label"></label>
</div>
```

Views/Categorias/Create.cshtml

```
@model Categoria
@{
  ViewData["Title"] = "Crear categoría";
}

<div class="form-auth">
  <h2 class="mb-3"><i class="bi bi-plus-circle"></i> Crear categoría</h2>
  <div class="card shadow-sm">
    <div class="card-body p-4">
      <form asp-action="Create" method="post">
        <partial name="_Form" model="Model" />
        <button type="submit" class="btn btn-warning">Guardar</button>
        <a asp-action="Index" class="btn btn-outline-secondary">Cancelar</a>
      </form>
    </div>
  </div>
</div>

@section Scripts { <partial name="_ValidationScriptsPartial" /> }
```

Views/Categorias/Edit.cshtml

```
@model Categoria
@{
  ViewData["Title"] = "Editar categoría";
}

<div class="form-auth">
  <h2 class="mb-3"><i class="bi bi-pencil-square"></i> Editar categoría</h2>
  <div class="card shadow-sm">
    <div class="card-body p-4">
      <form asp-action="Edit" method="post">
        <input type="hidden" asp-for="IdCategoria" />
        <partial name="_Form" model="Model" />
        <button type="submit" class="btn btn-warning">Guardar cambios</button>
      </form>
    </div>
  </div>
</div>
```

```

        <a asp-action="Index" class="btn btn-outline-secondary">Cancelar</a>
    </form>
</div>
</div>
</div>

@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

Views/Categorias/Delete.cshtml

```

@model Categoria
@{
    ViewData["Title"] = "Eliminar categoría";
}

<div class="form-auth">
    <div class="card shadow-sm border-danger">
        <div class="card-body p-4 text-center">
            <i class="bi bi-trash text-danger" style="font-size: 3rem;"></i>
            <h3 class="mt-2">¿Eliminar esta categoría?</h3>
            <h4>@Model.Nombre</h4>
            <p class="text-muted">@Model.Descripcion</p>
            @if (Model.Productos.Count > 0)
            {
                <div class="alert alert-warning small">
                    Hay <strong>@Model.Productos.Count</strong> producto(s) en esta categoría y la categoría es obligatoria
                    en cada producto, así que <strong>se desactivará</strong> en lugar de eliminarse.
                </div>
            }
            <form asp-action="Delete" method="post">
                <input type="hidden" asp-for="IdCategoria" />
                <button type="submit" class="btn btn-danger">Sí, @(Model.Productos.Count > 0 ? "desactivar" :
"eliminar")</button>
                <a asp-action="Index" class="btn btn-outline-secondary">Cancelar</a>
            </form>
        </div>
    </div>
</div>
</div>

```

Proveedores (_Form.cshtml, con RUC/NIT, teléfono y dirección):

Views/Proveedores/_Form.cshtml

```

@model Proveedor
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="form-group mb-3">
    <label asp-for="Nombre"></label>
    <input asp-for="Nombre" class="form-control" autofocus />
    <span asp-validation-for="Nombre" class="text-danger"></span>
</div>
<div class="row">
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Ruc"></label>
        <input asp-for="Ruc" class="form-control" />
        <span asp-validation-for="Ruc" class="text-danger"></span>
    </div>
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Telefono"></label>
        <input asp-for="Telefono" class="form-control" placeholder="Solo números, 8 a 15 dígitos" />
        <span asp-validation-for="Telefono" class="text-danger"></span>
    </div>

```

```

</div>
<div class="form-group mb-3">
  <label asp-for="Direccion"></label>
  <input asp-for="Direccion" class="form-control" />
  <span asp-validation-for="Direccion" class="text-danger"></span>
</div>
<div class="form-group form-check mb-3">
  <input asp-for="Estado" class="form-check-input" />
  <label asp-for="Estado" class="form-check-label"></label>
</div>

```

Views/Proveedores/Index.cshtml

```

@model List<Proveedor>
@{
  ViewData["Title"] = "Proveedores";
}

<div class="d-flex justify-content-between align-items-center mb-3">
  <h2 class="mb-0"><i class="bi bi-building"></i> Proveedores</h2>
  <a asp-action="Create" class="btn btn-warning"><i class="bi bi-plus-lg"></i> Nuevo proveedor</a>
</div>

<div class="card shadow-sm">
  <div class="table-responsive">
    <table class="table table-hover align-middle mb-0">
      <thead class="table-dark">
        <tr>
          <th>Nombre</th>
          <th>RUC / NIT</th>
          <th>Teléfono</th>
          <th>Dirección</th>
          <th class="text-center">Productos</th>
          <th class="text-center">Compras</th>
          <th class="text-center">Estado</th>
          <th class="text-end">Acciones</th>
        </tr>
      </thead>
      <tbody>
        @foreach (var p in Model)
        {
          <tr class="@((p.Estado ? "" : "table-secondary"))">
            <td class="fw-bold">@p.Nombre</td>
            <td>@p.Ruc</td>
            <td>@p.Telefono</td>
            <td class="small">@p.Direccion</td>
            <td class="text-center"><span class="badge bg-secondary">@p.Productos.Count</span></td>
            <td class="text-center"><span class="badge bg-secondary">@p.Compras.Count</span></td>
            <td class="text-center"><span class="badge @(p.Estado ? "bg-success" : "bg-secondary")">@(p.Estado ?
"Activo" : "Inactivo")</span></td>
            <td class="text-end text-nowrap">
              <a asp-action="Edit" asp-route-id="@p.IdProveedor" class="btn btn-sm btn-outline-primary"
title="Editar"><i class="bi bi-pencil"></i></a>
              <a asp-action="Delete" asp-route-id="@p.IdProveedor" class="btn btn-sm btn-outline-danger"
title="Eliminar"><i class="bi bi-trash"></i></a>
            </td>
          </tr>
        }
      </tbody>
    </table>
    @if (Model.Count == 0)
    {
      <tr><td colspan="8" class="text-center text-muted py-4">No hay proveedores.</td></tr>
    }
  </div>

```

```

        </tbody>
    </table>
</div>
</div>

```

Formas de pago (_Form.cshtml e Index.cshtml):

Views/FormasPago/_Form.cshtml

```

@model FormaPago
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="form-group mb-3">
    <label asp-for="Nombre"></label>
    <input asp-for="Nombre" class="form-control" autofocus />
    <span asp-validation-for="Nombre" class="text-danger"></span>
</div>
<div class="form-group mb-3">
    <label asp-for="Descripcion"></label>
    <input asp-for="Descripcion" class="form-control" />
    <span asp-validation-for="Descripcion" class="text-danger"></span>
</div>
<div class="form-group form-check mb-3">
    <input asp-for="Estado" class="form-check-input" />
    <label asp-for="Estado" class="form-check-label"></label>
</div>

```

Views/FormasPago/Index.cshtml

```

@model List<FormaPago>
@{
    ViewData["Title"] = "Formas de pago";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h2 class="mb-0"><i class="bi bi-credit-card"></i> Formas de pago</h2>
    <a asp-action="Create" class="btn btn-warning"><i class="bi bi-plus-lg"></i> Nueva forma de pago</a>
</div>

<div class="card shadow-sm">
    <div class="table-responsive">
        <table class="table table-hover align-middle mb-0">
            <thead class="table-dark">
                <tr>
                    <th>Nombre</th>
                    <th>Descripción</th>
                    <th class="text-center">Ventas</th>
                    <th class="text-center">Estado</th>
                    <th class="text-end">Acciones</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var f in Model)
                {
                    <tr class="@f.Estado ? "" : "table-secondary">
                        <td class="fw-bold">@f.Nombre</td>
                        <td class="text-muted">@f.Descripcion</td>
                        <td class="text-center"><span class="badge bg-secondary">@f.Ventas.Count</span></td>
                        <td class="text-center"><span class="badge @f.Estado ? "bg-success" : "bg-secondary">@f.Estado ?
"Activa" : "Inactiva"></span></td>
                        <td class="text-end text-nowrap">
                            <a asp-action="Edit" asp-route-id="@f.IdFormaPago" class="btn btn-sm btn-outline-primary"

```



```

        {
            <br /><small class="text-danger">¡Stock insuficiente!</small>
        }
    </td>
    <td class="text-end">@Formato.Precio(item.PrecioUnitario)</td>
    <td>
        <form asp-action="Actualizar" method="post" class="input-group input-group-sm">
            <input type="hidden" name="idProducto" value="@item.IdProducto" />
            <input type="number" name="cantidad" value="@item.Cantidad" min="0" max="@disponible"
class="form-control text-center" />
            <button type="submit" class="btn btn-outline-secondary" title="Actualizar"><i class="bi bi-
arrow-repeat"></i></button>
        </form>
    </td>
    <td class="text-end fw-bold">@Formato.Precio(item.Subtotal)</td>
    <td class="text-end">
        <form asp-action="Quitar" method="post">
            <input type="hidden" name="idProducto" value="@item.IdProducto" />
            <button type="submit" class="btn btn-sm btn-outline-danger" title="Quitar"><i class="bi bi-x-
lg"></i></button>
        </form>
    </td>
</tr>
</tbody>
</table>
</div>
<div class="card-footer d-flex justify-content-between">
    <a asp-controller="Productos" asp-action="Index" class="btn btn-outline-secondary btn-sm"><i class="bi bi-
arrow-left"></i> Seguir comprando</a>
    <form asp-action="Vaciar" method="post">
        <button type="submit" class="btn btn-outline-danger btn-sm"><i class="bi bi-trash"></i> Vaciar
carrito</button>
    </form>
</div>
</div>
</div>
</div>

<div class="col-lg-4">
    <div class="card shadow-sm">
        <div class="card-body">
            <h5 class="card-title">Resumen</h5>
            <div class="d-flex justify-content-between">
                <span>Productos</span>
                <span>@Model.Sum(i => i.Cantidad)</span>
            </div>
            <hr />
            <div class="d-flex justify-content-between fs-5 fw-bold">
                <span>Total</span>
                <span class="text-precio">@Formato.Precio(total)</span>
            </div>
            <a asp-controller="Ventas" asp-action="Checkout" class="btn btn-warning w-100 mt-3">
                <i class="bi bi-bag-check"></i> Finalizar compra
            </a>
            @if (User.Identity?.IsAuthenticated != true)
            {
                <small class="text-muted d-block mt-2 text-center">Te pediremos iniciar sesión o registrarte para
confirmar.</small>
            }
        </div>
    </div>
</div>
</div>
</div>

```

```
}
```

8.9 Vistas de ventas (Views/Ventas/)

Checkout.cshtml — finalizar compra (cliente)

Views/Ventas/Checkout.cshtml

```
@model CheckoutViewModel
@{
    ViewData["Title"] = "Finalizar compra";
}

<h2 class="mb-3"><i class="bi bi-bag-check"></i> Finalizar compra</h2>

<form asp-action="Checkout" method="post">
    <div class="row g-4">
        <div class="col-lg-7">
            <div class="card shadow-sm">
                <div class="card-body p-4">
                    <h5 class="card-title mb-3">Datos de la compra</h5>
                    <div asp-validation-summary="ModelOnly" class="text-danger"></div>

                    <div class="alert alert-light border small">
                        <i class="bi bi-geo-alt"></i> <strong>Entrega en:</strong> @Model.DireccionEntrega
                        &nbsp;&nbsp;&nbsp;<i class="bi bi-telephone"></i> @Model.Telefono
                        <a asp-controller="Account" asp-action="Perfil" class="ms-2">Cambiar</a>
                    </div>

                    <div class="form-group mb-3">
                        <label asp-for="IdFormaPago"></label>
                        <select asp-for="IdFormaPago" asp-items="ViewBag.FormasPago" class="form-select">
                            <option value="">-- Elegir --</option>
                        </select>
                        <span asp-validation-for="IdFormaPago" class="text-danger"></span>
                    </div>
                    <div class="form-group mb-3">
                        <label asp-for="Observaciones"></label>
                        <textarea asp-for="Observaciones" class="form-control" rows="3" placeholder="Ej: entregar por la tarde,
tocar timbre..."></textarea>
                        <span asp-validation-for="Observaciones" class="text-danger"></span>
                    </div>
                </div>
            </div>
        </div>
        <div class="col-lg-5">
            <div class="card shadow-sm">
                <div class="card-body">
                    <h5 class="card-title mb-3">Resumen</h5>
                    <table class="table table-sm align-middle">
                        <tbody>
                            @foreach (var item in Model.Items)
                            {
                                <tr>
                                    <td>
                                        @item.Nombre
                                        <br /><small class="text-muted">@item.Cantidad x
                                        @Formato.Precio(item.PrecioUnitario)</small>
                                    </td>
                                    <td class="text-end">@Formato.Precio(item.Subtotal)</td>
                                </tr>
                            }
                        </tbody>
                    </table>
                </div>
            </div>
        </div>
    </div>
</form>
```

```

    }
  </tbody>
</tfoot>
  <tr class="fw-bold fs-5">
    <td>Total</td>
    <td class="text-end text-precio">@Formato.Precio(Model.Total)</td>
  </tr>
</tfoot>
</table>
  <button type="submit" class="btn btn-warning w-100"><i class="bi bi-check2-circle"></i> Confirmar compra</button>
  <a asp-controller="Carrito" asp-action="Index" class="btn btn-link w-100">Volver al carrito</a>
</div>
</div>
</div>
</div>
</form>

@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

MisCompras.cshtml

Views/Ventas/MisCompras.cshtml

```

@model List<Venta>
@{
  ViewData["Title"] = "Mis compras";
}

<h2 class="mb-3"><i class="bi bi-bag"></i> Mis compras</h2>

@if (Model.Count == 0)
{
  <div class="card shadow-sm">
    <div class="card-body text-center py-5">
      <i class="bi bi-bag fs-1 text-muted"></i>
      <p class="lead mt-2">Todavía no realizaste ninguna compra.</p>
      <a asp-controller="Productos" asp-action="Index" class="btn btn-warning">Ir al catálogo</a>
    </div>
  </div>
}
else
{
  <div class="card shadow-sm">
    <div class="table-responsive">
      <table class="table table-hover align-middle mb-0">
        <thead class="table-dark">
          <tr>
            <th>N°</th>
            <th>Fecha</th>
            <th>Forma de pago</th>
            <th class="text-center">Productos</th>
            <th class="text-end">Total</th>
            <th class="text-center">Estado</th>
            <th></th>
          </tr>
        </thead>
        <tbody>
          @foreach (var v in Model)
          {
            <tr>
              <td class="fw-bold">#@v.IdVenta</td>

```

```

        <td>@Formato.Fecha(v.Fecha)</td>
        <td>@v.FormaPago?.Nombre</td>
        <td class="text-center">@v.Detalles.Sum(d => d.Cantidad)</td>
        <td class="text-end">@Formato.Precio(v.Total)</td>
        <td class="text-center"><span class="badge @Formato.EstadoBadge(v.Estado)">@v.Estado</span></td>
        <td class="text-end">
            <a asp-action="Details" asp-route-id="@v.IdVenta" class="btn btn-sm btn-outline-dark">Ver
detalle</a>
        </td>
    </tr>
    }
</tbody>
</table>
</div>
</div>
}

```

Details.cshtml — la misma vista para cliente y personal

Views/Ventas/Details.cshtml

```

@model Venta
@{
    ViewData["Title"] = $"Venta #{Model.IdVenta}";
    var esPersonal = User.EsPersonal();
}

<div class="d-flex flex-wrap justify-content-between align-items-center mb-3">
    <h2 class="mb-0"><i class="bi bi-receipt"></i> @ (esPersonal ? "Venta" : "Compra") #@Model.IdVenta</h2>
    <span class="badge fs-6 @Formato.EstadoBadge(Model.Estado)">@Model.Estado</span>
</div>

<div class="row g-4">
    <div class="col-lg-8">
        <div class="card shadow-sm">
            <div class="table-responsive">
                <table class="table align-middle mb-0">
                    <thead class="table-light">
                        <tr>
                            <th colspan="2">Producto</th>
                            <th class="text-end">Precio unit.</th>
                            <th class="text-center">Cantidad</th>
                            <th class="text-end">Subtotal</th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach (var d in Model.Detalles)
                        {
                            <tr>
                                <td style="width: 70px"></td>
                                <td>
                                    <a asp-controller="Productos" asp-action="Details" asp-route-id="@d.IdProducto" class="text-decoration-none">@d.Producto?.Nombre</a>
                                </td>
                                <td class="text-end">@Formato.Precio(d.PrecioUnitario)</td>
                                <td class="text-center">@d.Cantidad</td>
                                <td class="text-end fw-bold">@Formato.Precio(d.Subtotal)</td>
                            </tr>
                        }
                    </tbody>
                </table>
            </div>
        </div>
    </div>
    <div class="col-lg-4">
        <div class="card shadow-sm">
            <div class="table-responsive">
                <table class="table align-middle mb-0">
                    <thead class="table-light">
                        <tr>
                            <th colspan="2">Producto</th>
                            <th class="text-end">Precio unit.</th>
                            <th class="text-center">Cantidad</th>
                            <th class="text-end">Subtotal</th>
                        </tr>
                    </thead>
                    <tbody>
                        @foreach (var d in Model.Detalles)
                        {
                            <tr>
                                <td style="width: 70px"></td>
                                <td>
                                    <a asp-controller="Productos" asp-action="Details" asp-route-id="@d.IdProducto" class="text-decoration-none">@d.Producto?.Nombre</a>
                                </td>
                                <td class="text-end">@Formato.Precio(d.PrecioUnitario)</td>
                                <td class="text-center">@d.Cantidad</td>
                                <td class="text-end fw-bold">@Formato.Precio(d.Subtotal)</td>
                            </tr>
                        }
                    </tbody>
                </table>
            </div>
        </div>
    </div>

```

```

        <tr class="fw-bold fs-5">
            <td colspan="4" class="text-end">Total</td>
            <td class="text-end text-precio">@Formato.Precio(Model.Total)</td>
        </tr>
    </tfoot>
</table>
</div>
</div>
</div>

<div class="col-lg-4">
    <div class="card shadow-sm mb-3">
        <div class="card-body">
            <h5 class="card-title">Datos</h5>
            <p class="mb-1"><strong>Fecha:</strong> @Formato.Fecha(Model.Fecha)</p>
            @if (esPersonal && Model.Cliente != null)
            {
                <p class="mb-1"><strong>Cliente:</strong> @Model.Cliente.NombreCompleto
                (@Model.Cliente.TipoCliente)<br /><small class="text-muted">@Model.Cliente.Email ·
                @Model.Cliente.Telefono</small></p>
                <p class="mb-1"><strong>Dirección:</strong> @Model.Cliente.Direccion</p>
            }
            <p class="mb-1"><strong>Forma de pago:</strong> @Model.FormaPago?.Nombre</p>
            <p class="mb-1"><strong>Atendido por:</strong> @(Model.Empleado?.NombreCompleto ?? "— (compra web
pendiente)")</p>
            @if (!string.IsNullOrEmpty(Model.Observaciones))
            {
                <p class="mb-0"><strong>Observaciones:</strong> @Model.Observaciones</p>
            }
        </div>
    </div>

    @if (esPersonal)
    {
        <div class="card shadow-sm mb-3 border-warning">
            <div class="card-body">
                <h5 class="card-title"><i class="bi bi-gear"></i> Cambiar estado</h5>
                <form asp-action="CambiarEstado" method="post" class="d-flex gap-2">
                    <input type="hidden" name="id" value="@Model.IdVenta" />
                    <select name="estado" class="form-select">
                        @foreach (EstadoVenta s in Enum.GetValues<EstadoVenta>())
                        {
                            <option value="@s" selected="@s == Model.Estado">@s</option>
                        }
                    </select>
                    <button type="submit" class="btn btn-warning text-nowrap">Aplicar</button>
                </form>
                <small class="text-muted d-block mt-2">
                    <em>Completada</em> registra al empleado que la cierra. <em>Cancelada</em> y <em>Devuelta</em>
devuelven el stock.
                </small>
            </div>
        </div>
    }
    else if (Model.Estado == EstadoVenta.Pendiente)
    {
        <form asp-action="Cancelar" method="post" onsubmit="return confirm('¿Seguro que querés cancelar esta
compra?');">
            <input type="hidden" name="id" value="@Model.IdVenta" />
            <button type="submit" class="btn btn-outline-danger w-100"><i class="bi bi-x-circle"></i> Cancelar
compra</button>
        </form>
    }

```

```

        <a asp-action="@{esPersonal ? "Manage" : "MisCompras")" class="btn btn-outline-secondary w-100 mt-2"><i
class="bi bi-arrow-left"></i> Volver</a>
    </div>
</div>

```

Manage.cshtml — gestión de ventas (personal)

Views/Ventas/Manage.cshtml

```

@model List<Venta>
@{
    ViewData["Title"] = "Gestión de ventas";
    EstadoVenta? estado = ViewBag.Estado;
}

<div class="d-flex flex-wrap justify-content-between align-items-center mb-3 gap-2">
    <h2 class="mb-0"><i class="bi bi-receipt"></i> Ventas <small class="text-muted fs-6">(@Model.Count)</small></h2>
    <div class="d-flex gap-2 flex-wrap">
        <div class="btn-group">
            <a asp-action="Manage" class="btn btn-sm @(estado == null ? "btn-dark" : "btn-outline-dark")">Todas</a>
            @foreach (EstadoVenta s in Enum.GetValues<EstadoVenta>())
            {
                <a asp-action="Manage" asp-route-estado="@s" class="btn btn-sm @(estado == s ? "btn-dark" : "btn-outline-
dark")">@s</a>
            }
        </div>
        <a asp-action="Registrar" class="btn btn-sm btn-warning"><i class="bi bi-plus-lg"></i> Registrar venta</a>
    </div>
</div>

<div class="card shadow-sm">
    <div class="table-responsive">
        <table class="table table-hover align-middle mb-0">
            <thead class="table-dark">
                <tr>
                    <th>Nº</th>
                    <th>Fecha</th>
                    <th>Cliente</th>
                    <th>Empleado</th>
                    <th>Pago</th>
                    <th class="text-center">Items</th>
                    <th class="text-end">Total</th>
                    <th class="text-center">Estado</th>
                    <th></th>
                </tr>
            </thead>
            <tbody>
                @foreach (var v in Model)
                {
                    <tr>
                        <td class="fw-bold">#@v.IdVenta</td>
                        <td>@Formato.Fecha(v.Fecha)</td>
                        <td>@v.Cliente?.NombreCompleto<br /><small class="text-muted">@v.Cliente?.Email</small></td>
                        <td class="small">@(v.Empleado?.NombreCompleto ?? "—")</td>
                        <td class="small">@v.FormaPago?.Nombre</td>
                        <td class="text-center">@v.Detalles.Sum(d => d.Cantidad)</td>
                        <td class="text-end">@Formato.Precio(v.Total)</td>
                        <td class="text-center"><span class="badge @Formato.EstadoBadge(v.Estado)">@v.Estado</span></td>
                        <td class="text-end">
                            <a asp-action="Details" asp-route-id="@v.IdVenta" class="btn btn-sm btn-outline-dark">Ver</a>
                        </td>
                    </tr>
                }
            </tbody>
        </table>
    </div>

```

```

        </tr>
    }
    @if (Model.Count == 0)
    {
        <tr><td colspan="9" class="text-center text-muted py-4">No hay ventas.</td></tr>
    }
</tbody>
</table>
</div>
</div>

```

Registrar.cshtml — venta de mostrador con filas dinámicas

El JavaScript del final agrega y quita filas, **renumera** los nombres (`Items[0]`, `Items[1]`...) para que ASP.NET arme la lista, y calcula subtotales y total en vivo con los `data-precio` y `data-stock` de cada opción.

Views/Ventas/Registrar.cshtml

```

@model RegistrarVentaViewModel
@{
    ViewData["Title"] = "Registrar venta";
    var productos = (List<Producto>)ViewBag.Productos;
}

<h2 class="mb-3"><i class="bi bi-cash-coin"></i> Registrar venta (mostrador)</h2>

<form asp-action="Registrar" method="post" id="formVenta">
    <div class="card shadow-sm mb-3">
        <div class="card-body p-4">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="row">
                <div class="col-md-6 form-group mb-3">
                    <label asp-for="IdCliente"></label>
                    <select asp-for="IdCliente" asp-items="ViewBag.Clientes" class="form-select">
                        <option value="">-- Elegir cliente --</option>
                    </select>
                    <span asp-validation-for="IdCliente" class="text-danger"></span>
                    <small class="text-muted">Si el cliente no existe, pedile que se registre desde la web.</small>
                </div>
                <div class="col-md-6 form-group mb-3">
                    <label asp-for="IdFormaPago"></label>
                    <select asp-for="IdFormaPago" asp-items="ViewBag.FormasPago" class="form-select">
                        <option value="">-- Elegir --</option>
                    </select>
                    <span asp-validation-for="IdFormaPago" class="text-danger"></span>
                </div>
            </div>
            <div class="form-group mb-0">
                <label asp-for="Observaciones"></label>
                <input asp-for="Observaciones" class="form-control" />
            </div>
        </div>
    </div>

    <div class="card shadow-sm mb-3">
        <div class="card-header bg-white d-flex justify-content-between align-items-center">
            <span><i class="bi bi-box-seam"></i> Productos</span>
            <button type="button" class="btn btn-sm btn-outline-dark" id="agregarFila"><i class="bi bi-plus-lg"></i> Agregar
            fila</button>
        </div>

```

```

<div class="table-responsive">
  <table class="table align-middle mb-0" id="tablaItems">
    <thead class="table-light">
      <tr>
        <th style="min-width: 280px">Producto</th>
        <th class="text-end">Precio</th>
        <th class="text-center">Stock</th>
        <th style="width: 120px">Cantidad</th>
        <th class="text-end">Subtotal</th>
        <th></th>
      </tr>
    </thead>
    <tbody id="filas">
      @for (int i = 0; i < Model.Items.Count; i++)
      {
        <tr>
          <td>
            <select name="Items[@i.IdProducto]" class="form-select form-select-sm sel-producto">
              <option value="0" data-precio="0" data-stock="0">-- Elegir producto --</option>
              @foreach (var p in productos)
              {
                <option value="@p.IdProducto" data-
precio="@p.PrecioVenta.ToString(System.Globalization.CultureInfo.InvariantCulture)" data-stock="@p.Stock"
selected="@p.IdProducto == Model.Items[i].IdProducto)">@p.Nombre</option>
              }
            </select>
          </td>
          <td class="text-end precio">-</td>
          <td class="text-center stock">-</td>
          <td><input type="number" name="Items[@i].Cantidad" value="@Model.Items[i].Cantidad" min="1"
class="form-control form-control-sm cantidad" /></td>
          <td class="text-end subtotal">-</td>
          <td class="text-end"><button type="button" class="btn btn-sm btn-outline-danger quitar"
title="Quitar"><i class="bi bi-x-lg"></i></button></td>
        </tr>
      }
    </tbody>
    <tfoot>
      <tr class="fw-bold fs-5">
        <td colspan="4" class="text-end">Total</td>
        <td class="text-end text-precio" id="total">@Formato.Moneda 0.00</td>
        <td></td>
      </tr>
    </tfoot>
  </table>
</div>
</div>

<button type="submit" class="btn btn-warning"><i class="bi bi-check2-circle"></i> Confirmar venta</button>
<a asp-action="Manage" class="btn btn-outline-secondary">Cancelar</a>
</form>

@section Scripts {
  <partial name="_ValidationScriptsPartial" />
  <script>
    // Filas dinámicas: agregar/quitar productos y recalcular subtotales sin recargar la página
    (function () {
      const moneda = '@Formato.Moneda';
      const filas = document.getElementById('filas');

      function fmt(n) { return moneda + ' ' + n.toFixed(2).replace(/B(?=(\d{3})+(?!d))/g, ','); }

      function recalcular() {

```

```

let total = 0;
filas.querySelectorAll('tr').forEach(tr => {
  const opt = tr.querySelector('.sel-producto').selectedOptions[0];
  const precio = parseFloat(opt.dataset.precio || 0);
  const stock = parseInt(opt.dataset.stock || 0);
  const cant = tr.querySelector('.cantidad');
  const cantidad = parseInt(cant.value || 0);
  tr.querySelector('.precio').textContent = precio ? fmt(precio) : '-';
  tr.querySelector('.stock').textContent = opt.value !== '0' ? stock : '-';
  tr.querySelector('.stock').className = 'text-center stock ' + (opt.value !== '0' && cantidad > stock ? 'text-
danger fw-bold' : '');
  const sub = precio * cantidad;
  tr.querySelector('.subtotal').textContent = precio ? fmt(sub) : '-';
  total += sub;
});
document.getElementById('total').textContent = fmt(total);
}

function reindexar() {
  filas.querySelectorAll('tr').forEach((tr, i) => {
    tr.querySelector('.sel-producto').name = 'Items[' + i + '].IdProducto';
    tr.querySelector('.cantidad').name = 'Items[' + i + '].Cantidad';
  });
}

document.getElementById('agregarFila').addEventListener('click', () => {
  const nueva = filas.querySelector('tr').cloneNode(true);
  nueva.querySelector('.sel-producto').selectedIndex = 0;
  nueva.querySelector('.cantidad').value = 1;
  filas.appendChild(nueva);
  reindexar(); recalcular();
});

filas.addEventListener('click', e => {
  if (e.target.closest('.quitar') && filas.querySelectorAll('tr').length > 1) {
    e.target.closest('tr').remove();
    reindexar(); recalcular();
  }
});

filas.addEventListener('change', recalcular);
filas.addEventListener('input', recalcular);
recalcular();
})();
</script>
}

```

8.10 Vistas de compras (Views/Compras/)

Views/Compras/Index.cshtml

```

@model List<Compra>
@{
  ViewData["Title"] = "Compras a proveedores";
  EstadoCompra? estado = ViewBag.Estado;
}

<div class="d-flex flex-wrap justify-content-between align-items-center mb-3 gap-2">
  <h2 class="mb-0"><i class="bi bi-truck"></i> Compras a proveedores <small class="text-muted
fs-6">(@Model.Count)</small></h2>
  <div class="d-flex gap-2 flex-wrap">
    <div class="btn-group">

```

```

<a asp-action="Index" class="btn btn-sm @(estado == null ? "btn-dark" : "btn-outline-dark")">Todas</a>
@foreach (EstadoCompra s in Enum.GetValues<EstadoCompra>())
{
    <a asp-action="Index" asp-route-estado="@s" class="btn btn-sm @(estado == s ? "btn-dark" : "btn-outline-
dark")">@s</a>
}
</div>
<a asp-action="Registrar" class="btn btn-sm btn-warning"><i class="bi bi-plus-lg"></i> Registrar compra</a>
</div>
</div>

<div class="card shadow-sm">
<div class="table-responsive">
<table class="table table-hover align-middle mb-0">
<thead class="table-dark">
<tr>
<th>N°</th>
<th>Fecha</th>
<th>Proveedor</th>
<th>Empleado</th>
<th class="text-center">Items</th>
<th class="text-end">Total</th>
<th class="text-center">Estado</th>
<th></th>
</tr>
</thead>
<tbody>
@foreach (var c in Model)
{
    <tr>
<td class="fw-bold">#@c.IdCompra</td>
<td>@Formato.Fecha(c.Fecha)</td>
<td>@c.Proveedor?.Nombre</td>
<td class="small">@c.Empleado?.NombreCompleto</td>
<td class="text-center">@c.Detalles.Sum(d => d.Cantidad)</td>
<td class="text-end">@Formato.Precio(c.Total)</td>
<td class="text-center"><span class="badge @Formato.EstadoBadge(c.Estado)">@c.Estado</span></td>
<td class="text-end">
<a asp-action="Details" asp-route-id="@c.IdCompra" class="btn btn-sm btn-outline-dark">Ver</a>
</td>
</tr>
}
@if (Model.Count == 0)
{
    <tr><td colspan="8" class="text-center text-muted py-4">No hay compras registradas.</td></tr>
}
</tbody>
</table>
</div>
</div>

```

Views/Compras/Details.cshtml

```

@model Compra
@{
    ViewData["Title"] = $"Compra #{Model.IdCompra}";
}

<div class="d-flex flex-wrap justify-content-between align-items-center mb-3">
<h2 class="mb-0"><i class="bi bi-truck"></i> Compra #@Model.IdCompra</h2>
<span class="badge fs-6 @Formato.EstadoBadge(Model.Estado)">@Model.Estado</span>
</div>

```

```

<div class="row g-4">
  <div class="col-lg-8">
    <div class="card shadow-sm">
      <div class="table-responsive">
        <table class="table align-middle mb-0">
          <thead class="table-light">
            <tr>
              <th colspan="2">Producto</th>
              <th class="text-end">Precio unit.</th>
              <th class="text-center">Cantidad</th>
              <th class="text-end">Subtotal</th>
            </tr>
          </thead>
          <tbody>
            @foreach (var d in Model.Detalles)
            {
              <tr>
                <td style="width: 70px"></td>
                <td><a asp-controller="Productos" asp-action="Details" asp-route-id="@d.IdProducto" class="text-
decoration-none">@d.Producto?.Nombre</a></td>
                <td class="text-end">@Formato.Precio(d.PrecioUnitario)</td>
                <td class="text-center">@d.Cantidad</td>
                <td class="text-end fw-bold">@Formato.Precio(d.Subtotal)</td>
              </tr>
            }
          </tbody>
          <tfoot>
            <tr class="fw-bold fs-5">
              <td colspan="4" class="text-end">Total</td>
              <td class="text-end text-precio">@Formato.Precio(Model.Total)</td>
            </tr>
          </tfoot>
        </table>
      </div>
    </div>
  </div>

  <div class="col-lg-4">
    <div class="card shadow-sm mb-3">
      <div class="card-body">
        <h5 class="card-title">Datos</h5>
        <p class="mb-1"><strong>Fecha:</strong> @Formato.Fecha(Model.Fecha)</p>
        <p class="mb-1"><strong>Proveedor:</strong> @Model.Proveedor?.Nombre<br /><small class="text-
muted">RUC/NIT @Model.Proveedor?.Ruc · @Model.Proveedor?.Telefono</small></p>
        <p class="mb-1"><strong>Registrada por:</strong> @Model.Empleado?.NombreCompleto</p>
        @if (!string.IsNullOrEmpty(Model.Observaciones))
        {
          <p class="mb-0"><strong>Observaciones:</strong> @Model.Observaciones</p>
        }
      </div>
    </div>

    <div class="card shadow-sm mb-3 border-warning">
      <div class="card-body">
        <h5 class="card-title"><i class="bi bi-gear"></i> Cambiar estado</h5>
        <form asp-action="CambiarEstado" method="post" class="d-flex gap-2">
          <input type="hidden" name="id" value="@Model.IdCompra" />
          <select name="estado" class="form-select">
            @foreach (EstadoCompra s in Enum.GetValues<EstadoCompra>())
            {
              <option value="@s" selected="@ (s == Model.Estado)">@s</option>
            }
          </select>
        </form>
      </div>
    </div>
  </div>

```

```

    }
    </select>
    <button type="submit" class="btn btn-warning text-nowrap">Aplicar</button>
</form>
<small class="text-muted d-block mt-2">
    Al pasar a <em>Completada</em> ingresa la mercadería (sube el stock y actualiza el precio de compra).
    Cancelar una compra completada descuenta el stock, siempre que no quede negativo.
</small>
</div>
</div>

<a asp-action="Index" class="btn btn-outline-secondary w-100"><i class="bi bi-arrow-left"></i> Volver a
compras</a>
</div>
</div>

```

Views/Compras/Registrar.cshtml

```

@model RegistrarCompraViewModel
@{
    ViewData["Title"] = "Registrar compra";
    var productos = (List<Producto>)ViewBag.Productos;
}

<h2 class="mb-3"><i class="bi bi-truck"></i> Registrar compra a proveedor</h2>

<form asp-action="Registrar" method="post">
    <div class="card shadow-sm mb-3">
        <div class="card-body p-4">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="row">
                <div class="col-md-6 form-group mb-3">
                    <label asp-for="IdProveedor"></label>
                    <select asp-for="IdProveedor" asp-items="ViewBag.Proveedores" class="form-select">
                        <option value="">-- Elegir proveedor --</option>
                    </select>
                    <span asp-validation-for="IdProveedor" class="text-danger"></span>
                </div>
                <div class="col-md-6 form-group mb-3">
                    <label asp-for="Observaciones"></label>
                    <input asp-for="Observaciones" class="form-control" placeholder="N° de factura, condiciones..." />
                </div>
            </div>
        </div>
    </div>

    <div class="card shadow-sm mb-3">
        <div class="card-header bg-white d-flex justify-content-between align-items-center">
            <span><i class="bi bi-box-seam"></i> Productos</span>
            <button type="button" class="btn btn-sm btn-outline-dark" id="agregarFila"><i class="bi bi-plus-lg"></i> Agregar
            fila</button>
        </div>
        <div class="table-responsive">
            <table class="table align-middle mb-0">
                <thead class="table-light">
                    <tr>
                        <th style="min-width: 280px">Producto</th>
                        <th class="text-center">Stock actual</th>
                        <th style="width: 120px">Cantidad</th>
                        <th style="width: 160px">Precio unitario</th>
                        <th class="text-end">Subtotal</th>
                    </tr>

```

```

        </tr>
    </thead>
    <tbody id="filas">
        @for (int i = 0; i < Model.Items.Count; i++)
        {
            <tr>
                <td>
                    <select name="Items[@i.IdProducto]" class="form-select form-select-sm sel-producto">
                        <option value="0" data-precio="0" data-stock="0">-- Elegir producto --</option>
                        @foreach (var p in productos)
                        {
                            <option value="@p.IdProducto" data-
precio="@p.PrecioCompra.ToString(System.Globalization.CultureInfo.InvariantCulture)" data-stock="@p.Stock"
selected="@p.IdProducto == Model.Items[i].IdProducto)">@p.Nombre</option>
                        }
                    </select>
                </td>
                <td class="text-center stock"></td>
                <td><input type="number" name="Items[@i.Cantidad]" value="@Model.Items[i].Cantidad" min="1"
class="form-control form-control-sm cantidad" /></td>
                <td>
                    <div class="input-group input-group-sm">
                        <span class="input-group-text">@Formato.Moneda</span>
                        <input type="number" name="Items[@i.PrecioUnitario]" value="@((Model.Items[i].PrecioUnitario >
0 ? Model.Items[i].PrecioUnitario.ToString(System.Globalization.CultureInfo.InvariantCulture) : "")" step="0.01"
min="0.01" class="form-control precio-unitario" />
                    </div>
                </td>
                <td class="text-end subtotal"></td>
                <td class="text-end"><button type="button" class="btn btn-sm btn-outline-danger quitar"
title="Quitar"><i class="bi bi-x-lg"></i></button></td>
            </tr>
        }
    </tbody>
    <tfoot>
        <tr class="fw-bold fs-5">
            <td colspan="4" class="text-end">Total</td>
            <td class="text-end text-precio" id="total">@Formato.Moneda 0.00</td>
            <td></td>
        </tr>
    </tfoot>
</table>
</div>
</div>

<button type="submit" class="btn btn-warning"><i class="bi bi-check2-circle"></i> Registrar compra</button>
<a asp-action="Index" class="btn btn-outline-secondary">Cancelar</a>
</form>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
    <script>
        (function () {
            const moneda = '@Formato.Moneda';
            const filas = document.getElementById('filas');

            function fmt(n) { return moneda + ' ' + n.toFixed(2).replace(/B(?=(\d{3})+(?!\d))/g, ','); }

            function recalcular() {
                let total = 0;
                filas.querySelectorAll('tr').forEach(tr => {
                    const opt = tr.querySelector('.sel-producto').selectedOptions[0];
                    const precio = parseFloat(tr.querySelector('.precio-unitario').value || 0);

```

```

        const cantidad = parseInt(tr.querySelector('.cantidad').value || 0);
        tr.querySelector('.stock').textContent = opt.value !== '0' ? opt.dataset.stock : '-';
        const sub = precio * cantidad;
        tr.querySelector('.subtotal').textContent = sub ? fmt(sub) : '-';
        total += sub;
    });
    document.getElementById('total').textContent = fmt(total);
}

function reindexar() {
    filas.querySelectorAll('tr').forEach((tr, i) => {
        tr.querySelector('.sel-producto').name = 'Items[' + i + '].IdProducto';
        tr.querySelector('.cantidad').name = 'Items[' + i + '].Cantidad';
        tr.querySelector('.precio-unitario').name = 'Items[' + i + '].PrecioUnitario';
    });
}

document.getElementById('agregarFila').addEventListener('click', () => {
    const nueva = filas.querySelector('tr').cloneNode(true);
    nueva.querySelector('.sel-producto').selectedIndex = 0;
    nueva.querySelector('.cantidad').value = 1;
    nueva.querySelector('.precio-unitario').value = '';
    filas.appendChild(nueva);
    reindexar(); recalcular();
});

filas.addEventListener('click', e => {
    if (e.target.closest('.quitar') && filas.querySelectorAll('tr').length > 1) {
        e.target.closest('tr').remove();
        reindexar(); recalcular();
    }
});

// Al elegir un producto se propone su último precio de compra
filas.addEventListener('change', e => {
    if (e.target.classList.contains('sel-producto')) {
        const tr = e.target.closest('tr');
        const precio = e.target.selectedOptions[0].dataset.precio;
        if (precio && precio !== '0') tr.querySelector('.precio-unitario').value = precio;
    }
    recalcular();
});
filas.addEventListener('input', recalcular);
recalcular();
})();
</script>
}

```

8.11 Vistas de inventario (Views/Inventario/)

Views/Inventario/Index.cshtml

```

@model List<Inventario>
@{
    ViewData["Title"] = "Inventario";
    bool soloBajos = ViewBag.SoloBajos;
}

<div class="d-flex flex-wrap justify-content-between align-items-center mb-3 gap-2">
    <h2 class="mb-0"><i class="bi bi-clipboard-data"></i> Inventario <small class="text-muted fs-6">(<@Model.Count>)</small></h2>
    <div class="btn-group">

```

```

        <a asp-action="Index" class="btn btn-sm @(!soloBajos ? "btn-dark" : "btn-outline-dark")">Todos</a>
        <a asp-action="Index" asp-route-soloBajos="true" class="btn btn-sm @(soloBajos ? "btn-dark" : "btn-outline-
dark")"><i class="bi bi-exclamation-triangle"></i> Bajo mínimo</a>
    </div>
</div>

<div class="card shadow-sm">
    <div class="table-responsive">
        <table class="table table-hover align-middle mb-0">
            <thead class="table-dark">
                <tr>
                    <th></th>
                    <th>Producto</th>
                    <th>Categoría</th>
                    <th class="text-center">Stock actual</th>
                    <th class="text-center">Stock mínimo</th>
                    <th class="text-center">Estado</th>
                    <th>Última actualización</th>
                    <th class="text-end">Acciones</th>
                </tr>
            </thead>
            <tbody>
                @foreach (var i in Model)
                {
                    <tr>
                        <td></td>
                        <td><a asp-controller="Productos" asp-action="Details" asp-route-id="@i.IdProducto" class="text-
decoration-none">@i.Producto?.Nombre</a></td>
                        <td class="small">@i.Producto?.Categoria?.Nombre</td>
                        <td class="text-center fw-bold">@i.StockActual</td>
                        <td class="text-center">@i.StockMinimo</td>
                        <td class="text-center">
                            @if (i.StockActual == 0)
                            {
                                <span class="badge bg-danger">Sin stock</span>
                            }
                            else if (!i.VerificarStock())
                            {
                                <span class="badge bg-warning text-dark">Bajo mínimo</span>
                            }
                            else
                            {
                                <span class="badge bg-success">OK</span>
                            }
                        </td>
                        <td class="small">@Formato.Fecha(i.UltimaActualizacion)</td>
                        <td class="text-end">
                            <a asp-action="Ajustar" asp-route-id="@i.IdInventario" class="btn btn-sm btn-outline-primary"><i
class="bi bi-sliders"></i> Ajustar</a>
                        </td>
                    </tr>
                }
                @if (Model.Count == 0)
                {
                    <tr><td colspan="8" class="text-center text-muted py-4">@(soloBajos ? "No hay productos por debajo del
mínimo." : "No hay registros de inventario.")</td></tr>
                }
            </tbody>
        </table>
    </div>
</div>

```

```

@model Inventario
@{
    ViewData["Title"] = "Ajustar inventario";
}

<div class="form-auth">
    <h2 class="mb-3"><i class="bi bi-sliders"></i> Ajustar inventario</h2>
    <div class="card shadow-sm">
        <div class="card-body p-4">
            <div class="d-flex align-items-center gap-3 mb-3">
                
                <div>
                    <h5 class="mb-0">@Model.Producto?.Nombre</h5>
                    <small class="text-muted">Última actualización: @Formato.Fecha(Model.UltimaActualizacion)</small>
                </div>
            </div>
            <form asp-action="Ajustar" method="post">
                <div asp-validation-summary="ModelOnly" class="text-danger"></div>
                <input type="hidden" name="id" value="@Model.IdInventario" />
                <div class="row">
                    <div class="col-md-6 form-group mb-3">
                        <label class="form-label">Stock actual (recuento fisico)</label>
                        <input type="number" name="stockActual" value="@Model.StockActual" min="0" class="form-control"
required />
                    </div>
                    <div class="col-md-6 form-group mb-3">
                        <label class="form-label">Stock mínimo</label>
                        <input type="number" name="stockMinimo" value="@Model.StockMinimo" min="0" class="form-control"
required />
                    </div>
                </div>
                <div class="form-group mb-3">
                    <label class="form-label">Motivo del ajuste (opcional)</label>
                    <input type="text" name="motivo" class="form-control" placeholder="Ej: recuento anual, rotura, faltante..."
maxlength="200" />
                </div>
                <button type="submit" class="btn btn-warning">Guardar ajuste</button>
                <a asp-action="Index" class="btn btn-outline-secondary">Cancelar</a>
            </form>
        </div>
    </div>
</div>

```

8.12 Vistas de administración (Views/Admin/)

```

@{
    ViewData["Title"] = "Panel general";
    var stockBajo = (List<Inventario>)ViewBag.StockBajo;
    var ultimasVentas = (List<Venta>)ViewBag.UltimasVentas;
    decimal ingresos = ViewBag.IngresosVentas;
    decimal gastos = ViewBag.GastosCompras;
}

<h2 class="mb-4"><i class="bi bi-speedometer2"></i> Panel general</h2>

<div class="row g-3 mb-4">
    <div class="col-6 col-md-3">

```

```

<div class="card stat-card shadow-sm border-0 bg-primary text-white">
  <div class="card-body d-flex justify-content-between align-items-center">
    <div>
      <div class="stat-valor">@ViewBag.TotalProductos</div>
      <div>Productos</div>
    </div>
    <i class="bi bi-box-seam"></i>
  </div>
  <a asp-controller="Productos" asp-action="Manage" class="card-footer text-white text-decoration-none small">Administrar <i class="bi bi-arrow-right"></i></a>
</div>
</div>
<div class="col-6 col-md-3">
  <div class="card stat-card shadow-sm border-0 bg-warning text-dark">
    <div class="card-body d-flex justify-content-between align-items-center">
      <div>
        <div class="stat-valor">@ViewBag.TotalVentas</div>
        <div>Ventas (@ViewBag.VentasPendientes pendientes)</div>
      </div>
      <i class="bi bi-receipt"></i>
    </div>
    <a asp-controller="Ventas" asp-action="Manage" class="card-footer text-dark text-decoration-none small">Gestionar <i class="bi bi-arrow-right"></i></a>
  </div>
</div>
<div class="col-6 col-md-3">
  <div class="card stat-card shadow-sm border-0 bg-success text-white">
    <div class="card-body d-flex justify-content-between align-items-center">
      <div>
        <div class="stat-valor">@Formato.Precio(ingresos)</div>
        <div>Ventas completadas</div>
      </div>
      <i class="bi bi-cash-stack"></i>
    </div>
    <div class="card-footer small">Compras completadas: @Formato.Precio(gastos)</div>
  </div>
</div>
<div class="col-6 col-md-3">
  <div class="card stat-card shadow-sm border-0 bg-dark text-white">
    <div class="card-body d-flex justify-content-between align-items-center">
      <div>
        <div class="stat-valor">@ViewBag.TotalClientes</div>
        <div>Clientes · @ViewBag.TotalEmpleados empleados</div>
      </div>
      <i class="bi bi-people"></i>
    </div>
    @if (User.IsInRole("Admin"))
    {
      <a asp-action="Usuarios" class="card-footer text-white text-decoration-none small">Ver usuarios <i class="bi bi-arrow-right"></i></a>
    }
    else
    {
      <div class="card-footer small">&nbsp;</div>
    }
  </div>
</div>
</div>
<div class="row g-4">
  <div class="col-lg-6">
    <div class="card shadow-sm h-100">
      <div class="card-header bg-white"><i class="bi bi-clock-history"></i> Últimas ventas</div>

```

```

<div class="table-responsive">
  <table class="table table-sm align-middle mb-0">
    <tbody>
      @foreach (var v in ultimasVentas)
      {
        <tr>
          <td><a asp-controller="Ventas" asp-action="Details" asp-route-id="@v.IdVenta" class="fw-bold text-decoration-none">#@v.IdVenta</a></td>
          <td class="small">@Formato.Fecha(v.Fecha)</td>
          <td class="small">@v.Cliente?.NombreCompleto</td>
          <td class="text-end">@Formato.Precio(v.Total)</td>
          <td class="text-center"><span class="badge @Formato.EstadoBadge(v.Estado)">@v.Estado</span></td>
        </tr>
      }
      @if (ultimasVentas.Count == 0)
      {
        <tr><td class="text-center text-muted py-3">Todavía no hay ventas.</td></tr>
      }
    </tbody>
  </table>
</div>
</div>
</div>

<div class="col-lg-6">
  <div class="card shadow-sm h-100">
    <div class="card-header bg-white d-flex justify-content-between align-items-center">
      <span><i class="bi bi-exclamation-triangle text-warning"></i> Stock en o bajo el mínimo</span>
      <a asp-controller="Inventario" asp-action="Index" asp-route-soloBajos="true" class="small">Ver inventario</a>
    </div>
    <div class="table-responsive">
      <table class="table table-sm align-middle mb-0">
        <tbody>
          @foreach (var i in stockBajo)
          {
            <tr>
              <td style="width: 50px"></td>
              <td><a asp-controller="Inventario" asp-action="Ajustar" asp-route-id="@i.IdInventario" class="text-decoration-none">@i.Producto?.Nombre</a></td>
              <td class="text-center">
                <span class="badge @(i.StockActual == 0 ? "bg-danger" : "bg-warning text-dark")">@i.StockActual / mín. @i.StockMinimo</span>
              </td>
            </tr>
          }
          @if (stockBajo.Count == 0)
          {
            <tr><td class="text-center text-muted py-3">Todo el stock está en orden.</td></tr>
          }
        </tbody>
      </table>
    </div>
  </div>
</div>

<div class="col-lg-6">
  <div class="card shadow-sm">
    <div class="card-header bg-white"><i class="bi bi-trophy"></i> Productos más vendidos</div>
    <div class="table-responsive">
      <table class="table table-sm align-middle mb-0">

```

```
| Producto | Unidades | Importe |
| --- | --- | --- |

@foreach (var m in ViewBag.MasVendidos)
{
| @m.Nombre | @m.Cantidad | @Formato.Precio((decimal)m.Total) |

}
@if (((System.Collections.IList)ViewBag.MasVendidos).Count == 0)
{
| Todavía no hay ventas. | | |

}

```

Views/Admin/Usuarios.cshtml

```

@model List<Usuario>
@{
    ViewData["Title"] = "Usuarios";
    var roles = (Dictionary<int, IList<string>>)ViewBag.Roles;
    var miId = User.GetUsuarioId();
}



## Usuarios (@Model.Count)

Nuevo empleado



| Nombre | Email |
|--------|-------|
|--------|-------|


```

```

<th>Tipo</th>
<th>Teléfono</th>
<th>Registro</th>
<th class="text-center">Rol</th>
<th class="text-center">Estado</th>
<th class="text-end">Acciones</th>
</tr>
</thead>
<tbody>
  @foreach (var u in Model)
  {
    var soyYo = u.Id == miId;
    var rolesU = roles[u.Id];
    <tr class="@((u.Estado ? "" : "table-secondary"))">
      <td class="fw-bold">@u.NombreCompleto @(soyYo ? "(vos)" : "")</td>
      <td>@u.Email</td>
      <td>
        @switch (u)
        {
          case Empleado e:
            <span class="badge bg-info text-dark">Empleado</span> <small
class="text-muted">@e.Cargo</small>
            break;
          case Cliente c:
            <span class="badge bg-light text-dark border">Cliente</span> <small class="text-
muted">@c.TipoCliente</small>
            break;
        }
      </td>
      <td>@(((u as Empleado)?.Telefono ?? (u as Cliente)?.Telefono)</td>
      <td class="small">@Formato.Fecha(u.FechaCreacion)</td>
      <td class="text-center">
        @foreach (var r in rolesU)
        {
          <span class="badge @(r == "Admin" ? "bg-warning text-dark" : "bg-secondary")">@r</span>
        }
      </td>
      <td class="text-center">
        <span class="badge @(u.Estado ? "bg-success" : "bg-danger")">@(u.Estado ? "Activo" :
"Desactivado")</span>
      </td>
      <td class="text-end text-nowrap">
        @if (u is Empleado)
        {
          <a asp-action="EditarEmpleado" asp-route-id="@u.Id" class="btn btn-sm btn-outline-primary"
title="Editar"><i class="bi bi-pencil"></i></a>
        }
        @if (!soyYo)
        {
          if (u is Empleado)
          {
            var esAdmin = rolesU.Contains("Admin");
            <form asp-action="CambiarRol" method="post" class="d-inline">
              <input type="hidden" name="id" value="@u.Id" />
              <button type="submit" class="btn btn-sm @(esAdmin ? "btn-outline-secondary" : "btn-outline-
warning")" title="@((esAdmin ? "Pasar a Empleado" : "Hacer Admin"))">
                <i class="bi @(esAdmin ? "bi-person-dash" : "bi-person-check")"></i> @((esAdmin ? "Quitar
admin" : "Hacer admin")
              </button>
            </form>
          }
          else if (u is Cliente cli)
          {

```

```

        <form asp-action="CambiarTipoCliente" method="post" class="d-inline">
            <input type="hidden" name="id" value="@u.Id" />
            <button type="submit" class="btn btn-sm btn-outline-secondary" title="Cambiar tipo de cliente">
                <i class="bi bi-arrow-left-right"></i> @ (cli.TipoCliente == "Mayorista" ? "Regular" : "Mayorista")
            </button>
        </form>
    }
    <form asp-action="ToggleEstado" method="post" class="d-inline">
        <input type="hidden" name="id" value="@u.Id" />
        <button type="submit" class="btn btn-sm @(u.Estado ? "btn-outline-danger" : "btn-outline-success")"
title="@ (u.Estado ? "Desactivar" : "Activar")">
            <i class="bi @(u.Estado ? "bi-person-x" : "bi-person-check")"></i>
        </button>
    </form>
}
</td>
</tr>
}
</tbody>
</table>
</div>
</div>

```

Views/Admin/_EmpleadoForm.cshtml

```

@model EmpleadoViewModel
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="row">
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Nombre"></label>
        <input asp-for="Nombre" class="form-control" autofocus />
        <span asp-validation-for="Nombre" class="text-danger"></span>
    </div>
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Apellido"></label>
        <input asp-for="Apellido" class="form-control" />
        <span asp-validation-for="Apellido" class="text-danger"></span>
    </div>
</div>
<div class="row">
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Email"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Password"></label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
        <small class="text-muted">@(Model.Id == 0 ? "Mínimo 8 caracteres, con mayúsculas, minúsculas y números." :
"Dejar vacío para no cambiarla.")</small>
    </div>
</div>
<div class="row">
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Cargo"></label>
        <input asp-for="Cargo" class="form-control" placeholder="Vendedor, Cajero, Encargado de depósito..." />
        <span asp-validation-for="Cargo" class="text-danger"></span>
    </div>
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Telefono"></label>
        <input asp-for="Telefono" class="form-control" placeholder="Solo números, 8 a 15 dígitos" />
    </div>
</div>

```

```

        <span asp-validation-for="Telefono" class="text-danger"></span>
    </div>
</div>
<div class="form-group mb-3">
    <label asp-for="Direccion"></label>
    <input asp-for="Direccion" class="form-control" />
    <span asp-validation-for="Direccion" class="text-danger"></span>
</div>
<div class="row">
    <div class="col-md-6 form-group mb-3">
        <label asp-for="Rol"></label>
        <select asp-for="Rol" class="form-select">
            <option value="Empleado">Empleado (ventas, compras e inventario)</option>
            <option value="Admin">Admin (acceso total)</option>
        </select>
    </div>
    <div class="col-md-6 form-group form-check mb-3 mt-4 ms-2">
        <input asp-for="Estado" class="form-check-input" />
        <label asp-for="Estado" class="form-check-label"></label>
    </div>
</div>
</div>

```

Views/Admin/CrearEmpleado.cshtml

```

@model EmpleadoViewModel
@{
    ViewData["Title"] = "Nuevo empleado";
}

<h2 class="mb-3"><i class="bi bi-person-plus"></i> Nuevo empleado</h2>
<div class="card shadow-sm">
    <div class="card-body p-4">
        <form asp-action="CrearEmpleado" method="post">
            <partial name="_EmpleadoForm" model="Model" />
            <button type="submit" class="btn btn-warning">Crear empleado</button>
            <a asp-action="Usuarios" class="btn btn-outline-secondary">Cancelar</a>
        </form>
    </div>
</div>

@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

Views/Admin/EditarEmpleado.cshtml

```

@model EmpleadoViewModel
@{
    ViewData["Title"] = "Editar empleado";
}

<h2 class="mb-3"><i class="bi bi-pencil-square"></i> Editar empleado</h2>
<div class="card shadow-sm">
    <div class="card-body p-4">
        <form asp-action="EditarEmpleado" method="post">
            <input type="hidden" asp-for="Id" />
            <partial name="_EmpleadoForm" model="Model" />
            <button type="submit" class="btn btn-warning">Guardar cambios</button>
            <a asp-action="Usuarios" class="btn btn-outline-secondary">Cancelar</a>
        </form>
    </div>
</div>

```

8.13 wwwroot/css/site.css — estilos propios

wwwroot/css/site.css

```
html {
  font-size: 14px;
}

@media (min-width: 768px) {
  html {
    font-size: 16px;
  }
}

body {
  background-color: #f5f5f5;
}

.btn:focus, .btn:active:focus, .btn-link.nav-link:focus, .form-control:focus, .form-check-input:focus {
  box-shadow: 0 0 0 0.1rem white, 0 0 0 0.25rem #ffc107;
}

/* ---- Precios ---- */
.text-precio {
  color: #d9480f;
}

/* ---- Tarjetas de producto ---- */
.producto-card {
  transition: transform .15s ease, box-shadow .15s ease;
}

.producto-card:hover {
  transform: translateY(-3px);
  box-shadow: 0 .5rem 1rem rgba(0, 0, 0, .15) !important;
}

.producto-img-wrap {
  display: flex;
  align-items: center;
  justify-content: center;
  height: 200px;
  padding: 1rem;
  background: #fff;
  border-bottom: 1px solid #eee;
}

.producto-img {
  max-height: 100%;
  max-width: 100%;
  object-fit: contain;
}

/* Imagen grande en la página de detalle */
.producto-detalle-img {
  max-height: 420px;
  width: 100%;
  object-fit: contain;
  background: #fff;
  padding: 1.5rem;
}
```

```
border-radius: .5rem;
}

/* Miniatura en tablas de administración */
.thumb {
width: 56px;
height: 56px;
object-fit: contain;
background: #fff;
border: 1px solid #dee2e6;
border-radius: .25rem;
}

/* Contador del carrito en la barra */
.badge-carrito {
position: absolute;
top: 2px;
right: -4px;
font-size: .65rem;
}

/* Portada */
.hero {
background: linear-gradient(135deg, #212529 0%, #343a40 60%, #b45309 100%);
color: #fff;
border-radius: .75rem;
}

.categoria-chip {
display: inline-block;
padding: .4rem .9rem;
border-radius: 2rem;
background: #fff;
border: 1px solid #dee2e6;
color: #212529;
text-decoration: none;
font-size: .9rem;
}

.categoria-chip:hover, .categoria-chip.active {
background: #ffc107;
border-color: #ffc107;
color: #212529;
}

/* Tarjetas del panel de administración */
.stat-card .stat-valor {
font-size: 1.8rem;
font-weight: 700;
}

.stat-card .bi {
font-size: 2.2rem;
opacity: .35;
}

/* Formularios centrados (login, registro) */
.form-auth {
max-width: 480px;
margin: 0 auto;
}
```

Parte 9 — Migraciones y base de datos

9.1 Qué es una migración y cómo se generó

Una migración es una clase de C# generada automáticamente que describe los cambios a hacer en la base de datos (crear tablas, columnas, índices, claves foráneas). EF Core compara los modelos con la última "foto" de la base (`ApplicationDbContextModelSnapshot.cs`) y genera la diferencia.

La migración inicial del proyecto se generó con:

```
dotnet ef migrations add InitialCreate
```

Eso creó la carpeta `Migrations/` con tres archivos: `..._InitialCreate.cs` (el método `Up()` crea todas las tablas y `Down()` las borra), su `.Designer.cs` y el snapshot.

No hace falta ningún script SQL manual: EF Core crea automáticamente TODAS las tablas (las de Identity, la herencia Empleado/Cliente y las de la ferretería) a partir del modelo.

9.2 Cómo se aplica

1. **Automática al arrancar** (`Program.cs` → `context.Database.MigrateAsync()`): cada vez que la app arranca aplica las migraciones que falten. Ideal para Render.
2. **Manual:** `dotnet ef database update` desde la carpeta del proyecto.

EF lleva el registro de qué migraciones ya aplicó en la tabla `__EFMigrationsHistory`.

9.3 Tablas que se crean en Supabase

Tabla	Origen	Contenido
Usuarios	Identity (<code>Usuario</code>)	Todos los usuarios: email, hash de contraseña, nombre, apellido, estado, fecha de creación
Roles	Identity (<code>Rol</code>)	<code>Admin</code> , <code>Empleado</code> , <code>Cliente</code> con descripción
UsuarioRoles	Identity	Qué usuario tiene qué rol
UsuarioClaims, UsuarioLogins, UsuarioTokens, RolClaims	Identity	Tablas auxiliares (no las usamos directamente)
Empleados	Herencia TPT	<code>IdEmpleado</code> (= <code>IdUsuario</code>), cargo, teléfono, dirección
Clientes	Herencia TPT	<code>IdCliente</code> (= <code>IdUsuario</code>), teléfono, dirección, tipo de cliente
Categorías	Nuestra	Categorías
Proveedores	Nuestra	Proveedores
Productos	Nuestra	Productos (con precio de venta y compra, stock, mínimo, imagen)
Inventarios	Nuestra	Inventario de cada producto
FormasPago	Nuestra	Formas de pago
Ventas / <code>DetalleVentas</code>	Nuestra	Ventas y sus líneas

Tabla	Origen	Contenido
Compras / DetalleCompras	Nuestra	Compras a proveedores y sus líneas
__EFMigrationsHistory	EF Core	Migraciones aplicadas

Se pueden ver desde Supabase en **Table Editor**.

9.4 Cuando se cambia un modelo

Si más adelante se agrega o modifica un campo (por ejemplo `public string?Codigo` en `Producto`), se genera una nueva migración y la app la aplica sola al arrancar:

```
dotnet ef migrations add AgregarCodigoProducto
```

Si la base de Supabase ya tenía tablas de una versión anterior del proyecto (por ejemplo `Products` u `Orders`), hay que borrarlas (o crear un proyecto nuevo en Supabase) antes de arrancar, para que la migración cree el esquema nuevo sin conflictos.

Parte 10 — Ejecutar y probar el sistema

10.1 Ejecutar en la computadora (desarrollo)

1. Abrir una terminal en la carpeta del proyecto:

```
cd FerreteriaApp
```

2. Poner el connection string de Supabase en `appsettings.json` o en `appsettings.Development.json` (ver 2.5).
3. Restaurar paquetes y ejecutar:

```
dotnet restore
dotnet run
```

4. La consola muestra `Now listening on: http://localhost:5047`. Abrir esa URL. La primera vez tarda un poco más porque crea las tablas y carga los datos iniciales.

En Visual Studio: abrir `FerreteriaApp.csproj`, elegir el perfil **http** y apretar F5.

10.2 Usuarios iniciales

Rol	Email	Contraseña
Admin (empleado, cargo Administrador)	<code>juan@gmail.com</code>	<code>Juan1234</code>
Empleado (vendedor de ejemplo)	<code>vendedor@ferreteria.com</code>	<code>Vendedor123</code>

El administrador se define en `appsettings.json` → "Admin" (cambiarlo antes de entregar). Las contraseñas deben cumplir la regla del diagrama: 8 caracteres con mayúscula, minúscula y número. Los usuarios que se registran desde la web son **Cientes**; los empleados los crea el administrador en **Gestión → Usuarios y empleados**.

10.3 Recorrido de prueba completo

Como visitante

1. Portada: categorías con la cantidad de productos y destacados.
2. **Catálogo**: buscar "taladro", filtrar por "Herramientas Eléctricas", cambiar de página.
3. Entrar a un producto y **agregarlo al carrito** (sin cuenta): el numerito del carrito sube.
4. En el carrito, tocar **Finalizar compra**: pide iniciar sesión.

Como cliente

5. **Registrarse** (nombre, apellido, email, contraseña de 8+ con mayúscula/minúscula/número, teléfono de 8-15 dígitos, dirección). Probar una contraseña débil o un teléfono con letras: se muestran los mensajes de validación.
6. Al quedar logueado, el carrito sigue ahí. **Finalizar compra**: elegir forma de pago y confirmar.
7. Aparece la **compra #N** en estado *Pendiente*; en **Mis compras** está en la lista. En el catálogo el stock bajó.

8. **Cancelar compra:** pasa a *Cancelada* y el stock vuelve.
9. Comprar de nuevo (para que quede una venta pendiente) y editar el **perfil / cambiar contraseña**.

Como empleado (vendedor@ferreteria.com / [Vendedor123](#))

10. Al ingresar aparece el **Panel general** y el menú **Gestión**.
11. **Ventas:** abrir la venta web pendiente y pasarla a *Completada* (queda registrado el empleado); probar *Devolución* (repone stock).
12. **Registrar venta** de mostrador: elegir el cliente, forma de pago, agregar filas de productos y confirmar. Probar una cantidad mayor al stock: error "stock insuficiente".
13. **Compras a proveedores → Registrar compra:** proveedor, productos, cantidades y precio unitario. Queda *Pendiente*. Abrirla y pasarla a *Completada*: el stock sube y el precio de compra del producto se actualiza. Intentar cancelarla después de vender esas unidades: error "no puede quedar negativo".
14. **Inventario:** filtrar "Bajo mínimo", **ajustar** el stock de un producto tras un recuento.
15. **Productos:** crear uno nuevo (con categoría, proveedor, precios, stock, imagen) y editarlo.
16. Intentar entrar a **Categorías** o **Usuarios:** *Acceso denegado* (son solo de Admin).

Como administrador (juan@gmail.com / [Juan1234](#))

17. **Categorías, Proveedores y Formas de pago:** crear, editar y "eliminar" una que esté en uso (se desactiva; una categoría desactivada oculta sus productos del catálogo).
 18. **Usuarios y empleados:** crear un empleado, hacerlo Admin y volver a Empleado, desactivar al cliente registrado (ya no puede iniciar sesión) y reactivarlo, cambiarlo a Mayorista.
 19. **Eliminar producto:** uno sin movimientos se borra (con su inventario); uno vendido se desactiva.
-

Parte 11 — Publicar en internet: GitHub + Render + Supabase

La base de datos ya está en Supabase. Falta que la aplicación corra en un servidor: para eso se sube el código a GitHub y Render lo construye y lo publica.

11.1 Dockerfile — la receta para armar la aplicación

Dockerfile

```
# ----- Etapa 1: compilar y publicar -----
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

# Primero solo el .csproj para aprovechar la caché de Docker al restaurar paquetes
COPY FerreteriaApp.csproj ./
RUN dotnet restore

# Después el resto del código
COPY . ./
RUN dotnet publish FerreteriaApp.csproj -c Release -o /app/publish /p:UseAppHost=false

# ----- Etapa 2: imagen final (solo el runtime, más liviana) -----
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final
WORKDIR /app
COPY --from=build /app/publish ./

# Render inyecta la variable PORT; Program.cs la lee y escucha en ese puerto.
# Si no existe (por ejemplo corriendo el contenedor a mano), se usa 8080.
ENV PORT=8080
ENV ASPNETCORE_ENVIRONMENT=Production
EXPOSE 8080

ENTRYPOINT ["dotnet", "FerreteriaApp.dll"]
```

Tiene dos etapas: **build** (imagen con el SDK, restaura paquetes y hace `dotnet publish`) y **final** (imagen con solo el runtime, más liviana, que ejecuta `dotnet FerreteriaApp.dll`).

11.2 .dockerignore

.dockerignore

```
bin/
obj/
.git/
.gitignore
.vs/
.vscode/
.idea/
*.user
appsettings.Development.json
README.md
INSTRUCTIVO.md
INSTRUCTIVO.docx
INSTRUCTIVO.pdf
docs-src/
```

11.3 render.yaml — Blueprint de Render

render.yaml

```
# Blueprint de Render: al conectar el repositorio, Render lee este archivo
# y crea el servicio web automáticamente (Dashboard → New → Blueprint).
# También se puede crear el servicio a mano eligiendo "Docker" como runtime.
services:
- type: web
  name: ferreteria-app
  runtime: docker
  plan: free
  dockerfilePath: ./Dockerfile
  healthCheckPath: /
  envVars:
    - key: ASPNETCORE_ENVIRONMENT
      value: Production
    # Connection string de Supabase (Session pooler). Se carga desde el panel de Render
    # en Environment → Add Environment Variable, para no dejarla en el repositorio.
    - key: ConnectionStrings__DefaultConnection
      sync: false
    # Opcional: contraseña del administrador inicial (si no se define, se usa la de appsettings.json)
    - key: Admin__Password
      sync: false
```

`sync: false` significa "esta variable la cargo yo a mano en el panel" (no queda en el repositorio).

11.4 .gitignore

.gitignore

```
# Compilación
bin/
obj/
*.user
*.suo
.vs/
.vscode/
.idea/
*.swp

# Configuración local con credenciales (NO subir a GitHub)
appsettings.Development.json
```

11.5 Subir el proyecto a GitHub

1. Crear un repositorio vacío en GitHub (botón **New**). No agregar README ni .gitignore.
2. Desde la carpeta `FerreteriaApp`, verificar que `appsettings.json` **no** tenga la contraseña real (o que esté en `appsettings.Development.json`, que se ignora). Luego:

```
git init
git add .
git commit -m "Sistema de ferretería con ASP.NET Core MVC y Supabase"
git branch -M main
git remote add origin https://github.com/<tu-usuario>/<tu-repo>.git
git push -u origin main
```

11.6 Crear el servicio en Render

Opción A — Blueprint (usa `render.yaml`): New → **Blueprint**, conectar el repositorio y cargar las variables `ConnectionStrings__DefaultConnection` (connection string de Supabase) y, opcional, `Admin__Password`.

Opción B — Manual: New → **Web Service**, conectar el repositorio, runtime **Docker**, plan Free, y en **Environment Variables**:

Key	Value
<code>ConnectionStrings__DefaultConnection</code>	<code>Host=aws-0-...pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.xxx;Password=...;SSL Mode=Require;Trust Server Certificate=true</code>
<code>ASPNETCORE_ENVIRONMENT</code>	<code>Production</code>
<code>Admin__Password</code>	(opcional; debe cumplir la regla de 8 caracteres con mayúscula, minúscula y número)

Create Web Service: Render construye la imagen y publica la app en <https://ferreteria-app.onrender.com> (o el nombre elegido).

En .NET, el doble guion bajo `__` en una variable de entorno equivale a los dos puntos `:` de `appsettings.json`: `ConnectionStrings__DefaultConnection = ConnectionStrings:DefaultConnection`.

Cada `git push` a `main` vuelve a desplegar automáticamente. Los logs del arranque (migraciones, errores) están en la pestaña **Logs** de Render.

11.7 Notas sobre Render (plan Free)

- El servicio se "duerme" tras 15 minutos sin visitas; la primera visita después tarda ~30-60 s.
- El disco es **efímero**: las imágenes que suba el personal desde la app se pierden en cada nuevo deploy. Las 43 imágenes iniciales sí persisten porque están en el repositorio. Para imágenes nuevas conviene pegar la **URL de una imagen** alojada afuera (por ejemplo en Supabase Storage).
- La **sesión** (carrito) está en memoria: si Render reinicia el servicio, los carritos en curso se vacían. Las ventas ya confirmadas están en la base y no se pierden.

11.8 Cómo se conectan las tres piezas

```
Navegador del usuario
| https
▼
RENDER (corre nuestra app .NET dentro de Docker)
| connection string (variable de entorno)
▼
SUPABASE (PostgreSQL con las tablas creadas por EF Core)
```

Parte 12 — Personalización

En `appsettings.json`:

```
"Tienda": {  
  "Nombre": "Ferretería Central",  
  "Moneda": "Bs",  
  "ZonaHoraria": "America/La_Paz"  
}
```

- **Nombre:** barra, portada y pie de página. **Moneda:** símbolo de los precios (Bs, S/, \$). **ZonaHoraria:** para mostrar fechas (`America/La_Paz`, `America/Lima`, `America/Argentina/Buenos_Aires`). En `Render` también se pueden cambiar con `Tienda_Nombre`, etc.

Otros lugares para personalizar:

- Dirección, teléfono y horario del pie: `Views/Shared/_Layout.cshtml`.
 - Texto de la portada: `Views/Home/Index.cshtml`.
 - Colores: `wwwroot/css/site.css`.
 - Productos por página: `ProductosPorPagina` en `ProductosController.cs`.
 - Reglas de contraseña: `Program.cs` → `AddIdentity`.
 - Vendedor de ejemplo, proveedores y productos iniciales: `Data/DbInitializer.cs`.
-

Parte 13 — Solución de problemas frecuentes

Error / síntoma	Causa y solución
Timeout / no conecta a la base de datos	Se está usando el host de conexión directa (<code>db.xxx.supabase.co</code>), que es IPv6. Usar el connection string del Session pooler (2.2).
password authentication failed	Contraseña incorrecta en el connection string. Resetearla en Supabase → <i>Database</i> → <i>Reset database password</i> .
EntityFrameworkCore does not exist, IdentityDbContext<> could not be found	Los paquetes NuGet no se restauraron. Reemplazar el <code>.csproj</code> por el de 2.4.1, borrar <code>bin/</code> y <code>obj/</code> , correr <code>dotnet restore</code> y <code>dotnet build</code> .
dotnet ef no se reconoce como comando	<code>dotnet tool install --global dotnet-ef</code> y reabrir la terminal.
Unable to create an object of type ApplicationDbContext	Falta <code>Microsoft.EntityFrameworkCore.Design</code> o la terminal no está en la carpeta del <code>.csproj</code> .
relation "Products" already exists u otras tablas viejas	La base tenía otra versión del modelo. Borrar las tablas viejas (o crear un proyecto nuevo en Supabase) y volver a arrancar.
The view '...' was not found	Falta el <code>.cshtml</code> de esa acción; revisar el nombre exacto en <code>Views/<Controlador>/</code> .
400/403 al enviar un formulario	Falta el token antifalsificación: el <code><form></code> debe tener <code>method="post"</code> y usar los tag helpers.
Al registrarse dice que la contraseña no cumple	Regla del diagrama: mínimo 8 caracteres con mayúscula, minúscula y número (<code>Program.cs</code> → <code>AddIdentity</code>).
"El teléfono debe tener solo números (8 a 15 dígitos)"	Regla del diagrama (<code>[RegularExpression]</code> en los modelos). Sin espacios ni guiones.
El login siempre falla	Identity compara el <code>UserName</code> (= email). Revisar mayúsculas y que la cuenta no esté desactivada ("La cuenta está desactivada").
Al crear un producto el precio 99.50 se guarda como 9950	Falta la cultura invariante en <code>Program.cs</code> .
"stock insuficiente" o "no puede quedar negativo"	Reglas del diagrama: no se vende sin stock y no se cancela una compra si dejaría stock negativo. Revisar el Inventario .
La imagen subida no se ve	El <code><form></code> necesita <code>enctype="multipart/form-data"</code> y <code>Program.cs</code> debe tener <code>app.UseStaticFiles()</code> .
El carrito se vació solo	La sesión expira a las 2 horas de inactividad o cuando se reinicia el servidor.
En Render arranca pero da error 500	Falta <code>ConnectionStrings__DefaultConnection</code> o es incorrecta. Ver <i>Logs</i> .
Los precios se ven con otro símbolo	Cambiar <code>"Tienda:Moneda"</code> (o <code>Tienda__Moneda</code> en <i>Render</i>).

Parte 14 — Preguntas típicas para defender el proyecto

¿Cómo se implementó la herencia Usuario → Empleado / Cliente del diagrama? Con herencia de clases en C# (`class Empleado : Usuario`) y estrategia TPT en EF Core: una tabla `Usuarios` con los datos comunes y `Empleados` / `Cientes` con los propios, cuya clave primaria es también clave foránea a `Usuarios`. Un usuario es exactamente una de las dos cosas.

¿Por qué Identity si el diagrama tiene su propia clase Usuario? Porque programar el login y el guardado de contraseñas a mano es inseguro. `Usuario` hereda de `IdentityUser<int>` (así el `idUsuario` es un `int` como en el diagrama) y las tablas se renombran para respetar los nombres. Identity aporta hash de contraseñas, cookies y roles ya probados.

¿Dónde está el rol del usuario? En `UsuarioRoles` (tabla estándar de Identity que relaciona `Usuarios` con `Roles`). La aplicación asigna exactamente un rol: `Admin`, `Empleado` o `Cliente`.

¿Por qué el carrito no es una tabla? Porque no está en el diagrama. Vive en la sesión del servidor (JSON) y cualquier visitante puede armarlo; al confirmar se convierte en `Venta` + `DetalleVenta`, que sí son tablas.

¿Para qué sirve Inventario si Producto ya tiene stock? `Producto.Stock` es el dato operativo (catálogo, ventas); `Inventario` es el registro de control con stock actual, mínimo y **última actualización**, que permite auditar cuándo cambió y listar lo que está bajo mínimo. Se mantienen sincronizados en `Producto.ActualizarStock()`.

¿Cómo se cumple "no permitir vender si no hay stock"? `Venta.AgregarDetalle` verifica `producto.Stock >= cantidad` y `Producto.ActualizarStock` lanza una excepción si quedaría negativo. Además el checkout vuelve a leer el stock de la base justo antes de confirmar, dentro de una transacción.

¿Y "no permitir compra con stock negativo"? Al cancelar una compra ya completada se descuenta lo que había ingresado; si eso dejara stock negativo, la excepción hace que la transacción se deshaga y se informa el error.

¿Por qué el checkout usa una transacción? Porque crea la venta, sus detalles y descuenta el stock de varios productos. O se hace todo o no se hace nada.

¿Por qué `DetalleVenta` guarda `PrecioUnitario` y `Subtotal`? Para que el historial refleje lo que se cobró en ese momento aunque el precio del producto cambie después (como una factura).

¿Qué diferencia hay entre una venta web y una de mostrador? La web la crea el cliente desde el carrito y queda *Pendiente* sin empleado; un empleado la completa (`cerrarVenta` registra quién). La de mostrador la registra un empleado eligiendo cliente y productos, y nace *Completada*.

¿Qué pasa con el stock en cada estado de venta? *Pendiente* y *Completada* lo tienen descontado; *Cancelada* y *Devuelta* lo reponen. Al cambiar de un grupo a otro, `CambiarEstado` ajusta el stock.

¿Cómo se guardan las contraseñas? Identity guarda un *hash* con sal en `Usuarios.PasswordHash`; la contraseña en texto plano nunca se almacena.

¿Cómo se validan los datos? Con Data Annotations en los modelos (`[Required]`, `[Range]`, `[RegularExpression]`...), que ASP.NET aplica en el servidor (`ModelState.IsValid`) y en el navegador

(jQuery Validation), más las reglas de contraseña de Identity y las verificaciones de stock en el código.

¿Qué hace `[ValidateAntiForgeryToken]`? Cada formulario lleva un token oculto único. Si otro sitio intentara enviar un formulario en nombre del usuario (CSRF), no tendría el token y se rechaza.

¿Qué es una migración? Un archivo generado por EF Core que crea o modifica las tablas a partir de los modelos. Reemplaza los scripts SQL manuales y permite versionar la base junto con el código.

¿Por qué el Session pooler de Supabase? La conexión directa es solo IPv6 en muchas redes y da timeout; el pooler tiene IPv4 y administra las conexiones.

¿Por qué `DateTime.UtcNow`? El servidor de Render está en otra zona horaria; se guarda en UTC y se convierte a la hora local solo al mostrar (`Formato.Fecha`).

Anexo A — README.md del proyecto

A continuación se reproduce íntegro el archivo [README.md](#) que acompaña al código:

FerreteriaApp — Sistema de Ferretería con ASP.NET Core MVC (.NET 10) y Supabase

Sistema web completo para una ferretería: catálogo de productos con imágenes, carrito de compras, pedidos y un panel de administración. Desarrollado con **ASP.NET Core MVC**, **Entity Framework Core**, **PostgreSQL en Supabase** y desplegado en **Render** con Docker.

¿Nuevo en ASP.NET? Leé [INSTRUCTIVO.md](#) (también en [INSTRUCTIVO.docx](#) / [INSTRUCTIVO.pdf](#)): explica todos los conceptos y el proyecto archivo por archivo, paso a paso.

1. Funcionalidades

El sistema implementa el **diagrama de clases UML de la ferretería** (ver sección 3.1): usuarios con roles y herencia Empleado/Cliente, categorías, proveedores, productos con inventario, ventas con detalle y forma de pago, y compras a proveedores con detalle.

Público (sin iniciar sesión)

- Página de inicio con categorías y productos destacados.
- Catálogo con **búsqueda**, **filtro por categoría** y **paginación**.
- Detalle de producto con productos relacionados.
- **Carrito** (se guarda en la sesión; no hace falta cuenta para armarlo).
- Registro (crea un **Cliente**) e inicio de sesión.

Cliente (rol **Cliente**)

- **Finalizar compra** eligiendo la forma de pago: se genera una **Venta** en estado *Pendiente* con sus **DetalleVenta** y se descuenta el stock (todo en una transacción).
- **Mis compras**: historial con estados y detalle. Puede cancelar mientras esté *Pendiente*.
- Editar su perfil y cambiar la contraseña.

Empleado (rol **Empleado**)

- **Panel general** con ventas, compras, stock bajo y productos más vendidos.
- **Registrar venta de mostrador** (elige cliente, forma de pago y productos): queda *Completada* a nombre del empleado.
- **Gestionar ventas**: filtrar por estado y cambiarlo (*Pendiente* → *Completada* → *Devuelta* / *Cancelada*). Cancelar o devolver repone el stock.
- **Registrar compras a proveedores** con detalle; al completarlas ingresa la mercadería (sube el stock y actualiza el precio de compra). No se permite cancelar una compra si dejaría stock negativo.
- **Inventario**: stock actual, mínimo, última actualización, filtro "bajo mínimo" y ajuste manual.
- Crear y editar **productos** (con subida de imagen).

Administrador (rol **Admin**)

- Todo lo del empleado, más: **categorías, proveedores, formas de pago**, eliminar productos, y **usuarios**: alta/edición de empleados, rol Admin/Empleado, activar/desactivar cuentas y tipo de cliente (Regular/Mayorista).

Validaciones (según el diagrama)

- Email único y con formato válido. Contraseña de **mínimo 8 caracteres con mayúsculas, minúsculas y números**. Teléfono solo números de **8 a 15 dígitos**. Estado no nulo.
- Producto: nombre obligatorio (máx. 100), precios de venta y compra > 0, stock y stock mínimo ≥ 0, categoría y proveedor obligatorios.
- Venta/Compra: fecha no nula, total > 0, cantidad > 0, precio unitario > 0, no vender sin stock, no dejar stock negativo.

Datos iniciales (seed automático)

La primera vez que arranca, la app crea sola los roles, el administrador, un vendedor de ejemplo, 4 formas de pago, 8 categorías, 6 proveedores, los **43 productos** con sus imágenes y su **inventario** inicial.

2. Tecnologías

Componente	Tecnología
Framework web	ASP.NET Core MVC — .NET 10 (LTS)
Acceso a datos	Entity Framework Core 10 + Npgsql
Base de datos	PostgreSQL (Supabase)
Autenticación	ASP.NET Core Identity con claves int (cookies, roles Admin , Empleado , Cliente)
Carrito	Sesión de ASP.NET Core (no es una tabla)
Front-end	Razor Views + Bootstrap 5 + Bootstrap Icons
Deploy	Docker en Render

3. Estructura del proyecto

```
FerreteriaApp/  
├── Controllers/  
│   ├── HomeController.cs      Inicio y página de error  
│   ├── AccountController.cs   Login, registro (Cliente), logout, perfil, cambiar contraseña  
│   ├── ProductosController.cs Catálogo público + CRUD de productos (personal)  
│   ├── CategoriasController.cs CRUD de categorías (Admin)  
│   ├── ProveedoresController.cs CRUD de proveedores (Admin)  
│   ├── FormasPagoController.cs CRUD de formas de pago (Admin)  
│   ├── CarritoController.cs   Carrito en sesión (agregar, actualizar, quitar, vaciar)  
│   ├── VentasController.cs    Checkout web, mis compras, venta de mostrador, gestión de estados  
│   ├── ComprasController.cs   Compras a proveedores y su ingreso al stock  
│   ├── InventarioController.cs Inventario y ajustes manuales  
│   └── AdminController.cs     Panel general, usuarios y empleados  
├── Models/  
│   ├── Usuario.cs            Base de Identity (int) + nombre, apellido, estado, fechaCreacion  
│   ├── Rol.cs                 Rol de Identity (int) + descripción  
│   └── Empleado.cs / Cliente.cs Heredan de Usuario (tablas Empleados y Clientes)
```

- └─ Categoria.cs, Proveedor.cs, Producto.cs, Inventario.cs, FormaPago.cs
- └─ Venta.cs (+ enum EstadoVenta), DetalleVenta.cs
- └─ Compra.cs (+ enum EstadoCompra), DetalleCompra.cs
- └─ CarritoItem.cs Ítem del carrito (en sesión)
- └─ *ViewModel.cs Formularios (login, registro, perfil, checkout, venta, compra, empleado)
- └─ Data/
 - └─ ApplicationDbContext.cs Único DbContext (Identity + tablas de la ferretería)
 - └─ DbInitializer.cs Seed: roles, admin, vendedor, formas de pago, categorías, proveedores, productos, inventario
- └─ Helpers/
 - └─ Formato.cs Precios, fechas y colores de estado
 - └─ CarritoSesion.cs Leer/guardar el carrito en la sesión
 - └─ UsuarioExtensions.cs Id numérico del usuario logueado, EsPersonal()
 - └─ SpanishIdentityErrorDescriber.cs Mensajes de Identity en español
- └─ Migrations/ Migraciones de EF Core (ya generadas)
- └─ Views/ Vistas Razor (Home, Account, Productos, Categorías, Proveedores, FormasPago, Carrito, Ventas, Compras, Inventario, Admin, Shared)
- └─ wwwroot/
 - └─ css/site.css Estilos propios
 - └─ images/productos/ Imágenes de los productos
- └─ appsettings.json Configuración (connection string, nombre de la tienda, admin)
- └─ Program.cs
- └─ Dockerfile Para el deploy en Render
- └─ render.yaml Blueprint de Render

3.1 Del diagrama de clases a las tablas

Clase del diagrama	Tabla	Notas
Usuario	Usuarios	Es el usuario de ASP.NET Core Identity con clave int (IdUsuario). Identity aporta Email , PasswordHash (la contraseña nunca se guarda en texto plano) y las columnas de seguridad. Se agregan Nombre , Apellido , Estado , FechaCreacion .
Rol	Roles	Rol de Identity (IdRol , Nombre , Descripcion). La relación Usuario–Rol queda en UsuarioRoles ; la app asigna un rol por usuario (Admin , Empleado o Cliente).
Empleado hereda Usuario	Empleados	Herencia TPT (una tabla por tipo): IdEmpleado es PK y a la vez FK a Usuarios.IdUsuario . Campos: Cargo , Telefono , Direccion .
Cliente hereda Usuario	Clientes	Igual que Empleado: IdCliente = PK/FK. Campos: Telefono , Direccion , TipoCliente .
Categoria	Categorías	IdCategoria , Nombre , Descripcion , Estado .
Proveedor	Proveedores	IdProveedor , Nombre , Ruc , Telefono , Direccion , Estado .
Producto	Productos	IdProducto , Nombre , Descripcion , PrecioVenta , PrecioCompra , Stock , StockMinimo , Estado , IdCategoria , IdProveedor + ImagenUrl (campo extra para la foto).
Inventario	Inventarios	IdInventario , IdProducto , StockActual , StockMinimo , UltimaActualizacion . Uno por producto, se actualiza en cada venta, compra o ajuste.
FormaPago	FormasPago	IdFormaPago , Nombre , Descripcion , Estado .
Venta	Ventas	IdVenta , Fecha , IdCliente , IdEmpleado (nulo hasta que un empleado completa una venta web), IdFormaPago , Total , Estado (enum EstadoVenta), Observaciones .

Clase del diagrama	Tabla	Notas
DetalleVenta	DetalleVentas	IdDetalleVenta, IdVenta, IdProducto, Cantidad, PrecioUnitario, Subtotal.
Compra	Compras	IdCompra, Fecha, IdProveedor, IdEmpleado, Total, Estado (enum EstadoCompra), Observaciones.
DetalleCompra	DetalleCompras	IdDetalleCompra, IdCompra, IdProducto, Cantidad, PrecioUnitario, Subtotal.
EstadoVenta, EstadoCompra	(enum)	Se guardan como número: Pendiente=0, Completada=1, Cancelada=2, Devuelta=3.

Los **métodos** del diagrama están implementados en las clases: `Producto.ActualizarStock()` / `VerificarStock()`, `Inventario.ActualizarStock()` / `VerificarStock()`, `Venta.CalcularTotal()` / `AgregarDetalle()` / `ValidarDatos()` / `CerrarVenta()`, `DetalleVenta.CalcularSubtotal()`, `Compra.CalcularTotal()` / `AgregarDetalle()` / `ValidarDatos()`, `DetalleCompra.CalcularSubtotal()`. Los que dependen del usuario logueado (`login`, `cambiarPassword`, `realizarCompra`, `verHistorial`, `registrarVenta`, `registrarCompra`, `gestionarInventario`) están en los controladores.

El **carrito** no es una tabla: se guarda en la sesión del servidor y al confirmar se convierte en una **Venta**. Así la base de datos queda exactamente como el diagrama.

4. Prerrequisitos

- **.NET 10 SDK (LTS)**. Verificar con `dotnet --version` (debe empezar con 10.).
- Visual Studio 2022 (17.14+) o Visual Studio Code con la extensión *C# Dev Kit*.
- Una cuenta y un proyecto creado en [Supabase](#).
- Una cuenta en [GitHub](#) y otra en [Render](#) para el deploy.
- (Opcional) La herramienta de EF Core para crear migraciones nuevas:

```
dotnet tool install --global dotnet-ef
```

5. Obtener el connection string de Supabase

1. En el panel de Supabase: **Project Settings → Database** (o el botón **Connect** arriba).
2. En *Connection string* elegir la pestaña **Session pooler** (no *Direct connection*: el pooler tiene salida IPv4 y evita el error de timeout por IPv6).
3. Copiar host, puerto (normalmente 5432) y usuario (formato `postgres.xxxxxxxxxxxxx`).
4. Armar la cadena así:

```
Host=aws-0-<region>.pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.<tu-project-ref>;Password=<TU_PASSWORD>;SSL Mode=Require;Trust Server Certificate=true
```

Si olvidaste la contraseña de la base de datos, se puede resetear en *Project Settings → Database → Reset database password*.

6. Ejecutar en la computadora (desarrollo)

1. Clonar o copiar el proyecto y entrar a la carpeta:

```
cd FerreteriaApp
```

2. Poner el connection string. Hay dos opciones: - **Opción A (simple)**: reemplazar el valor de `DefaultConnection` en `appsettings.json`. - **Opción B (recomendada)**: crear el archivo `appsettings.Development.json` (está en `.gitignore`, así la contraseña no se sube a GitHub):

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=aws-0-
<region>.pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.<tu-project-
ref>;Password=<TU_PASSWORD>;SSL Mode=Require;Trust Server Certificate=true"
  }
}
```

3. Restaurar paquetes y ejecutar:

```
dotnet restore
dotnet run
```

4. Abrir la URL que muestra la consola (por ejemplo <http://localhost:5047>).

No hace falta correr ningún script SQL ni `dotnet ef database update`: al arrancar, `Program.cs` aplica las migraciones automáticamente (crea todas las tablas de Identity y de la ferretería en Supabase) y carga los datos iniciales.

Usuarios iniciales

Rol	Email	Contraseña
Admin (empleado, cargo Administrador)	juan@gmail.com	Juan1234
Empleado (vendedor de ejemplo)	vendedor@ferreteria.com	Vendedor123

El administrador se define en `appsettings.json` → sección "`Admin`" (cambiarlo antes de entregar/desplegar). Las contraseñas deben cumplir la regla del diagrama: 8 caracteres con mayúscula, minúscula y número. Los usuarios que se registran desde la web son **Cientes**; los empleados los crea el administrador en **Gestión** → **Usuarios y empleados**.

7. Migraciones (solo si se cambian los modelos)

La migración inicial ya está en la carpeta `Migrations/`. Si más adelante se agrega o modifica un campo en algún modelo, se genera una nueva migración y la app la aplica sola al arrancar:

```
dotnet ef migrations add NombreDelCambio
```

Para aplicarla a mano (opcional):

```
dotnet ef database update
```

8. Deploy en Render

8.1 Subir el proyecto a GitHub

Desde la carpeta `FerreteriaApp` (verificar antes que `appsettings.json` **no** tenga la contraseña real, o que esté en `appsettings.Development.json` que se ignora):

```
git init
git add .
git commit -m "Sistema de ferretería con ASP.NET Core MVC y Supabase"
git branch -M main
git remote add origin https://github.com/<tu-usuario>/<tu-repo>.git
git push -u origin main
```

8.2 Crear el servicio en Render

Opción A — Blueprint (usa `render.yaml`): 1. En Render: **New → Blueprint** y conectar el repositorio de GitHub. 2. Render detecta `render.yaml` y crea el servicio `ferreteria-app` (plan Free, runtime Docker). 3. Cuando pida las variables de entorno, cargar: - `ConnectionStrings__DefaultConnection` = el connection string de Supabase (paso 5). - `Admin__Password` = contraseña que se quiera para el administrador (opcional).

Opción B — Manual: 1. **New → Web Service**, conectar el repositorio. 2. *Language / Runtime:* **Docker**. Plan: Free. 3. En **Environment Variables** agregar:

Key	Value
<code>ConnectionStrings__DefaultConnection</code>	<code>Host=aws-0-...pooler.supabase.com;Port=5432;Database=postgres;Username=postgres.xxx;Password=...;SSL Mode=Require;Trust Server Certificate=true</code>
<code>ASPNETCORE_ENVIRONMENT</code>	<code>Production</code>
<code>Admin__Password</code>	(opcional)

4. **Create Web Service.** Render construye la imagen con el `Dockerfile` y publica la app en <https://ferreteria-app.onrender.com> (o el nombre elegido).

En .NET, el doble guion bajo `__` en una variable de entorno equivale a los dos puntos : de `appsettings.json`. Por eso `ConnectionStrings__DefaultConnection` reemplaza a `ConnectionStrings:DefaultConnection`.

Cada `git push` a `main` vuelve a desplegar automáticamente.

8.3 Notas sobre Render (plan Free)

- El servicio se "duerme" tras 15 minutos sin visitas; la primera visita después tarda ~30-60 s.
 - El disco es **efímero**: las imágenes que suba el administrador desde la app se pierden en cada nuevo deploy. Las 43 imágenes iniciales sí persisten porque están en el repositorio. Para que las imágenes nuevas persistan, se puede pegar la **URL de una imagen** (campo *URL de imagen*) en lugar de subir el archivo, por ejemplo una imagen alojada en Supabase Storage.
-

9. Personalización

En `appsettings.json`:

```
"Tienda": {  
  "Nombre": "Ferretería Central",  
  "Moneda": "Bs",  
  "ZonaHoraria": "America/La_Paz"  
}
```

- **Nombre**: se muestra en la barra, la portada y el pie de página.
- **Moneda**: símbolo con el que se muestran los precios (Bs, S/, \$, etc.).
- **ZonaHoraria**: zona horaria para mostrar las fechas de los pedidos (se guardan en UTC).

Dirección, teléfono y horario del pie de página: `Views/Shared/_Layout.cshtml`.

10. Solución de problemas frecuentes

Error / síntoma	Causa y solución
<i>Timeout</i> / no conecta a la base de datos	Se está usando el host de conexión directa (<code>db.xxx.supabase.co</code>), que es IPv6. Usar el connection string del Session pooler (paso 5).
<code>password authentication failed</code>	Contraseña incorrecta en el connection string. Resetearla en Supabase → <i>Database</i> → <i>Reset database password</i> .
<code>EntityFrameworkCore does not exist, IdentityDbContext<> could not be found</code>	Los paquetes NuGet no se restauraron. Borrar <code>bin/</code> y <code>obj/</code> , correr <code>dotnet restore</code> y <code>dotnet build</code> .
<code>dotnet ef</code> no se reconoce como comando	Instalar la herramienta: <code>dotnet tool install --global dotnet-ef</code> y reabrir la terminal.
<code>The view '...' was not found</code>	Falta el <code>.cshtml</code> correspondiente a esa acción; revisar el nombre exacto en <code>Views/<Controlador>/</code> .
400 al enviar un formulario	Falta el token antifalsificación: el <code><form></code> debe tener <code>method="post"</code> y usar los tag helpers (<code>asp-action</code> , <code>asp-for</code>).
<code>relation "Products" already exists</code> u otras tablas viejas en Supabase	La base tenía la versión anterior del modelo. Borrar las tablas viejas (o crear un proyecto nuevo en Supabase) y volver a arrancar: las migraciones crean el esquema nuevo.
En Render el deploy falla en <code>dotnet restore</code>	Revisar que el <code>Dockerfile</code> esté en la raíz del repositorio junto al <code>.csproj</code> .
En Render arranca pero da error 500	Falta la variable <code>ConnectionStrings__DefaultConnection</code> o tiene un valor incorrecto. Ver <i>Logs</i> en el panel de Render.
Los precios se ven con símbolo distinto	Cambiar <code>"Tienda:Moneda"</code> en <code>appsettings.json</code> (o la variable de entorno <code>Tienda_Moneda</code> en Render).